

**PENGEMBANGAN SISTEM PELAPORAN MASALAH
LINGKUNGAN DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK
MENDUKUNG *SMART CITY***

Topik: *Application Development dan Intelligence System*

Oleh

Brian Theodore 2602080030

Moh. Khoirul Umam Al Amin 2602198472

Andrea Octaviani 2602075895



**Computer Science
Computer Science Study Program
School of Computer Science
Universitas Bina Nusantara
Bandung
2026**

**PENGEMBANGAN SISTEM PELAPORAN MASALAH
LINGKUNGAN DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK
MENDUKUNG *SMART CITY***

SKRIPSI

Diajukan sebagai salah satu syarat
untuk gelar kesarjanaan pada
Program Studi Teknik Informatika
Jenjang Pendidikan Strata-1

Oleh

Brian Theodore	2602080030
Moh. Khoirul Umam Al Amin	2602198472
Andrea Octaviani	2602075895



**Computer Science
Computer Science Study Program
School of Computer Science
Universitas Bina Nusantara
Bandung
2026**

LEMBAR PERNYATAAN ORISINALITAS

Pernyataan Kesiapan Skripsi untuk Ujian Pendadaran

Kami, Brian Theodore

Moh. Khoirul Umam Al Amin

Andrea Octaviani

Dengan ini menyatakan bahwa Skripsi yang berjudul:

**PENGEMBANGAN SISTEM PELAPORAN MASALAH LINGKUNGAN
DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK MENDUKUNG SMART
CITY
*DEVELOPMENT OF AN ENVIRONMENTAL PROBLEM REPORTING
SYSTEM
INTEGRATING IMAGE AND TEXT DATA TO SUPPORT SMART CITY
INITIATIVES***

adalah benar hasil karya kami dan belum pernah diajukan sebagai karya ilmiah, sebagian atau seluruhnya, atas nama kami atau pihak lain.



Brian Theodore

2602080030



Moh. Khoirul Umam Al Amin

2602198472



Andrea Octaviani

2602075895

Disetujui oleh Pembimbing

Saya setuju Skripsi tersebut layak diajukan untuk Ujian Pendadaran

Bandung, 16 Desember 2025

Budi Juarto, S.T., M.Kom.

D6670

Universitas Bina Nusantara

Computer Science Program
Computer Science Study Program
School of Computer Science
Skripsi Sarjana Teknik Informatika

**DEVELOPMENT OF AN ENVIRONMENTAL PROBLEM REPORTING
SYSTEM
INTEGRATING IMAGE DATA TO SUPPORT SMART CITY INITIATIVES**

Brian Theodore	2602080030
Moh. Khoirul Umam Al Amin	2602198472
Andrea Octaviani	2602075895

ABSTRACT

The slow manual triage process in urban environmental complaint systems, in which a single incoming report can take nearly two hours to be routed to the responsible government agency, represents a critical bottleneck in smart city governance. Existing reporting platforms also fail to exploit the complementary signals that citizens already provide: a photograph of the incident and a free-text description of what happened. This research proposes a multimodal classification system that fuses both modalities, combining a frozen DINOv3-large image encoder with a frozen multilingual-E5-large text encoder to produce dense embeddings that are concatenated through early fusion and classified by a CatBoost head into Jakarta CRM (Citizen Relationship Management) agency categories (dinas). The trained pipeline is exposed through a FastAPI inference server with a multipart /predict

endpoint that consumes both an image file and a Bahasa Indonesia narrative (laporan), and is consumed by a Flutter Android application that captures the photograph, GPS coordinates, and report text on the user device. The frozen pre-trained encoders are exported to ONNX (opset 17) and reused as feature extractors so that the trainable surface is reduced to the CatBoost head, which keeps the training cost low and the embeddings semantically robust without domain-specific fine-tuning. A deterministic rule-based routing engine then maps the predicted dinas to four operational clusters that correspond to relevant Indonesian local agencies (Dinas Lingkungan Hidup, Dinas Bina Marga, Dinas Perhubungan, and BPBD), enabling near-instant report distribution without human intervention. Evaluation is reported in terms of macro F1-score, balanced accuracy, and a confusion matrix that traces the residual error patterns across visually similar categories. The resulting system demonstrates that early fusion of strong self-supervised image features with multilingual text embeddings is a practical recipe for low-resource public-service classification, and that the separation between a heavy server-side inference path and a light Flutter client preserves user experience while keeping the model upgradeable from a single deployment surface.

Keywords: *smart city, multimodal classification, early fusion, DINOv3, multilingual E5, CatBoost, FastAPI, Flutter, Jakarta CRM, ONNX Runtime.*

Universitas Bina Nusantara

**Computer Science Program
Computer Science Study Program
School of Computer Science
Skripsi Sarjana Teknik Informatika**

**PENGEMBANGAN SISTEM PELAPORAN MASALAH LINGKUNGAN
DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK MENDUKUNG *SMART
CITY***

Brian Theodore	2602080030
Moh. Khoirul Umam Al Amin	2602198472
Andrea Octaviani	2602075895

ABSTRAK

Proses triase manual pada sistem pelaporan masalah lingkungan perkotaan, yaitu kondisi di mana satu laporan yang masuk dapat memerlukan waktu hampir dua jam untuk diteruskan ke instansi pemerintah yang bertanggung jawab, merupakan hambatan kritis dalam tata kelola *smart city*. Selain itu, platform pelaporan yang ada belum memanfaatkan secara penuh dua sinyal komplementer yang telah disediakan warga, yaitu foto kejadian dan narasi laporan dalam Bahasa Indonesia. Penelitian ini mengusulkan sistem klasifikasi multimodal yang menggabungkan kedua modalitas tersebut. Sebuah *encoder* citra DINOv3 *large* yang dibekukan (*frozen*) dan *encoder* teks *multilingual-E5 large* yang juga dibekukan menghasilkan vektor representasi (*embedding*) padat. Kedua *embedding* digabungkan melalui skema fusi awal (*early fusion*) dengan operasi konkatenasi, kemudian diklasifikasikan oleh algoritma CatBoost ke dalam kategori dinas pada Jakarta CRM (*Citizen Relationship*

Management). *Pipeline* pelatihan diekspos melalui server inferensi FastAPI dengan *endpoint* /predict berbasis *multipart* yang menerima berkas gambar dan narasi laporan, dan dikonsumsi oleh aplikasi Android berbasis Flutter yang menangkap foto, koordinat GPS, dan teks laporan langsung di perangkat pengguna. Kedua *encoder* pra-latih diekspor ke format ONNX (*opset* 17) dan dipakai sebagai *feature extractor*, sehingga permukaan parameter yang dilatih hanya tersisa pada algoritma CatBoost. Pendekatan ini menekan biaya pelatihan sekaligus mempertahankan ketahanan semantik *embedding* tanpa perlu *fine-tuning* berskala besar pada domain perkotaan. Mesin perutean berbasis aturan secara deterministik memetakan prediksi dinas ke empat kluster operasional yang merujuk pada instansi pemerintah daerah Indonesia (Dinas Lingkungan Hidup, Dinas Bina Marga, Dinas Perhubungan, dan BPBD) sehingga laporan dapat didistribusikan hampir instan tanpa intervensi manusia. Evaluasi dilaporkan menggunakan *macro F1-score*, *balanced accuracy*, dan *confusion matrix* untuk memetakan pola kesalahan antar kategori yang secara visual saling menyerupai. Hasil rancangan menunjukkan bahwa fusi awal antara fitur citra hasil pelatihan mandiri (*self-supervised*) yang kuat dengan *embedding* teks multibahasa merupakan resep praktis untuk klasifikasi layanan publik berdaya rendah, dan bahwa pemisahan antara jalur inferensi berat di sisi server dengan klien Flutter yang ringan menjaga pengalaman pengguna sekaligus memudahkan pembaruan model dari satu permukaan *deployment*.

Kata Kunci: *smart city*, klasifikasi multimodal, *early fusion*, DINOv3, *multilingual E5*, CatBoost, FastAPI, Flutter, Jakarta CRM, *ONNX Runtime*.

KATA PENGANTAR

Puji dan syukur kepada Tuhan Yang Maha Esa, atas berkat dan rahmatNya sehingga penulis dapat menyelesaikan laporan Skripsi ini dengan baik dan tepat waktu. Penulis juga tidak lupa berterimakasih kepada pihak yang telah membantu penulis dalam memberikan bimbingan, dukungan, dan doa. Oleh karena itu, pada kesempatan ini penulis ingin mengucapkan terima kasih kepada:

1. Ibu Dr. Nelly, S.Kom., M.M., selaku Rektor Binus University.
2. Bapak Dr. Ir. Derwin Suhartono, S.Kom., MTI, selaku Dekan of School of Computer Science and Head of Computer Science Binus Bandung.
3. Bapak Budi Juarto, S.T., M.Kom., selaku dosen pembimbing yang telah banyak meluangkan waktu untuk memberikan bimbingan, petunjuk, dukungan, dan saran kepada penulis dalam menyelesaikan laporan Skripsi ini.
4. Segenap Dosen Fakultas School of Computer Science yang telah mendidik dan memberikan ilmu selama kuliah dan seluruh staf yang selalu sabar melayani segala administrasi selama proses penelitian ini.
5. Orang tua dan saudara-saudari yang senantiasa mendampingi dengan doa, semangat, dan dukungan materi, sehingga penulis mampu melalui proses pendidikan dan menyelesaikan skripsi ini dengan baik.
6. Semua pihak yang telah membantu dan mendukung penulis selama pembuatan laporan Skripsi ini yang tidak dapat penulis sebutkan satu per satu.

Karena keterbatasan pengetahuan maupun pengalaman penulis, maka penulis yakin masih banyak kekurangan dalam laporan Skripsi ini, Oleh karena itu penulis sangat mengharapkan saran dan kritik yang membangun dari pembaca demi kesempurnaan dokumen skripsi ini.

Bandung, 16 Desember 2025

Penulis

DAFTAR ISI

HALAMAN JUDUL	i
HALAMAN SAMPUL	ii
LEMBAR PERNYATAAN ORISINALITAS	iii
ABSTRAK	v
KATA PENGANTAR	ix
DAFTAR ISI	xi
DAFTAR TABEL	xx
DAFTAR GAMBAR	xxii
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	6
1.3 Ruang Lingkup	7
1.4 Tujuan dan Manfaat Penelitian	9
1.4.1 Tujuan Penelitian	9
1.4.2 Manfaat Penelitian	10
1.5 Metode Penelitian	11
1.5.1 Pengumpulan dan Persiapan Data	11
1.5.2 Ekstraksi Embedding dengan Encoder Beku	12
1.5.3 Pelatihan Kepala CatBoost dan Ekspor Model	12

1.5.4	Pengembangan Server FastAPI dan Aplikasi Flutter	13
1.6	Sistematika Penulisan	13
BAB 2	TINJAUAN PUSTAKA	15
2.1	<i>Smart City</i> dan <i>Smart Environment</i>	15
2.2	Sistem Pelaporan dan Partisipasi Publik	16
2.3	<i>Web Scraping</i> dan Akuisisi Data	17
2.3.1	Definisi dan Latar Belakang	17
2.4	Kecerdasan Buatan (<i>Artificial Intelligence</i>)	18
2.5	<i>Machine Learning</i>	18
2.5.1	Definisi Formal	18
2.5.2	Klasifikasi Multi-kelas	18
2.5.3	<i>Bias-Variance Trade-off</i> dan Regularisasi	19
2.5.4	<i>Gradient Boosting</i> dan CatBoost	19
2.6	<i>Deep Learning</i>	20
2.6.1	Perceptron, Multilayer Perceptron, dan Backpropagation	20
2.7	Pemrosesan Data Multimodal: Ringkasan	21
2.8	Ekstraksi Fitur (<i>Feature Extraction</i>)	21
2.8.1	Definisi	21
2.8.2	Fitur <i>Hand-Crafted</i>	22
2.8.3	Fitur Hasil Pelatihan dan <i>Transfer Learning</i>	22
2.8.4	Ruang <i>Embedding</i> dan Sifat Geometrisnya	22
2.9	Reduksi Dimensi: Ringkasan	23
2.10	Mekanisme <i>Attention</i>	23
2.10.1	Motivasi: Keterbatasan RNN pada Konteks Panjang	23
2.10.2	<i>Scaled Dot-Product Attention</i>	24
2.10.3	<i>Multi-Head Attention</i>	24
2.10.4	<i>Self-Attention</i> versus <i>Cross-Attention</i>	24
2.10.5	Kompleksitas dan Implikasi Skalabilitas	25
2.11	Arsitektur <i>Transformer</i>	25

2.11.1	<i>Encoder-Decoder</i>	25
2.11.2	<i>Positional Encoding</i> , Normalisasi, dan <i>Feed-Forward</i>	25
2.11.3	<i>Vision Transformer</i> (ViT)	26
2.12	Arsitektur <i>Backbone</i> Visual Modern	26
2.12.1	DINOv3	26
2.12.1.1	Garis Keturunan: DINO, DINOv2, DINOv3	26
2.12.1.2	Mekanisme <i>Self-Distillation Tanpa Label</i>	27
2.12.1.3	Token CLS dan Token Patch	28
2.12.1.4	Sifat <i>Emergent</i> pada Skala Besar	28
2.13	Model <i>Encoder</i> Teks	29
2.13.1	Keluarga E5 dan <i>multilingual-E5</i>	30
2.13.2	Perbandingan Singkat	31
2.14	Fusi Multimodal	31
2.14.1	Taksonomi Skema Fusi	31
2.14.2	Bentuk-Bentuk Fusi	32
2.14.3	Pertimbangan Kalibrasi Skala	32
2.15	ONNX dan Inferensi Lintas Kerangka	33
2.15.1	Spesifikasi ONNX	33
2.15.2	ONNX Runtime	33
2.15.3	Validasi Paritas Numerik	34
2.16	Inferensi <i>On-device</i> versus <i>On-server</i>	34
2.16.1	Definisi	34
2.16.2	Trade-off Antara Kedua Pendekatan	35
2.16.3	Teknik Optimisasi untuk <i>On-device</i>	35
2.16.4	Teknik Optimisasi untuk <i>On-server</i>	36
2.16.5	Implikasi pada Penelitian Ini	36
2.17	<i>Unified Modeling Language</i> (UML) pada Pengembangan Aplikasi .	37
2.17.1	Definisi dan Sejarah	37
2.17.2	Peran UML pada Pengembangan Aplikasi Mobile	37
2.18	Aplikasi <i>Mobile</i> : Android dan iOS	38

2.19	Flutter dan <i>Stack</i> Pengembangan Mobile	38
2.19.1	Flutter sebagai <i>Framework</i>	38
2.20	Supabase sebagai <i>Backend</i> Basis Data	39
2.20.1	Definisi dan Latar Belakang	39
2.20.2	<i>Row Level Security</i> (RLS)	39
2.21	FastAPI dan <i>Web Framework</i> untuk <i>Serving</i>	40
2.22	Docker dan Kontainerisasi	40
2.23	Metrik Evaluasi Klasifikasi Multi-Kelas	41
2.23.1	<i>Macro F1-Score</i>	41
2.23.2	Metrik Pendukung	42
2.24	Teknik Pelatihan Model	42
2.24.1	<i>Transfer Learning</i>	42
2.24.2	Validasi dan Pemilihan Model	43
2.25	Penelitian Terkait	43
2.25.1	Radford et al. (2021): CLIP, Pretraining Kontrasif Citra dan Teks	43
2.25.1.1	Gambaran Umum	43
2.25.1.2	Analisis dan Metode	44
2.25.1.3	Kesimpulan	44
2.25.2	Audebert et al. (2019): Multimodal Deep Networks untuk Klasifikasi Dokumen	45
2.25.2.1	Gambaran Umum	45
2.25.2.2	Analisis dan Metode	45
2.25.2.3	Kesimpulan	45
2.25.3	Ofli, Alam, dan Imran (2020): Multimodal Deep Learning untuk Disaster Response	46
2.25.3.1	Gambaran Umum	46
2.25.3.2	Analisis dan Metode	46
2.25.3.3	Kesimpulan	46

2.25.4	Alam, Ofli, dan Imran (2018): CrisisMMD, Dataset Multi-modal Bencana	47
2.25.4.1	Gambaran Umum	47
2.25.4.2	Analisis dan Metode	47
2.25.4.3	Kesimpulan	47
2.25.5	Koshy dan Elango (2023): Multimodal Tweet Classification dengan Transformer Bidirectional Attention	48
2.25.5.1	Gambaran Umum	48
2.25.5.2	Analisis dan Metode	48
2.25.5.3	Kesimpulan	48
2.25.6	Ringkasan Perbandingan	49
BAB 3	METODE PENELITIAN	51
3.1	Diagram Alir Kerangka Berpikir	51
3.1.1	Kerangka Berpikir Tahap Akuisisi Data	53
3.1.2	Kerangka Berpikir Tahap Pelatihan Model	53
3.1.3	Kerangka Berpikir Tahap Deployment	53
3.2	Analisis Kebutuhan	53
3.2.1	Analisis User	53
3.2.1.1	Latar Belakang Institusi	58
3.2.2	Analisis Aplikasi Sejenis	58
3.2.3	Rumusan dan Solusi Kebutuhan	58
3.3	Akuisisi dan Persiapan Dataset	59
3.3.1	Sumber Data CRM Jakarta	59
3.3.2	Struktur Data Multimodal	60
3.3.3	Metadata Tambahan	61
3.3.4	Web Scraping	61
3.3.5	Pembersihan dan Normalisasi	62
3.3.6	Pemetaan ke Sembilan Instansi Target	63
3.3.6.0.1	Definisi terminologi.	63

3.3.7	Pembagian Data Stratifikasi	64
3.3.8	Bobot Kelas	65
3.4	Lingkungan Komputasi: Pelatihan di Cloud via RunPod	65
3.4.1	Justifikasi Cloud Compute	65
3.4.2	Konfigurasi Pod RunPod	66
3.4.3	Workflow Pelatihan dan Manajemen Lingkungan	66
3.5	Pemrosesan, Ekstraksi <i>Embedding</i> , dan Fusi Dini	67
3.6	Algoritma Klasifikasi: CatBoost dengan PGS	68
3.6.1	Justifikasi Pemilihan CatBoost sebagai <i>Classifier Head</i> . . .	68
3.6.2	Hyperparameter	68
3.6.3	<i>Posterior Gaussian Sampling</i> (PGS)	69
3.6.4	Pelatihan dengan Validation Set dan Early Stopping	70
3.6.5	Pemilihan Model Terbaik	70
3.7	Eksperimen Komparatif (Ablasi)	70
3.7.1	Ablasi Matriks Multimodal	70
3.7.2	Ablasi PGS On/Off	71
3.7.3	Tabel Rancangan Eksperimen	71
3.8	Metrik Evaluasi	71
3.9	Pipeline Ekspor ONNX	73
3.9.1	Ekspor Encoder Citra	73
3.9.2	Ekspor Encoder Teks	73
3.9.3	Ekspor CatBoost	74
3.9.4	Prosedur Validasi Paritas Numerik	74
3.10	Perancangan Sistem Server (FastAPI)	75
3.10.1	Arsitektur Server	75
3.10.2	Endpoint <code>/health</code> dan <code>/predict</code>	75
3.10.3	Skema Request dan Response	75
3.10.4	Output Inferensi dan Confidence Level	77
3.10.5	Dockerization	77
3.11	Perancangan Skema Basis Data (Supabase)	77

3.11.1	Ringkasan Tabel dan Kontrol Akses	78
3.11.2	Bukti Penyelesaian Laporan	78
3.12	Perancangan Sistem Aplikasi Mobile (Flutter)	80
3.12.1	Use Case Diagram	80
3.12.2	Use Case Description	81
3.12.3	Activity Diagram	81
3.12.4	Class Diagram	82
3.12.5	Sequence Diagram	82
3.12.6	Modul dan Layanan Flutter	82
3.13	Mekanisme Perutean Instansi	85
3.13.1	Keluaran Model Tetap Sembilan Instansi	85
3.13.2	Routing Instansi Terdekat Berbasis Lokasi	85
3.13.3	Kasus Confidence Rendah dan Fallback	86
3.14	Perancangan Layar Aplikasi	86
3.14.1	Layar Splash dan Onboarding	86
3.14.2	Layar Buat Laporan	86
3.14.3	Layar Tinjauan Hasil AI	87
3.14.4	Layar Rekomendasi Instansi	87
3.14.5	Layar Riwayat Laporan	87
3.14.6	Layar Peta Laporan	87
3.14.7	Layar Profil dan Pengaturan	88
3.15	Skenario Pengujian dan Metrik Evaluasi	88
3.15.1	Pembagian Data	88
3.15.2	Metrik Evaluasi	88
3.15.3	Skenario Ablasi	89
3.15.4	Validasi Paritas ONNX	89
3.15.5	Pengujian Aplikasi Flutter	89
3.15.6	Catatan tentang Pemilihan Algoritma	90
BAB 4	HASIL DAN PEMBAHASAN	91

4.1	Lingkungan Pengujian	91
4.2	Persiapan dan Eksplorasi Data	91
4.3	Hasil Eksperimen Utama	93
4.3.1	Metrik Per-Kelas	94
4.4	Hasil Ablasi Modalitas	95
4.5	Hasil Ablasi Kalibrasi (PGS)	96
4.6	Hasil Ablasi Pasangan Encoder	97
4.7	Validasi Paritas ONNX	98
4.8	Hasil Implementasi Aplikasi Flutter	99
4.8.1	Skema Basis Data Supabase	99
4.8.2	Alur Pelaporan oleh Warga	100
4.8.3	Alur Verifikasi oleh Operator Instansi	100
4.8.4	Alur Moderasi	100
4.8.5	Demo Perutean Instansi Terdekat	102
4.9	Pembahasan dan Analisis Kesalahan	102
4.9.1	Pasangan Kelas yang Sering Tertukar	102
4.9.2	Sumber Ambiguitas	104
4.9.3	Keterbatasan Dataset	104
4.9.4	Diskusi Keputusan Engineering	105
BAB 5	SIMPULAN DAN SARAN	106
5.1	Simpulan	106
5.2	Saran	107
	DAFTAR PUSTAKA	110
	LAMPIRAN	120
BAB A	Mockup Tampilan Aplikasi Smart City Reporter	120
BAB B	Contoh Sampel Multimodal Tambahan	124

BAB C	Rincian Pemrosesan, Ekstraksi <i>Embedding</i>, dan Fusi Multimodal Dini	125
BAB D	Skema Basis Data Supabase Lengkap	129
BAB E	Konfigurasi RunPod, Workflow SSH, dan Manajemen Lingkungan	133
BAB F	Landasan Teori Pemrosesan Multimodal dan Reduksi Dimensi .	135
BAB G	Landasan Teori Pelengkap	143
BAB H	Deskripsi <i>Use Case</i> SmartCityApps	149

DAFTAR TABEL

2.1	Perbandingan model <i>encoder</i> teks.	31
2.2	Perbandingan inferensi <i>on-device</i> dan <i>on-server</i>	35
2.3	Perbandingan penelitian ini dengan pekerjaan multimodal terdahulu.	49
3.1	Perbandingan aplikasi pelaporan publik sejenis.	59
3.2	Pemetaan rumusan masalah ke komponen solusi.	59
3.3	Distribusi sampel per instansi target setelah pemetaan.	64
3.4	Pembagian dataset stratifikasi.	65
3.5	Konfigurasi <i>pod</i> RunPod untuk pelatihan dan ekstraksi <i>embedding</i>	66
3.6	Hyperparameter CatBoost.	69
3.7	Matriks rancangan eksperimen ablasi ($4 \text{ encoder citra} \times 4 \text{ encoder teks} \times 2 \text{ varian head}$).	72
3.8	Spesifikasi <i>endpoint</i> server FastAPI.	76
3.9	Ringkasan <i>use case</i> utama SmartCityApps.	81
3.10	Modul dan layanan utama aplikasi Flutter.	84
4.1	Lingkungan perangkat lunak utama.	92
4.2	Distribusi sampel per instansi (dinas) pada set train.	93
4.3	Metrik kinerja konfigurasi terbaik pada set uji (DINOv3 large + mE5 large + CatBoost+PGS).	94
4.4	Metrik per-kelas pada set uji (DINOv3 large + mE5 large + CatBoost+PGS).	94
4.5	Hasil ablasi modalitas pada set uji.	95
4.6	Hasil ablasi kalibrasi pada set uji.	96
4.7	Macro F1 (CatBoost+PGS) pada matrix grid pasangan encoder.	97

4.8	Validasi paritas numerik PyTorch terhadap ONNX pada set uji. . . .	99
4.9	Contoh perutean instansi terdekat dari hasil klasifikasi dan koordinat laporan.	102
C.1	Daftar <i>encoder</i> citra yang dievaluasi.	127
C.2	Daftar <i>encoder</i> teks yang dievaluasi.	127
D.1	Atribut tabel <code>profiles</code>	129
D.2	Atribut tabel <code>reports</code>	131
D.3	Atribut tabel <code>report_history</code>	132
F.1	Perbandingan algoritma tokenizer subword.	140
H.1	<i>Use case description</i> untuk Mendaftar atau Masuk Akun.	149
H.2	<i>Use case description</i> untuk Mengambil Foto Laporan.	150
H.3	<i>Use case description</i> untuk Menulis Narasi Laporan.	150
H.4	<i>Use case description</i> untuk Mengirim Laporan untuk Klasifikasi. . .	151
H.5	<i>Use case description</i> untuk Meninjau Hasil Klasifikasi AI.	151
H.6	<i>Use case description</i> untuk Mengoreksi Kategori Manual.	152
H.7	<i>Use case description</i> untuk Menyimpan Laporan ke Riwayat. . . .	152
H.8	<i>Use case description</i> untuk Melihat Riwayat dan Peta Laporan. . . .	153

DAFTAR GAMBAR

3.1	Diagram alir kerangka berpikir penelitian secara keseluruhan.	52
3.2	Diagram alir tahap akuisisi data dari <i>web scraping</i> portal CRM Jakarta sampai pembagian data stratifikasi.	54
3.3	Fase persiapan pelatihan: setup pod RunPod, instalasi dependensi, dan ekstraksi <i>embedding</i> citra dan teks ke berkas <i>cache</i>	55
3.4	Fase eksperimen: pelatihan <i>classifier head</i> CatBoost+PGS untuk 16 pasangan <i>encoder</i> , pemilihan model terbaik, dan ekspor ONNX dengan validasi paritas numerik.	56
3.5	Diagram aktivitas tahap <i>deployment</i> mencakup ekspor ONNX, server FastAPI untuk inferensi server-side, aplikasi Android SmartCityApps sebagai klien, perutean instansi terdekat, dan pengujian sistem.	57
3.6	Contoh struktur data multimodal: foto ojek parkir di area halte bus, narasi singkat warga, dan label <i>Dinas Perhubungan</i> . Tiga kolom yang ditampilkan adalah kolom <i>Gambar</i> , kolom <i>Laporan</i> , dan kolom <i>Label</i>	60
3.7	Alur inferensi multimodal server-side dari citra dan teks ke prediksi 9 instansi, confidence, dan routing instansi terdekat berdasarkan koordinat.	73
3.8	Diagram komponen dan deployment SmartCityApps: klien Android, backend FastAPI untuk inferensi server-side, HuggingFace sebagai model registry/storage, dan Supabase.	76
3.9	Skema basis data Supabase untuk laporan, prediksi, penugasan instansi, dan bukti penyelesaian.	79

3.10	<i>Use case diagram</i> SmartCityApps dengan prediksi server-side, confidence level, routing instansi terdekat, dan bukti penyelesaian. . . .	80
3.11	<i>Activity diagram</i> siklus laporan SmartCityApps dari pelaporan, inferensi server-side, routing instansi terdekat, sampai bukti penyelesaian.	82
3.12	<i>Class diagram</i> lapisan service dan repositori aplikasi Flutter. . . .	83
3.13	<i>Sequence diagram</i> pengiriman laporan, inferensi server-side, penyimpanan assignment, dan bukti penyelesaian.	83
4.1	Distribusi sembilan instansi (dinas) pada set train.	92
4.2	Ablasi modalitas pada CatBoost dan CatBoost+PGS.	96
4.3	Perbandingan balanced accuracy dan macro F1 dengan dan tanpa PGS.	97
4.4	Heatmap macro F1 pada matrix grid pasangan encoder.	98
4.5	ERD basis data Supabase untuk laporan, prediksi, assignment instansi terdekat, bukti penyelesaian, dan kebijakan per peran.	101
4.6	<i>Confusion matrix</i> ternormalisasi (per baris) pada set uji untuk konfigurasi terbaik DINOv3 large + mE5 large + CatBoost+PGS. . . .	103
A.1	Mockup layar <i>splash</i> dan <i>onboarding</i>	120
A.2	Mockup layar buat laporan.	121
A.3	Mockup layar tinjauan hasil AI.	121
A.4	Mockup layar rekomendasi instansi.	122
A.5	Mockup layar riwayat laporan.	122
A.6	Mockup layar peta laporan.	123
A.7	Mockup layar profil dan pengaturan.	123

- B.1 Contoh 2 struktur data multimodal: foto saluran air mampet dengan narasi panjang yang menjelaskan dampak banjir terhadap warga sekitar, dipetakan ke label *Kelurahan*. Narasi yang panjang menyediakan konteks tekstual kuat, sedangkan citra memvalidasi kondisi fisik saluran. 124
- B.2 Contoh 3 struktur data multimodal: foto pembatas jalan dengan cat yang sudah pudar, narasi permohonan pengecatan ulang oleh warga, dipetakan ke label *Dinas Perhubungan*. Kondisi visual pudar pada citra menjadi sinyal utama, sedangkan narasi menegaskan jenis tindakan yang diharapkan. 124

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pelaporan masalah lingkungan oleh masyarakat saat ini masih menghadapi hambatan struktural yang signifikan. Pada platform Cepat Respon Masyarakat (CRM) yang dikelola oleh Pemerintah Provinsi DKI Jakarta, sebanyak 185.852 laporan warga ditindaklanjuti sepanjang tahun 2024 melalui tiga belas kanal pengaduan terintegrasi, dengan lebih dari 95.000 laporan diterima hanya pada semester pertama tahun yang sama ([Jakarta Smart City, 2024b,a](#)). Pada tingkat nasional, Sistem Pengelolaan Pengaduan Pelayanan Publik Nasional (SP4N-LAPOR!) yang dikelola oleh Kementerian Pendayagunaan Aparatur Negara dan Reformasi Birokrasi (KemenPANRB) telah menerima 2,1 juta laporan kumulatif dengan 679 instansi terhubung, yaitu 34 kementerian, 101 lembaga, dan 544 pemerintah daerah ([Kementerian Pendayagunaan Aparatur Negara dan Reformasi Birokrasi, 2024](#)). Volume laporan yang besar tersebut menuntut proses triase yang cepat dan akurat agar laporan dapat segera diteruskan kepada instansi yang berwenang.

Realitas operasional menunjukkan bahwa proses triase manual menjadi sumber keterlambatan utama. Laporan resmi KemenPANRB mencatat rerata waktu tindak lanjut nasional sebesar 6,1 hari kerja pada awal 2023, dengan target maksimum 10 hari kerja ([Kementerian Pendayagunaan Aparatur Negara dan Reformasi Birokrasi, 2023](#)). Pada tingkat satu laporan secara individual, penelitian sebelumnya yang mengamati alur internal CRM Jakarta melaporkan bahwa rerata waktu yang dibutuhkan petugas untuk membaca, menafsirkan, dan menugaskan satu laporan kepada

instansi yang tepat mencapai hampir dua jam ([Hanif et al., 2024](#)). Keterlambatan tersebut bukan semata disebabkan oleh keterbatasan kapasitas instansi, melainkan oleh sifat manual dari proses triase. Petugas dituntut membaca setiap laporan yang masuk, menafsirkan deskripsi yang sering kali singkat dan ambigu, kemudian secara manual menentukan instansi yang paling tepat untuk menanganinya. Dengan volume laporan yang terus meningkat seiring pertumbuhan populasi perkotaan, pendekatan manual ini tidak lagi berkelanjutan ([Siret and Sabadie, 2022](#)).

Bukti tambahan dari sisi praktis pemerintah daerah menunjukkan bahwa tingkat tindak lanjut efektif laporan warga melalui platform CRM Provinsi DKI Jakarta hanya mencapai 93,2 persen ([Harian Haluan, 2024](#)), yang berarti masih terdapat ribuan laporan yang tidak dapat diselesaikan secara optimal akibat penugasan yang lambat atau kurang tepat. Pada konteks akademis, penelitian [Hanif et al. \(2024\)](#) menunjukkan bahwa otomatisasi penugasan laporan multimodal pada CRM Jakarta menggunakan kombinasi DINOv2, multilingual-E5, dan Posterior Gaussian Sampling dapat mencapai akurasi 80,5 persen pada delapan kategori instansi, jauh lebih cepat daripada proses manual yang membutuhkan hingga dua jam per laporan. Penelitian [Zahro et al. \(2025\)](#) pada platform LaporGub Provinsi Jawa Tengah juga melaporkan akurasi 78,5 persen menggunakan XLM-RoBERTa dengan focal loss pada 53.774 laporan, sekalipun penelitian tersebut hanya menggunakan modalitas teks. Studi [Shen and Hu \(2025\)](#) memperkuat temuan tersebut dengan menyimpulkan bahwa pemanfaatan model multimodal berskala besar dalam tata kelola kota meningkatkan akurasi identifikasi kejadian secara signifikan dibandingkan proses manual konvensional, sekaligus mereduksi beban kerja petugas dalam proses kategorisasi laporan.

Masalah lingkungan perkotaan seperti jalanan rusak, kecelakaan, sampah menumpuk, vandalisme, dan pohon tumbang memiliki karakteristik yang khas, yaitu masalah-masalah tersebut dapat dideteksi melalui dua kanal informasi yang saling melengkapi. Kanal pertama adalah citra (foto) yang merekam kondisi nyata di lokasi kejadian dan menyediakan bukti visual yang kuat. Kanal kedua adalah narasi teks dalam Bahasa Indonesia yang ditulis oleh pelapor untuk menjelaskan konteks, urgensi,

dan rincian peristiwa yang tidak selalu terlihat pada foto. Kombinasi kedua modalitas ini sering kali jauh lebih informatif daripada salah satunya saja. Sayangnya, banyak sistem pelaporan publik hanya memperlakukan citra sebagai lampiran pasif dan teks sebagai isian formulir tanpa mengintegrasikan keduanya untuk membantu kategorisasi otomatis. Akibatnya, proses triase tetap bertumpu pada penilaian petugas dan rentan terhadap inkonsistensi serta keterlambatan (Firgia et al., 2022).

Kemajuan terbaru dalam bidang *Artificial Intelligence* (AI), khususnya pada arsitektur *deep learning* berbasis *Transformer*, memungkinkan integrasi pengenalan citra dan pemahaman teks secara bersamaan dalam satu *pipeline* klasifikasi. Pada sisi citra, model *self-supervised* berskala besar seperti DINOv3 (Siméoni et al., 2025) mampu menghasilkan representasi visual padat (*dense embedding*) yang transferable ke berbagai domain tanpa membutuhkan pelabelan ulang yang masif. Pada sisi teks, model *encoder* multibahasa seperti *multilingual-E5* (Wang et al., 2024) mampu menyandikan kalimat Bahasa Indonesia ke ruang vektor yang sejalan dengan ratusan bahasa lainnya, sehingga laporan warga yang ditulis secara informal sekalipun dapat diolah menjadi fitur diskriminatif. Pendekatan multimodal ini menjadi anchor metodologis yang dipakai pada penelitian ini, yaitu memperluas pekerjaan Hanif et al. (2024) dengan mengganti DINOv2 menjadi DINOv3, memperluas cakupan kategori menjadi sembilan dinas, serta menyediakan jalur ekspor ONNX yang dilayani melalui FastAPI dan dikonsumsi oleh aplikasi mobile.

Pendekatan multimodal yang diusulkan dalam penelitian ini memilih skema fusi awal (*early fusion*) dengan konkatenasi *embedding*, kemudian melatih algoritma klasifikasi berbasis CatBoost di atas vektor fusi tersebut. CatBoost (Prokhorenko et al., 2018) adalah varian *gradient boosting* pada pohon keputusan yang sangat kompetitif untuk masukan tabular berdimensi tinggi seperti *embedding* padat. Pemilihan CatBoost sebagai *classifier head* pada *pipeline* fusi multimodal ini didasarkan pada beberapa pertimbangan praktis, yaitu kebutuhan komputasi pelatihan yang rendah, ketahanan terhadap *overfitting* pada dataset berukuran sedang, dukungan *ordered boosting* yang mengurangi *target leakage*, kemampuan kalibrasi probabilitas melalui Posterior Gaussian Sampling (Ustimenko et al., 2023), serta

kemampuan ekspor model akhir ke format ONNX untuk inferensi yang konsisten lintas *platform*. Karena kedua *encoder* dibekukan (*frozen*) selama pelatihan, permukaan parameter yang dilatih hanya tersisa pada algoritma CatBoost, sehingga eksperimen dapat dijalankan secara cepat dan reproducible. Argumen yang lebih lengkap mengenai pemilihan CatBoost dipaparkan pada Bab 3 bagian Justifikasi Pemilihan CatBoost.

Untuk menjembatani *pipeline* multimodal ini ke perangkat pengguna, penelitian membangun sebuah server inferensi berbasis FastAPI dengan *endpoint* `/predict` multipart yang menerima berkas gambar, narasi laporan, latitude, dan longitude, kemudian mengembalikan instansi prediksi beserta peluang masing-masing kelas dan assignment instansi terdekat. Sisi klien dibangun sebagai aplikasi Android SmartCityApps menggunakan *framework* Flutter, dengan *state management* Riverpod, navigasi GoRouter, otentikasi dan persistensi cloud melalui Supabase yang menyediakan basis data PostgreSQL, penyimpanan berkas, dan otentikasi tiga peran (Warga Pelapor, Operator Instansi, dan Moderator) (Google LLC, 2024b; Rouselet, 2024; Supabase Inc., 2024). Pemisahan antara komputasi inferensi berat di sisi server dengan antarmuka pengguna yang ringan di sisi mobile memberikan tiga keuntungan, yaitu fleksibilitas pembaruan model tanpa rilis ulang aplikasi, konsumsi daya yang rendah pada perangkat pengguna, dan kemampuan menjalankan model berukuran besar yang tidak praktis untuk *deployment* sepenuhnya *on-device*. Total ukuran artefak ONNX (*image encoder* 1,15 GB, *text encoder*, dan *classifier head*) jauh melebihi batas wajar paket aplikasi Android, sehingga keputusan untuk menyajikan inferensi melalui server adalah pilihan yang konsisten dengan praktik baik *edge AI deployment* (Amor and Gutiérrez, 2025; Wang et al., 2025b). Komunikasi antara klien dan server menggunakan protokol HTTP standar dengan format *multipart/form-data* sehingga gambar, teks, dan koordinat dapat dikirim dalam satu permintaan tunggal.

Setelah model menghasilkan prediksi dinas, backend FastAPI menjalankan routing instansi terdekat berdasarkan kelas prediksi dan koordinat latitude-longitude laporan. Model tetap menghasilkan keluaran multiclass pada sembilan instansi target;

tahap routing tidak mengubah keluaran tersebut menjadi kluster baru, melainkan memilih unit instansi aktif yang paling dekat dan relevan dengan lokasi kejadian. Hasil yang dikirim ke aplikasi mencakup instansi prediksi, nilai *confidence*, skor probabilitas setiap kelas, nama instansi tujuan, dan estimasi jarak.

Penelitian ini akan mengeksplorasi penggunaan arsitektur *early-fusion multi-modal* yang menggabungkan *encoder* visual DINOv3-large (Siméoni et al., 2025) dan *encoder* teks multilingual-E5-large (Wang et al., 2024) dengan kepala klasifikasi *gradient boosting* CatBoost (Prokhorenkova et al., 2018) yang dilengkapi mekanisme *Posterior Gaussian Sampling* (Ustimenko et al., 2023) dalam mengklasifikasikan laporan warga Jakarta CRM ke dalam sembilan kategori dinas operasional. Penelitian ini mencoba mengisi *gap* dengan memperkenalkan kombinasi *frozen encoder* visual-bahasa modern dan *gradient-boosted head* pada domain laporan warga berbahasa Indonesia, yang sepanjang penelusuran penulis belum pernah dilakukan oleh karya terkait (Radford et al., 2021; Audebert et al., 2019; Offi et al., 2020; Alam et al., 2018; Koshy and Elango, 2023) yang umumnya berfokus pada domain bencana, dokumen, atau visual umum berbahasa Inggris. Selain itu, penelitian ini juga akan menggunakan dataset autentik hasil *web scraping* dari portal aduan publik Jakarta CRM yang berisi pasangan citra dan narasi tekstual berbahasa Indonesia, dipetakan ke sembilan instansi target setelah konsolidasi dari 487 nama SKPD mentah. Dalam konteks ini, penulis berencana melakukan penelitian untuk dapat meningkatkan efisiensi dan akurasi perutean laporan warga ke instansi yang tepat sekaligus mengukur ketidakpastian prediksi melalui ensemble virtual yang ditawarkan PGS. Dengan menerapkan rancangan ini, sistem yang dibangun memiliki potensi besar untuk membantu memetakan keluhan warga ke dinas yang relevan secara konsisten dan dapat dipertanggungjawabkan. Oleh karena itu, tujuan penelitian ini adalah untuk mengeksplorasi kemampuan arsitektur *early-fusion* multimodal berbasis DINOv3-large dan multilingual-E5-large dengan kepala CatBoost+PGS dalam mengklasifikasikan laporan warga Jakarta berdasarkan citra dan narasi tekstual. Dengan demikian, penelitian ini diharapkan dapat memberikan solusi yang lebih efektif dan efisien dalam perutean laporan warga, sekaligus meng-

hasilkan *pipeline* lintas-platform berupa server inferensi FastAPI dan klien Android berbasis Flutter, serta memberikan wawasan baru mengenai penggunaan model AI multimodal modern, khususnya kombinasi DINOv3, multilingual-E5, dan CatBoost+PGS, pada domain layanan publik berbahasa Indonesia. Penelitian ini oleh karenanya mengusulkan pengembangan prototipe sistem pelaporan masalah lingkungan berbasis fusi multimodal citra dan teks dengan judul: **“PENGEMBANGAN SISTEM PELAPORAN MASALAH LINGKUNGAN DENGAN INTEGRASI DATA GAMBAR DAN TEKS UNTUK MENDUKUNG *SMART CITY*”**.

1.2 Rumusan Masalah

Berdasarkan uraian latar belakang di atas, penelitian ini merumuskan lima pertanyaan penelitian utama yang dirancang agar dapat diverifikasi secara empiris:

1. Bagaimana arsitektur klien-server berbasis Flutter, FastAPI, dan Supabase dapat memenuhi kebutuhan fungsional pelaporan multimodal pada SmartCityApps dengan tetap menjaga konsumsi sumber daya perangkat pengguna seminimum mungkin?
2. Sejauh mana kombinasi DINOv3 *large*, *multilingual-E5 large*, dan CatBoost dengan skema *early fusion* dapat mengklasifikasikan laporan warga Jakarta CRM ke sembilan kategori dinas, diukur menggunakan *macro F1 score*, *balanced accuracy*, dan *confusion matrix*?
3. Sejauh mana ekspor model ke format ONNX mempertahankan paritas numerik terhadap implementasi PyTorch, diukur menggunakan selisih probabilitas maksimum pada set uji?
4. Bagaimana backend menentukan instansi tujuan terdekat berdasarkan sembilan kategori hasil prediksi serta koordinat latitude-longitude laporan?
5. Mengapa skema inferensi *on-server* berbasis FastAPI dipilih ketimbang *on-device* berbasis ONNX Runtime Mobile untuk arsitektur DINOv3 *large* dan

multilingual-E5 large, serta bagaimana konsekuensinya terhadap latensi, keterse-diaan, dan biaya operasional?

1.3 Ruang Lingkup

Ruang lingkup penelitian ini dibatasi sebagai berikut:

1. Domain dan Kategori Laporan

- (a) Domain penelitian adalah aduan publik Jakarta yang dikelola melalui kanal CRM (*Citizen Relationship Management*) Provinsi DKI Jakarta.
- (b) Kategori dinas yang diklasifikasikan dibatasi pada sembilan kategori target hasil pemetaan dari 487 nama Satuan Kerja Perangkat Daerah (SKPD) mentah, dengan ketidakseimbangan kelas yang ditangani melalui pembobotan kelas (*class weights*).
- (c) Penelitian tidak membahas masalah lingkungan yang tidak dapat didektesi secara visual maupun tekstual oleh pelapor warga, misalnya pencemaran udara mikro atau pencemaran air secara kimiawi.

2. Modalitas dan Data

- (a) Modalitas masukan terdiri atas citra (foto laporan) dan teks (narasi laporan dalam Bahasa Indonesia, kolom *laporan*).
- (b) Pengumpulan data dilakukan dari catatan CRM Jakarta dengan proses pembersihan label, normalisasi teks, dan penanganan ketidakseimbangan kelas melalui pembobotan kelas (*class weights*).
- (c) *Encoder* citra dan teks bersifat dibekukan (*frozen*); permukaan yang dilatih hanya algoritma CatBoost.

3. Arsitektur dan Teknologi

- (a) *Encoder* citra adalah DINOv3 *large* (Siméoni et al., 2025) yang menghasilkan *embedding* dengan dimensi 1024.

- (b) *Encoder* teks adalah *multilingual-E5 large* (Wang et al., 2024) yang menghasilkan *embedding* dengan dimensi 1024.
- (c) Skema fusi yang digunakan adalah *early fusion* berbasis konkatenasi, menghasilkan vektor fitur berdimensi 2048 yang menjadi masukan bagi CatBoost.
- (d) Server inferensi diimplementasikan menggunakan FastAPI dengan *endpoint* `/health` dan `/predict multipart`.
- (e) Aplikasi mobile diimplementasikan menggunakan Flutter pada platform Android dengan *backend* berbasis Supabase yang menyediakan otentikasi, basis data PostgreSQL, dan penyimpanan berkas. Platform iOS dibahas pada Bab 2 secara teoritis sebagai cakupan ekosistem mobile, namun implementasi dan pengujian iOS berada di luar lingkup penelitian.

4. Fitur Utama Aplikasi

- (a) Pengguna dapat mengambil foto atau memilih gambar dari galeri sebagai bukti visual masalah lingkungan.
- (b) Pengguna dapat menulis narasi laporan dalam Bahasa Indonesia untuk melengkapi gambar.
- (c) Sistem mengirimkan kombinasi gambar, teks, latitude, dan longitude ke server FastAPI, kemudian menampilkan instansi prediksi beserta tingkat kepercayaan (*confidence*), peringkat prediksi teratas, dan instansi tujuan terdekat.
- (d) Sistem merekomendasikan instansi pemerintah daerah terdekat berdasarkan hasil klasifikasi dan koordinat latitude-longitude laporan.
- (e) Pengguna dapat mengoreksi hasil klasifikasi secara manual sebelum menyimpan laporan.
- (f) Riwayat laporan beserta koordinat GPS lokasi kejadian disimpan secara aman di Supabase dengan *row-level security* per peran pengguna.

5. Peran Pengguna

- (a) Warga Pelapor, yaitu masyarakat umum yang melaporkan masalah lingkungan perkotaan.
- (b) Operator Instansi, yaitu petugas dinas pemerintah daerah yang menerima dan memverifikasi laporan sesuai instansi yang ditugaskan.
- (c) Moderator (Super Admin), yaitu pengguna dengan hak akses lintas instansi untuk meninjau seluruh laporan dan mengubah penugasan.

6. Limitasi

- (a) Sistem yang dibangun merupakan prototipe dan belum terintegrasi dengan sistem informasi pemerintah daerah yang sesungguhnya seperti CRM Jakarta produksi atau SP4N-LAPOR! KemenPANRB.
- (b) Pengujian terbatas pada platform Android. Implementasi iOS, walaupun tersedia secara konseptual karena Flutter mendukung kedua platform, tidak menjadi target rilis penelitian ini.
- (c) Performa model bergantung pada kualitas foto dan kelayakan narasi laporan. Foto yang sangat buram atau narasi yang kosong dapat menurunkan akurasi klasifikasi.
- (d) Keandalan layanan secara keseluruhan bergantung pada ketersediaan server FastAPI yang menjadi titik inferensi tunggal serta layanan Supabase sebagai penyedia basis data dan otentikasi.

1.4 Tujuan dan Manfaat Penelitian

1.4.1 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah diuraikan, penelitian ini memiliki lima tujuan utama:

1. Mengembangkan prototipe aplikasi Android berbasis Flutter, FastAPI, dan Supabase untuk pelaporan masalah lingkungan yang mengintegrasikan klasifikasi multimodal (citra dan teks) berbasis AI, dengan tetap menjaga konsumsi sumber daya perangkat pengguna seminimum mungkin.
2. Membangun arsitektur *early fusion* yang mengonkatenasi *embedding* DINOv3 *large* dan *multilingual-E5 large* dengan algoritma klasifikasi CatBoost untuk memetakan laporan warga ke sembilan kategori dinas pada Jakarta CRM, diukur menggunakan *macro F1 score*, *balanced accuracy*, dan *confusion matrix*.
3. Membangun *pipeline* ekspor *encoder* citra, *encoder* teks, dan *classifier head* ke format ONNX serta server inferensi FastAPI, dengan validasi paritas numerik antara implementasi PyTorch dan ONNX pada selisih probabilitas maksimum di bawah ambang yang ditetapkan.
4. Merancang mekanisme perutean instansi otomatis yang memilih instansi tujuan terdekat berdasarkan hasil klasifikasi 9 instansi dan koordinat latitude-longitude laporan.
5. Menelaah dan mendokumentasikan keputusan deployment skema inferensi *on-server* ketimbang *on-device* pada konteks model berskala besar, sehingga konsekuensinya terhadap latensi, ketersediaan, dan biaya operasional dapat dijelaskan secara empiris.

1.4.2 Manfaat Penelitian

1. **Manfaat bagi Masyarakat:** Aplikasi ini mempermudah proses pelaporan masalah lingkungan dengan memberikan panduan klasifikasi otomatis berbasis citra dan teks. Laporan yang dihasilkan menjadi lebih terstruktur dan informatif bagi instansi yang dituju.
2. **Manfaat bagi Pemerintah Daerah:** Mekanisme perutean otomatis memiliki potensi untuk memangkas waktu triase manual secara signifikan, dari rerata

hampir dua jam per laporan menjadi hampir instan, sehingga laporan dapat didistribusikan ke instansi yang tepat tanpa intervensi manusia. Operator Instansi memperoleh antrean laporan yang sudah terkategori sesuai bidang masing-masing.

3. **Manfaat bagi Ilmu Pengetahuan:** Penelitian ini memberikan kontribusi berupa demonstrasi praktis fusi awal antara fitur citra hasil pelatihan mandiri (*self-supervised*) yang kuat dengan *embedding* teks multibahasa pada domain layanan publik Bahasa Indonesia, serta *pipeline* lengkap dari pelatihan model berbasis PyTorch hingga inferensi berbasis ONNX yang dilayani melalui FastAPI.
4. **Manfaat bagi Ekosistem *Smart City*:** Sistem ini membuktikan bahwa pemisahan antara jalur inferensi berat di sisi server dengan klien Flutter yang ringan merupakan strategi yang layak untuk membangun layanan publik cerdas berbasis AI multimodal, sehingga model dapat diperbarui dari satu permukaan *deployment* tanpa harus merilis ulang aplikasi pengguna.
5. **Manfaat bagi Pengembangan Lanjutan:** Dokumentasi keputusan deployment *on-server* dengan justifikasi ukuran model dan kebutuhan komputasi memberikan dasar yang dapat ditelusuri untuk penelitian lanjutan yang mengeksplorasi distilasi atau kuantisasi guna memungkinkan inferensi *on-device* sebagai *fallback*.

1.5 Metode Penelitian

Metodologi penelitian ini disusun dalam empat tahap utama yang saling berkesinambungan.

1.5.1 Pengumpulan dan Persiapan Data

Dataset Jakarta CRM dikumpulkan dari kanal aduan publik dengan setiap baris mencatat foto kejadian, narasi laporan dalam Bahasa Indonesia, dan label dinas

penanggung jawab. Data kemudian diproses melalui tahapan pembersihan label (penghapusan duplikat, normalisasi nama dinas), pembersihan teks (normalisasi unicode, penghapusan tanda baca berlebih, *whitespace collapsing*), serta penangan ketidakseimbangan kelas melalui pembobotan kelas (*class weights*) pada algoritma klasifikasi, bukan melalui pengubahan jumlah sampel (*resampling*). Pembagian data dilakukan secara stratifikasi ke dalam partisi *train* (43.241 sampel), *validation* (9.266 sampel), dan *test* (9.266 sampel), dengan menjaga proporsi label tiap kelas tetap konsisten pada setiap partisi.

1.5.2 Ekstraksi Embedding dengan Encoder Beku

Setiap foto dilewatkan melalui *encoder* citra DINOv3 *large* versi ONNX untuk menghasilkan vektor representasi berdimensi 1024. Pra-proses citra mengikuti pengaturan standar yang diwajibkan oleh DINOv3, yaitu *resize* sisi pendek, *center crop* ke ukuran target, normalisasi piksel dengan *mean* dan *std* ImageNet, serta konversi ke format tensor NCHW. Setiap narasi laporan dilewatkan melalui *tokenizer multilingual-E5* dan kemudian *encoder* teks versi ONNX untuk menghasilkan vektor representasi yang juga berdimensi 1024. Kedua *encoder* bersifat dibekukan (*frozen*); tidak ada gradien yang mengalir kembali ke parameter *encoder* selama pelatihan.

1.5.3 Pelatihan Kepala CatBoost dan Ekspor Model

Vektor citra dan teks dikonkatenasi sehingga membentuk fitur berdimensi 2048. *Classifier head* berbasis CatBoost dilatih pada vektor fusi tersebut dengan tujuan meminimalkan *logloss* multi-kelas. *Hyperparameter* CatBoost yang ditelusuri meliputi *iterations*, *depth*, *learning_rate*, *l2_leaf_reg*, dan *bagging_temperature*, dengan kriteria pemilihan model terbaik berbasis *macro F1 score* pada set validasi. Untuk meningkatkan kualitas estimasi probabilitas, *classifier head* dikalibrasi menggunakan Posterior Gaussian Sampling (PGS) (Ustimenko et al., 2023). Model akhir disimpan dalam format *.cbm* (*native* CatBoost) dan diekspor ke format

ONNX. Validasi paritas numerik dilakukan dengan membandingkan probabilitas keluaran model *native* dengan model ONNX pada sampel set uji.

1.5.4 Pengembangan Server FastAPI dan Aplikasi Flutter

Server inferensi dibangun menggunakan FastAPI yang memuat ketiga *artifact* ONNX (*encoder* citra, *encoder* teks, dan algoritma CatBoost) sekali pada saat inisialisasi. *Endpoint* `/predict` menerima permintaan *multipart/form-data* berisi berkas gambar, kolom teks laporan, latitude, dan longitude, kemudian menjalankan jalur fusi awal yang sama dengan jalur pelatihan. Aplikasi Flutter SmartCityApps dibangun dengan arsitektur berbasis fitur, manajemen *state* Riverpod, navigasi GoRouter, dan persistensi cloud Supabase. Layanan klasifikasi pada Flutter mengirim permintaan ke *endpoint* FastAPI, menerima respons berbentuk kategori dinas, probabilitas, confidence, dan assignment instansi terdekat, lalu meneruskan hasil ke layar tinjauan AI yang menyediakan peluang koreksi manual oleh pengguna, peta lokasi GPS, serta opsi penyimpanan laporan ke basis data Supabase dengan otentikasi tiga peran.

1.6 Sistematika Penulisan

Skripsi ini disusun ke dalam lima bab dengan rincian berikut:

Bab 1 Pendahuluan memaparkan latar belakang permasalahan triase manual laporan warga pada platform CRM Jakarta dan SP4N-LAPOR!, rumusan lima pertanyaan penelitian, ruang lingkup, tujuan dan manfaat penelitian, serta metode penelitian secara ringkas dan sistematika penulisan.

Bab 2 Tinjauan Pustaka mengulas landasan teori yang melatari penelitian, mencakup konsep *Smart City* dan partisipasi publik, dasar AI dan *deep learning*, arsitektur *Transformer* dan *Vision Transformer*, *encoder* citra DINOv3 dan *encoder* teks *multilingual-E5*, skema fusi multimodal, lingkungan deployment yang mencakup ONNX serta inferensi *on-device* dibandingkan *on-server*, ekosistem Flutter dan Supabase, serta penelitian terkait yang mendahului pekerjaan ini.

Bab 3 Metode Penelitian menguraikan tahapan penelitian secara rinci, mulai dari diagram alir kerangka berpikir, analisis kebutuhan dengan tiga peran pengguna, akuisisi dan persiapan dataset Jakarta CRM, lingkungan komputasi pelatihan, pemrosesan multimodal, ekspor ONNX, justifikasi pemilihan CatBoost sebagai *classifier head*, rancangan aplikasi Flutter SmartCityApps dan server FastAPI, routing instansi terdekat, diagram UML dan arsitektur sistem, serta skenario pengujian dan metrik evaluasi.

Bab 4 Hasil dan Pembahasan menyajikan lingkungan pengujian, persiapan dan eksplorasi dataset, hasil eksperimen utama dengan kombinasi DINOv3 *large*, *multilingual-E5 large*, dan CatBoost yang dikalibrasi PGS, hasil ablasi modalitas, hasil ablasi pasangan *encoder*, validasi paritas ONNX, hasil implementasi aplikasi Flutter, serta pembahasan dan analisis kesalahan.

Bab 5 Simpulan dan Saran merangkum jawaban atas lima rumusan masalah berdasarkan hasil eksperimen dan implementasi, kemudian memberikan saran untuk penelitian lanjutan yang dapat memperluas pekerjaan ini.

BAB 2

TINJAUAN PUSTAKA

Bab ini memaparkan landasan teoretis yang menjadi pijakan rancangan sistem klasifikasi multimodal pada penelitian ini. Pemaparan disusun dari konsep yang paling umum, yaitu *smart city* dan partisipasi publik, menuju konsep yang lebih spesifik, yaitu akuisisi data melalui *web scraping*, kerangka kecerdasan buatan, pra-proses citra dan teks, ekstraksi fitur dan reduksi dimensi, mekanisme *attention*, arsitektur *Transformer*, keluarga *backbone* visual dan *encoder* teks modern, fusi multimodal, ekosistem inferensi ONNX, permodelan UML, kontainerisasi Docker, hingga *stack* aplikasi mobile dan *web framework*. Setiap bagian berusaha menjelaskan tidak hanya definisi, melainkan juga motivasi, sejarah singkat, dan formulasi matematis bila diperlukan, sebagai dasar bagi keputusan metodologis yang dijelaskan pada Bab Metode Penelitian.

2.1 *Smart City dan Smart Environment*

Smart City secara umum didefinisikan sebagai kerangka kerja perkotaan yang memanfaatkan teknologi informasi dan komunikasi untuk meningkatkan kualitas hidup warga, mengoptimalkan operasional kota, serta memastikan keberlanjutan ekonomi dan lingkungan. Esensi dari *smart city* terletak pada integrasi enam pilar utama, yaitu *smart governance*, *smart mobility*, *smart environment*, *smart economy*, *smart people*, dan *smart living*. Di antara enam pilar tersebut, *smart governance* dan *smart environment* menjadi fondasi penting untuk menciptakan interaksi yang harmonis antara pemerintah dan masyarakat melalui sistem layanan publik yang responsif (Siret and Sabadie, 2022).

Salah satu tantangan terbesar dalam mewujudkan *smart environment* adalah pengelolaan laporan masalah lingkungan perkotaan yang datang dari berbagai sumber secara bersamaan. Menurut [Sadiq and Omlin \(2025\)](#), kota pintar bergantung pada infrastruktur penginderaan yang beragam, mulai dari perangkat *Internet of Things* (IoT), kamera pengawas, hingga *smartphone* warga yang menghasilkan data dalam jumlah masif. Kemampuan untuk memproses dan menafsirkan data tersebut secara otomatis, baik citra maupun narasi tekstual, merupakan komponen kritis dalam membangun sistem tata kelola kota yang adaptif. Penelitian [Shen and Hu \(2025\)](#) secara khusus mendemonstrasikan bahwa integrasi model multimodal berskala besar yang menggabungkan citra dan teks ke dalam sistem tata kelola kota mampu meningkatkan akurasi identifikasi kejadian secara signifikan dan mereduksi beban kerja manual petugas dalam proses kategorisasi laporan masyarakat.

Konsep *smart environment* pada kota berkembang menghadapi kompleksitas khusus, antara lain heterogenitas warga pelapor (literasi digital, perangkat, dan akses jaringan), keragaman karakter masalah lingkungan, serta struktur instansi pemerintah daerah yang berbeda-beda di setiap wilayah. Hal ini membuat sistem pelaporan tidak dapat sekadar mengadopsi solusi generik dari literatur, melainkan harus dirancang dengan kepekaan terhadap konteks lokal.

2.2 Sistem Pelaporan dan Partisipasi Publik

Sistem pelaporan publik adalah saluran formal yang memungkinkan warga menyampaikan keluhan, saran, atau temuan terkait fasilitas dan layanan kota kepada pemerintah daerah. Pada era pra-digital, kanal pelaporan bertumpu pada surat resmi, kotak saran, atau telepon *call center*, dengan keterlambatan respons yang diukur dalam hitungan hari. Era web membuka kanal formulir daring yang memangkas waktu pengiriman, namun tetap memerlukan operator manusia untuk menafsirkan dan memilah laporan ke instansi yang tepat. Era *mobile* memperkenalkan aplikasi berbasis ponsel yang memuat formulir pelaporan dengan dukungan unggah foto dan koordinat GPS. Walau demikian, sebagian besar tahap kategorisasi laporan di-

lakukan oleh petugas *back office*.

Hasil penelitian [Firgia et al. \(2022\)](#) menunjukkan bahwa digitalisasi sistem pengaduan melalui platform *mobile* dan web mempercepat pengiriman data dari masyarakat ke pemerintah, sekaligus mengurangi kesalahan data yang disebabkan oleh proses manual. Partisipasi publik merupakan komponen esensial dalam pembangunan berkelanjutan; sistem pelaporan yang dirancang dengan baik harus mampu mendorong keterlibatan warga untuk meningkatkan respons pemerintah terhadap ancaman lingkungan. Integrasi kecerdasan buatan ke dalam sistem pelaporan, khususnya melalui klasifikasi citra dan teks, membawa dimensi baru yang memungkinkan laporan diproses, dikategorikan, dan didistribusikan ke instansi yang tepat sehingga mengurangi intervensi manual. Hal tersebut tidak hanya mempercepat waktu respons, tetapi juga mengurangi *human error* pada tahap kategorisasi.

2.3 Web Scraping dan Akuisisi Data

2.3.1 Definisi dan Latar Belakang

Web scraping adalah teknik otomatis untuk mengekstraksi data terstruktur dari halaman web yang awalnya dirancang untuk konsumsi manusia ([Mitchell, 2018](#)). Pada masa ketika antarmuka pemrograman aplikasi (*Application Programming Interface*, API) yang resmi belum tersedia atau terbatas, *web scraping* menjadi cara utama bagi peneliti, analis, dan pengembang untuk membangun korpus data dari kanal publik. Pada konteks penelitian sosial dan tata kelola kota, *web scraping* sering digunakan untuk mengumpulkan data dari portal aduan, media sosial, situs berita, dan situs pemerintah, terutama saat data tersebut tidak tersedia dalam bentuk *dataset* unduhan.

Halaman target *scraping* dibedakan menjadi halaman statis (konten sudah ada pada respons HTML server) dan halaman dinamis (konten dirender JavaScript di peramban). Alur umum *web scraping* mengikuti lima tahap, yaitu identifikasi sumber, pengunduhan HTTP, penguraian (*parsing*) HTML, pembersihan dan normalisasi, serta penyimpanan ke berkas terstruktur. Pada penelitian ini akuisisi data

CRM Jakarta memakai `requests` untuk pengunduhan, `BeautifulSoup` dan `lxml` untuk penguraian, serta `Selenium` untuk halaman dinamis. Uraian rinci klasifikasi halaman, alur lima tahap, dan perbandingan pustaka *scraping* disajikan pada Lampiran G.

2.4 Kecerdasan Buatan (*Artificial Intelligence*)

Kecerdasan buatan (*Artificial Intelligence*, AI) adalah bidang ilmu komputer yang membangun sistem yang dapat melakukan tugas yang biasanya membutuhkan kecerdasan manusia, seperti memahami bahasa, mengenali objek, dan belajar dari pengalaman (Russell and Norvig, 2021). Sejak kebangkitannya pada 2012 yang dipicu ketersediaan data berskala besar (contohnya ImageNet (Russakovsky et al., 2015)), komputasi GPU, dan terobosan *deep learning* seperti AlexNet (Krizhevsky et al., 2012), AI bergeser dari rekayasa aturan manual ke pendekatan berbasis data. Hubungan AI, *machine learning* (ML), dan *deep learning* (DL) bersifat konsentris: ML adalah sub-bidang AI yang belajar pola dari data tanpa aturan eksplisit, dan DL adalah kelas ML berbasis jaringan saraf banyak lapis yang mempelajari representasi hierarkis dari data mentah (LeCun et al., 2015; Goodfellow et al., 2016).

2.5 *Machine Learning*

2.5.1 Definisi Formal

Mitchell (1997) memberikan definisi klasik yang masih relevan, yaitu sebuah program komputer dikatakan belajar dari pengalaman E terhadap kelas tugas T dengan ukuran kinerja P jika kinerjanya pada tugas-tugas di T , sebagaimana diukur oleh P , meningkat seiring dengan pengalaman E .

2.5.2 Klasifikasi Multi-kelas

Klasifikasi multi-kelas adalah varian *supervised learning* dengan ruang label yang berdimensi $K > 2$ kelas yang saling eksklusif. Model menghasilkan vektor probabilitas $\mathbf{p} \in \mathbb{R}^K$ dengan $\sum_k p_k = 1$ melalui fungsi *softmax*:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad (2.1)$$

dengan z_k adalah *logit* pada kelas ke- k . Pelatihan biasanya meminimalkan *categorical cross-entropy*:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log p_k, \quad (2.2)$$

dengan y_k adalah label *one-hot*.

2.5.3 Bias-Variance Trade-off dan Regularisasi

Setiap model belajar pada keseimbangan antara *bias*, yaitu kesalahan akibat asumsi yang terlalu kaku, dan *variance*, yaitu kesalahan akibat sensitivitas berlebihan terhadap data pelatihan. Model dengan kapasitas terlalu rendah cenderung *underfit*, sementara model dengan kapasitas terlalu tinggi cenderung *overfit*. Beberapa teknik regularisasi yang umum digunakan meliputi regularisasi L1 dan L2 pada bobot model, *dropout*, *weight decay*, dan augmentasi data.

2.5.4 Gradient Boosting dan CatBoost

Gradient boosting adalah metode *ensemble* yang membangun prediksi sebagai jumlah dari banyak *weak learner*, biasanya pohon keputusan, dengan setiap pohon dilatih untuk memperkecil residu fungsi tujuan dari iterasi sebelumnya. Bentuk umum prediksi pada iterasi ke- M adalah:

$$F_M(\mathbf{x}) = F_0(\mathbf{x}) + \sum_{m=1}^M \eta h_m(\mathbf{x}), \quad (2.3)$$

dengan η adalah *learning rate* dan h_m adalah pohon ke- m yang dilatih untuk meminimalkan turunan negatif fungsi tujuan terhadap F_{m-1} .

CatBoost adalah varian *gradient boosting* yang dikembangkan oleh Yandex (Prokhorenko-va et al., 2018). Tiga inovasi utama CatBoost adalah:

1. *Ordered boosting*, yaitu strategi yang menghindari *prediction shift* dengan

membangun perkiraan residu pada setiap titik data hanya dari sub-himpunan pelatihan yang tidak menyertakan titik tersebut.

2. Pohon yang berbentuk *oblivious (symmetric trees)*, yaitu pohon di mana setiap level menggunakan kondisi pemisahan yang sama. Pohon *oblivious* cepat dieksekusi pada inferensi karena dapat dihitung sebagai indeks bit-level, dan menyediakan regularisasi struktural alami.
3. Penanganan fitur kategorikal native melalui *target statistics* yang aman terhadap *leakage*.

2.6 Deep Learning

2.6.1 Perceptron, Multilayer Perceptron, dan Backpropagation

Unit komputasi dasar pada jaringan saraf adalah *perceptron*, yang menghitung jumlah berbobot dari masukan ditambah bias dan melewatkannya melalui fungsi aktivasi:

$$y = \sigma(\mathbf{w}^\top \mathbf{x} + b). \quad (2.4)$$

Multilayer Perceptron (MLP) menumpuk banyak lapisan *perceptron* dengan fungsi aktivasi non-linear (ReLU, GELU, SiLU) sehingga model dapat mempelajari fungsi kompleks; pelatihan dilakukan dengan *backpropagation* melalui aturan rantai.

Convolutional Neural Networks (CNN) memanfaatkan operasi konvolusi untuk menangkap pola lokal pada citra, dengan tumpukan konvolusi-aktivasi-*pooling* yang membangun *receptive field* hierarkis. Arsitektur penting dalam sejarahnya meliputi AlexNet (Krizhevsky et al., 2012) dan ResNet (He et al., 2016) dengan koneksi residual. Sampai 2020 CNN mendominasi tugas visi, hingga Dosovitskiy et al. (2021) menunjukkan bahwa *Vision Transformer* (ViT) berbasis *self-attention* dapat mengunggulinya bila dilatih pada data berskala sangat besar.

Foundation models adalah model berskala besar yang dilatih pada data sangat banyak lalu dipakai ulang pada banyak tugas hilir, umumnya melalui *self-supervised learning*. Dua paradigma utamanya pada visi adalah pembelajaran kontras (*contrastive learning*: SimCLR, MoCo, DINO) dan *Masked Image Modeling* (MIM: MAE (He et al., 2022), BEiT, EVA (Fang et al., 2023)). Paradigma inilah yang melahirkan *encoder* DINOv3 yang dipakai pada penelitian ini.

2.7 Pemrosesan Data Multimodal: Ringkasan

Pra-proses data multimodal pada penelitian ini terdiri atas dua jalur. Jalur citra didelegasikan ke *AutoImageProcessor* HuggingFace (kelas *DINOv3ViTImageProcessorFast*) yang melakukan *resize* ke 224×224 , *square padding* untuk mempertahankan seluruh konten, normalisasi per kanal dengan rerata dan simpangan baku ImageNet (Persamaan F.1), serta konversi tensor ke format NCHW. Augmentasi data tidak diterapkan karena *encoder* dibekukan dan hanya dipakai pada modus inferensi. Jalur teks meliputi normalisasi unicode NFC, pembersihan karakter kontrol, dan tokenisasi dengan *SentencePiece* pada *encoder* multilingual-E5 atau *WordPiece* pada *encoder* berbasis BERT. Empat algoritma subword utama yang berkaitan dengan studi ini (BPE, WordPiece, SentencePiece, Unigram LM) diringkas pada Tabel F.1. Latar teoretis lengkap, mencakup formula normalisasi piksel, pertimbangan aspek rasio, kompresi JPEG, EXIF, serta perbandingan rinci algoritma tokenizer, disajikan pada Lampiran F bagian F.1 dan F.2.

2.8 Ekstraksi Fitur (*Feature Extraction*)

2.8.1 Definisi

Ekstraksi fitur adalah transformasi data masukan mentah menjadi representasi yang lebih ringkas dan diskriminatif untuk tugas hilir. Tujuannya adalah menyederhanakan permukaan keputusan model klasifikasi tanpa kehilangan informasi yang relevan. Pada visi tradisional, ekstraksi fitur dilakukan dengan algoritma berbasis aturan; pada *deep learning* modern, ekstraksi fitur menjadi bagian dari jaringan

yang dilatih secara *end-to-end*.

2.8.2 Fitur *Hand-Crafted*

Sebelum era *deep learning*, ekstraksi fitur citra mengandalkan algoritma berbasis aturan yang dirancang oleh ahli, antara lain:

- **SIFT** (*Scale-Invariant Feature Transform*) (Lowe, 2004): mendeteksi titik kunci pada skala-ruang dan mendeskripsikan tetangga lokalnya menggunakan histogram orientasi gradien yang invariant terhadap skala dan rotasi.
- **HOG** (*Histogram of Oriented Gradients*) (Dalal and Triggs, 2005): membagi citra ke dalam *cell* kecil dan menghitung histogram orientasi gradien per *cell*.
- **SURF** dan **ORB**: alternatif yang lebih cepat dari SIFT untuk aplikasi *real-time*.

Pada teks, fitur klasik meliputi *Bag-of-Words* (BoW) dan *Term Frequency-Inverse Document Frequency* (TF-IDF), yang merepresentasikan teks sebagai vektor frekuensi kata yang dibobot terhadap distribusi kata di seluruh korpus.

2.8.3 Fitur Hasil Pelatihan dan *Transfer Learning*

Pada *deep learning*, fitur dipelajari secara otomatis dari data, sehingga representasi internal jaringan dapat berfungsi sebagai *embedding* yang diskriminatif. Salah satu keuntungan praktis dari pendekatan ini adalah *transfer learning*, yaitu menggunakan jaringan yang sudah dilatih pada tugas dengan banyak data untuk tugas baru dengan data terbatas. Pada konfigurasi *linear probing*, hanya algoritma klasifikasi yang dilatih sementara *backbone* dibekukan; pada konfigurasi *fine-tuning* penuh, seluruh bobot dilatih ulang dengan *learning rate* yang kecil.

2.8.4 Ruang *Embedding* dan Sifat Geometrisnya

Embedding hasil *encoder* berada dalam ruang vektor berdimensi tinggi. Beberapa ukuran kemiripan yang berguna pada ruang ini meliputi kemiripan kosinus:

$$\text{cos_sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}, \quad (2.5)$$

yang umum dipakai pada model *contrastive* seperti CLIP dan E5. Ruang ini dapat divisualisasikan dengan teknik reduksi dimensi seperti t-SNE atau UMAP untuk inspeksi kualitatif.

2.9 Reduksi Dimensi: Ringkasan

Vektor representasi yang dihasilkan oleh *encoder* citra dan teks pada penelitian ini berdimensi 1024, sehingga vektor fusi awal berdimensi 2048. Reduksi dimensi seperti *Principal Component Analysis* (PCA) merupakan teknik klasik untuk mereduksi dimensi linear dengan menjaga varians maksimum, namun pada penelitian ini PCA **tidak diterapkan** karena dua alasan. Pertama, *classifier head* CatBoost berbasis pohon keputusan *oblivious* yang melakukan pemilihan dimensi secara implisit melalui *split* sehingga tidak terpengaruh *curse of dimensionality* secara signifikan. Kedua, reduksi linear berpotensi menghilangkan informasi diskriminatif yang dibutuhkan untuk membedakan kelas dinas yang dekat secara semantik. Pembahasan rinci mengenai formulasi PCA, prosedur *Singular Value Decomposition*, kriteria *explained variance ratio*, serta alternatif non-linear seperti *Kernel PCA*, *Autoencoder*, t-SNE, UMAP, dan LDA disajikan pada Lampiran F bagian F.3.

2.10 Mekanisme Attention

2.10.1 Motivasi: Keterbatasan RNN pada Konteks Panjang

Pada arsitektur sekuensial seperti *Recurrent Neural Networks* (RNN) dan *Long Short-Term Memory* (LSTM), informasi diteruskan tahap demi tahap melalui *hidden state*. Pada konteks yang panjang, sinyal informasi awal cenderung menghilang (*vanishing gradient*) atau meledak (*exploding gradient*), sehingga kapasitas mengingat ketergantungan jarak jauh terbatas. Mekanisme *attention* pertama kali diperkenalkan oleh Bahdanau et al. (2015) untuk *neural machine translation*, se-

bagai cara membiarkan *decoder* mengambil informasi langsung dari setiap posisi *encoder* berdasarkan relevansi.

2.10.2 Scaled Dot-Product Attention

Mekanisme *attention* modern, sebagaimana dirumuskan oleh Vaswani et al. (2017), adalah *scaled dot-product attention* dengan tiga peran vektor: *query* ($\mathbf{Q}$), *key* ($\mathbf{K}$), dan *value* ($\mathbf{V}$). Skor relevansi antara setiap *query* dengan setiap *key* dihitung sebagai *dot product*, kemudian diskalakan dengan akar dimensi untuk menjaga stabilitas numerik:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.6)$$

Hasil *softmax* adalah matriks berbobot yang menjumlahkan *value* secara berbobot. Penskalaan dengan $\sqrt{d_k}$ mencegah skor yang terlalu besar pada dimensi tinggi, yang akan membuat *softmax* saturasi.

2.10.3 Multi-Head Attention

Multi-head attention adalah perluasan *single-head attention* yang menjalankan beberapa proyeksi linear paralel dari *query*, *key*, dan *value*, kemudian mengkonkatenasi keluaran setiap *head* dan memproyeksikannya kembali ke dimensi d_{model} . Motivasinya adalah bahwa satu *head* terbatas pada satu pola hubungan antar token, sementara teks dan citra memuat banyak pola hubungan secara simultan. Setiap *head* bekerja pada subruang berdimensi $d_k = d_{\text{model}}/h$ sehingga total komputasi tetap sebanding dengan *single-head attention*. Formulasi matematis lengkap *multi-head attention* beserta matriks proyeksi per *head* disajikan pada Lampiran G.

2.10.4 Self-Attention versus Cross-Attention

Self-attention adalah varian di mana $\mathbf{Q}$, $\mathbf{K}$, dan $\mathbf{V}$ berasal dari sumber barisan yang sama, sehingga model belajar hubungan antar elemen di dalam barisan. Pada *cross-attention*, $\mathbf{Q}$ berasal dari satu barisan (contohnya *decoder*) dan $\mathbf{K}, \mathbf{V}$ berasal

dari barisan lain (contohnya *encoder*). *Cross-attention* merupakan pondasi banyak *Transformer* multimodal yang menyatukan token visual dan tekstual.

2.10.5 Kompleksitas dan Implikasi Skalabilitas

Kompleksitas waktu *self-attention* adalah $O(n^2d)$ dengan n panjang barisan dan d dimensi *embedding*. Kuadratisme pada n ini menjadi hambatan utama untuk konteks yang sangat panjang dan memicu munculnya berbagai varian seperti *Linformer*, *Performer*, *Longformer*, dan *Flash Attention* yang mengoptimalkan memori dan komputasi.

2.11 Arsitektur Transformer

2.11.1 Encoder-Decoder

Arsitektur *Transformer* yang asli (Vaswani et al., 2017) terdiri dari tumpukan *encoder* dan *decoder*. Setiap blok *encoder* memuat *multi-head self-attention* diikuti *feed-forward network* (FFN), masing-masing dilengkapi sambungan residual dan *layer normalization*. Setiap blok *decoder* memuat *masked self-attention* (untuk menjaga sifat autoregresif), *cross-attention* ke keluaran *encoder*, dan FFN. Pada tugas klasifikasi, hanya bagian *encoder* yang biasanya digunakan.

2.11.2 Positional Encoding, Normalisasi, dan Feed-Forward

Karena *self-attention* bersifat permutasi-invariant, posisi token disuntikkan secara eksplisit melalui skema *positional encoding*, baik sinusoidal, *learned*, maupun *Rotary Position Embedding* (RoPE) (Su et al., 2024) yang memutar vektor *query* dan *key* sesuai posisi. Setiap blok *Transformer* juga memuat *layer normalization* (LN), sambungan residual yang memudahkan aliran gradien, dan *feed-forward network* (FFN) berupa dua proyeksi linear dengan aktivasi GELU. Pada *Transformer* modern, LN umumnya diterapkan dalam skema *pre-norm* (sebelum sub-modul) karena lebih stabil pada model dalam yang sangat besar.

2.11.3 Vision Transformer (ViT)

Vision Transformer (Dosovitskiy et al., 2021) mengadaptasi *Transformer* ke domain visi dengan empat langkah utama, yaitu (1) *patchify* citra menjadi potongan tetap (contohnya 16x16), (2) proyeksi linear setiap *patch* menjadi vektor *embedding*, (3) penambahan token kelas ($[CLS]$) dan *positional embedding*, dan (4) penumpukan blok *Transformer encoder*. Token CLS pada lapisan terakhir digunakan sebagai representasi citra.

2.12 Arsitektur Backbone Visual Modern

Bagian ini memaparkan tiga keluarga *backbone* visual modern yang menjadi rujukan utama dalam literatur klasifikasi citra, yaitu DINOv3, EVA-02, dan Hiera. Pemahaman terhadap ketiganya memberi konteks tentang lanskap *vision foundation model* kontemporer.

2.12.1 DINOv3

2.12.1.1 Garis Keturunan: DINO, DINOv2, DINOv3

DINO (*self-Distillation with NO labels*) diperkenalkan oleh Caron et al. (2021) sebagai metode *self-supervised learning* pada ViT. Kontribusi utamanya adalah skema *self-distillation* antara *student* dan *teacher* yang berbagi arsitektur, di mana *teacher* adalah rerata bergerak eksponensial (*Exponential Moving Average*, EMA) dari *student*. Tujuan pelatihan adalah membuat distribusi keluaran *student* pada augmentasi suatu citra cocok dengan distribusi keluaran *teacher* pada augmentasi lain dari citra yang sama.

DINOv2 (Oquab et al., 2023) memperluas DINO dalam beberapa dimensi penting, yaitu (1) skala dataset yang jauh lebih besar (LVD-142M, sekitar 142 juta citra), (2) resep pelatihan yang lebih stabil dengan teknik regularisasi tambahan, (3) varian model dari ViT-S, B, L, sampai g, dan (4) peluncuran sebagai *foundation model* terbuka. DINOv2 menjadi salah satu *encoder* citra paling banyak digunakan untuk

tugas hilir tanpa *fine-tuning*.

DINOv3 (Siméoni et al., 2025) adalah generasi berikutnya yang menyempurnakan DINOv2 dengan beberapa peningkatan, yaitu peningkatan skala data dan model, perbaikan kualitas *patch embedding* untuk *dense prediction*, peningkatan *robustness* terhadap pergeseran distribusi (*distribution shift*), serta efisiensi inferensi yang lebih baik. DINOv3 dikenalkan sebagai *vision foundation model* yang siap pakai pada berbagai tugas, termasuk klasifikasi, segmentasi semantik, dan ko-responsensi padat.

2.12.1.2 Mekanisme Self-Distillation Tanpa Label

Pada DINO dan turunannya, dua jaringan yang sama secara arsitektural dilatih bersamaan, yaitu jaringan *student* g_{θ_s} dan jaringan *teacher* g_{θ_t} . Untuk setiap citra masukan x , dua himpunan augmentasi diturunkan, yaitu *global crops* (besar, biasanya dua) dan *local crops* (kecil, biasanya delapan). Strategi multi-crop ini berperan ganda, yaitu memberi banyak pasang positif dan mendorong *student* untuk merekonstruksi konteks dari potongan kecil.

Setiap jaringan menghasilkan distribusi probabilitas pada K *prototype* keluaran melalui *head* proyeksi yang ditambahkan di atas *backbone*:

$$P_s(x) = \text{softmax}\left(\frac{g_{\theta_s}(x)}{\tau_s}\right), \quad P_t(x) = \text{softmax}\left(\frac{g_{\theta_t}(x) - c}{\tau_t}\right), \quad (2.7)$$

dengan τ_s dan τ_t adalah suhu *softmax* pada *student* dan *teacher*, dan c adalah *centering teacher* yang dihitung sebagai rerata bergerak dari keluaran *teacher* pada *batch*. *Centering* mencegah keruntuhan ke distribusi seragam, sementara *sharpening* (yaitu $\tau_t < \tau_s$) mencegah keruntuhan ke distribusi terlalu tajam.

Loss yang diminimalkan adalah *cross-entropy* antara distribusi *teacher* dan *student* pada seluruh pasangan augmentasi:

$$\mathcal{L}_{\text{DINO}} = \sum_{x \in V} \sum_{x' \in V', x' \neq x} -P_t(x)^\top \log P_s(x'), \quad (2.8)$$

dengan V adalah himpunan *global crops* dan V' adalah himpunan seluruh *crops*. Parameter *teacher* diperbarui secara EMA dari *student*:

$$\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s. \quad (2.9)$$

Tidak ada label dataset yang diperlukan, sehingga DINO dapat dilatih pada citra yang sangat banyak dan tidak terkurasi.

2.12.1.3 Token CLS dan Token Patch

Setelah pelatihan, ViT menyediakan dua jenis representasi yang berguna, yaitu (1) token CLS yang berfungsi sebagai *embedding* global citra, dan (2) token *patch* yang berfungsi sebagai *embedding* lokal pada masing-masing *patch*. DINO terkenal karena token CLS-nya menghasilkan kluster semantik yang baik tanpa pelabelan, sementara token *patch*-nya bahkan dapat digunakan untuk segmentasi tanpa supervisi melalui pemetaan kemiripan kosinus.

2.12.1.4 Sifat *Emergent* pada Skala Besar

Pada skala model dan data yang besar, DINO dan turunannya menunjukkan beberapa sifat *emergent*, yaitu:

- **Robustness:** representasi yang konsisten pada distribusi yang bergeser, contohnya kondisi pencahayaan rendah, foto buram, perspektif tidak biasa.
- **Dense correspondence:** kemampuan menemukan padanan piksel yang konsisten antar dua citra dari objek yang sama.
- **Segmentasi tanpa supervisi:** pengelompokan piksel berdasarkan kemiripan token *patch*, tanpa pelabelan piksel.
- **Transfer ke domain baru:** performa yang kuat ketika digunakan sebagai *frozen encoder* pada domain hilir, termasuk medis, satelit, dan urban.

Selain DINOv3, penelitian ini juga mengevaluasi dua *backbone* visual modern sebagai pembanding pada eksperimen ablasi. **EVA-02** (Fang et al., 2023) menggabungkan *Masked Image Modeling* (MIM) dengan penyesuaian fitur CLIP sebagai target supervisi, serta memperbaiki ViT melalui Sub-LN, SwiGLU FFN, dan 2D RoPE. **Hiera** (Ryali et al., 2023) menyederhanakan ViT dengan menghapus *positional embedding* eksplisit dan memakai *pooling attention* hierarkis yang dilatih dengan resep MAE (He et al., 2022), sehingga lebih ringan dan cepat. Uraian rinci kedua arsitektur ini, termasuk inovasi dan paradigma pra-latih MAE, disajikan pada Lampiran G. Hasil eksperimen (Bab 4) menunjukkan DINOv3 dan EVA-02 unggul, sementara Hiera konsisten paling lemah pada domain laporan publik.

2.13 Model *Encoder* Teks

Bagian ini memaparkan model *encoder* teks yang relevan dengan klasifikasi dokumen Bahasa Indonesia, yaitu BERT, mBERT, IndoBERT, Cendol-mT5, BGE-M3, dan keluarga E5 multibahasa.

Sebagai pembanding pada eksperimen ablasi, penelitian ini juga mengevaluasi empat *encoder* teks lain. **BERT** (Devlin et al., 2019) dan varian multibahasanya **mBERT** (104 bahasa) menjadi tonggak *Transformer encoder* dua arah berbasis MLM. **IndoBERT** (Koto et al., 2020; Wilie et al., 2020) adalah keluarga BERT yang dilatih khusus pada korpus Bahasa Indonesia sehingga lebih kaya pada idiom dan istilah lokal. **Cendol-mT5** (Cahyawijaya et al., 2024) memanfaatkan *encoder* mT5 (Xue et al., 2021) yang dilanjutkan pra-latih dan *instruction tuning* pada Bahasa Indonesia dan bahasa daerah. **BGE-M3** (Chen et al., 2024) adalah *embedding model* multibahasa multifungsi (*dense, sparse, multi-vector*) berbasis XLM-RoBERTa *large* dengan *embedding* 1024-d. Uraian rinci keempat *encoder* pembanding ini disajikan pada Lampiran G; bagian berikut memfokuskan pembahasan pada keluarga E5 yang menjadi *encoder* teks terpilih pada *pipeline* akhir.

2.13.1 Keluarga E5 dan *multilingual-E5*

E5 (*Embeddings from bidirectional Encoder representations*) adalah keluarga *text encoder* yang dilatih khusus untuk menghasilkan *embedding* kalimat yang baik melalui pelatihan kontras dua tahap pada pasangan teks yang relevan secara semantik. Tahap pertama adalah *contrastive pre-training* pada miliaran pasangan teks lemah dari sumber publik (judul-deskripsi, pertanyaan-jawaban di forum, dan sebagainya), dan tahap kedua adalah *fine-tuning* pada dataset *retrieval* berkualitas tinggi dengan kontras berbobot.

multilingual-E5 (Wang et al., 2024) memperluas E5 ke skenario multibahasa dengan basis arsitektur XLM-RoBERTa. Tiga varian dirilis, yaitu *multilingual-e5-small*, *multilingual-e5-base*, dan *multilingual-e5-large*. Varian *large* memiliki sekitar 560 juta parameter dan menghasilkan *embedding* berdimensi 1024. Salah satu konvensi penting pada E5 adalah penggunaan prefiks pada teks masukan, yaitu *query*: $\langle$ kueri $\rangle$ untuk teks pendek seperti pertanyaan, dan *passage*: $\langle$ dokumen $\rangle$ untuk teks panjang seperti dokumen. Prefiks ini bukan sekadar *cosmetic*, melainkan dipakai pula saat pelatihan untuk membedakan peran teks dalam pasangan kontras. Varian *multilingual-e5-large-instruct* memperkenalkan kemampuan mengikuti instruksi tugas pada prefiks, contohnya *Instruct*: *Classify Indonesian civic report*\n*Query*: , sehingga *embedding* yang dihasilkan dapat dikondisikan pada tugas hilir tertentu.

Embedding yang dihasilkan E5 diambil dari rata-rata token (*mean pooling*) atas vektor token terakhir, kemudian dinormalisasi L2. Properti normalisasi L2 ini menjadikan kemiripan kosinus pada Persamaan 2.5 setara dengan *dot product* yang lebih murah secara komputasi. Pada *benchmark* MIRACL yang mengevaluasi *retrieval* 18 bahasa, *multilingual-E5* bersaing dengan model multibahasa berukuran jauh lebih besar dan menjadi salah satu *encoder* multibahasa terbaik yang tersedia secara terbuka.

2.13.2 Perbandingan Singkat

Tabel berikut merangkum perbandingan singkat enam *encoder* di atas pada beberapa dimensi penting.

Tabel 2.1. Perbandingan model *encoder* teks.

Model	Tokenizer	Bahasa	Tujuan Pra-latih	Catatan
BERT	WordPiece	Inggris	MLM + NSP	Tonggak awal
mBERT	WordPiece	104 bahasa	MLM	Transfer lintas bahasa
IndoBERT	WordPiece	Indonesia	MLM	<i>Benchmark IndoNLU</i>
Cendol-mT5	SentencePiece	Indonesia + daerah	<i>Span corruption</i> + instruksi	Encoder-decoder mT5
BGE-M3	SentencePiece	100+ bahasa	Distilasi multi-fungsi	<i>Dense + sparse + multi-vector</i>
mE5	SentencePiece	100+ bahasa	Kontrasif (XLM-R)	Mendukung <i>instruct</i> prefix

2.14 Fusi Multimodal

2.14.1 Taksonomi Skema Fusi

Pada *multimodal learning*, skema fusi mengatur titik di mana informasi dari modalitas yang berbeda digabungkan. Menurut [Baltrušaitis et al. \(2019\)](#) dan [Ramachandram and Taylor \(2017\)](#), fusi dapat dikelompokkan menjadi tiga kategori utama:

1. **Early fusion** (*feature-level fusion*): fitur dari setiap modalitas digabungkan pada tahap awal, sebelum melewati lapisan klasifikasi. Bentuk paling sederhana adalah konkatenasi vektor.
2. **Late fusion** (*decision-level fusion*): setiap modalitas memiliki klasifikasi terpisah yang menghasilkan keputusan, dan keputusan tersebut digabung dengan

voting, rata-rata, atau *weighted average*.

3. **Hybrid atau intermediate fusion**: kombinasi dari kedua pendekatan, contohnya fusi pada lapisan menengah jaringan dalam.

2.14.2 Bentuk-Bentuk Fusi

Selain konkatenasi sederhana, beberapa skema fusi yang lazim adalah:

- **Konkatenasi**: $\mathbf{e}_{\text{fuse}} = [\mathbf{e}_1; \mathbf{e}_2]$, sederhana dan menjaga seluruh informasi setiap modalitas.
- **Cross-attention fusion**: menggunakan modul *cross-attention* untuk membuat token dari satu modalitas memperhatikan token dari modalitas lain. Contoh: Perceiver IO, Flamingo.
- **Gated fusion**: memakai *gate* terpelajar yang mengatur kontribusi setiap modalitas, mirip dengan *Mixture of Experts*.
- **Bilinear pooling**: produk luar dua vektor sebelum proyeksi, yang kaya secara ekspresif namun mahal pada dimensi tinggi.

2.14.3 Pertimbangan Kalibrasi Skala

Salah satu risiko teknis pada *early fusion* adalah perbedaan skala antar modalitas. Apabila *embedding* satu modalitas memiliki magnitudo rata-rata jauh lebih besar daripada modalitas lain, model dapat secara implisit terlalu banyak bergantung pada modalitas yang dominan. Normalisasi L2 mengatasi hal ini secara sederhana. Alternatif yang lebih kompleks adalah *batch normalization* per modalitas atau pelatihan kalibrasi.

2.15 ONNX dan Inferensi Lintas Kerangka

2.15.1 Spesifikasi ONNX

Open Neural Network Exchange (ONNX) adalah standar format terbuka yang dirancang sebagai jembatan interoperabilitas antar-*framework machine learning* (Microsoft, 2024a). ONNX mendefinisikan grafik komputasi (*computation graph*) yang bersifat *framework-agnostic*, yang memungkinkan model yang dikembangkan dalam PyTorch, TensorFlow, atau *framework* lainnya diekspor ke representasi tunggal yang dapat dieksekusi oleh berbagai *runtime* pada berbagai platform.

Representasi ONNX terdiri atas serangkaian operator standar yang didefinisikan dalam spesifikasi *opset* bernomor. *Opset* dengan versi yang lebih baru mendukung operator yang lebih komprehensif. Saat ekspor, PyTorch melakukan *tracing* grafik komputasi model dan menerjemahkannya ke operator ONNX yang setara, dengan opsi optimasi seperti *constant folding* yang mengeliminasi sub-grafik konstan untuk mengurangi ukuran model dan latensi inferensi.

2.15.2 ONNX Runtime

ONNX Runtime adalah *inference engine* lintas platform yang dioptimalkan untuk mengeksekusi model ONNX secara efisien (Microsoft, 2024a). *Runtime* ini mendukung beberapa *execution provider*, antara lain:

- CPU *Execution Provider* (default).
- CUDA *Execution Provider* untuk GPU NVIDIA.
- NNAPI untuk Android.
- CoreML untuk iOS dan macOS.
- OpenVINO, TensorRT, dan *provider* lain yang dioptimalkan vendor.

2.15.3 Validasi Paritas Numerik

Sebelum model ONNX digunakan dalam produksi, perlu dipastikan bahwa keluarannya konsisten secara numerik dengan model asal. Validasi dilakukan dengan menjalankan kedua model pada *batch* sampel yang sama dan menghitung selisih absolut maksimum (*max abs diff*) di seluruh komponen vektor keluaran. Ambang batas yang umum dipakai adalah 10^{-5} , yang memastikan prediksi top-1 identik pada kedua jalur.

2.16 Inferensi *On-device* versus *On-server*

Pemilihan permukaan *deployment* model berimplikasi langsung terhadap latensi, ketersediaan, biaya operasional, ukuran biner aplikasi, daya baterai perangkat pengguna, dan kemampuan pembaruan model. Bagian ini mengulas dua pendekatan utama yang umum dipakai pada sistem AI berbasis perangkat *mobile*, yaitu inferensi *on-device* dan inferensi *on-server*.

2.16.1 Definisi

Inferensi on-device mengacu pada eksekusi model di dalam perangkat pengguna, baik pada CPU, GPU *mobile*, maupun *Neural Processing Unit* (NPU) yang tersedia pada *chipset* modern. Pada Android, jalur eksekusi yang umum dipakai adalah ONNX Runtime Mobile ([Microsoft, 2024b](#)), TensorFlow Lite ([Google LLC, 2024c](#)), dan NNAPI sebagai abstraksi akselerator perangkat keras. Pada iOS dan macOS, Core ML menjadi jalur resmi yang dipublikasikan oleh Apple ([Apple Inc., 2024](#)).

Inferensi on-server mengacu pada eksekusi model di luar perangkat pengguna, biasanya pada *cluster* komputasi yang menjalankan *web service* sebagai antarmuka inferensi. Klien (aplikasi *mobile*) mengirim permintaan HTTP yang berisi masukan, lalu menerima respons berisi hasil prediksi. Pada penelitian ini, jalur *on-server* dibangun dengan FastAPI ([Ramírez, 2024](#)) yang memuat tiga *artifact* ONNX (*encoder* citra, *encoder* teks, dan *classifier head*) sekali pada inisialisasi.

2.16.2 Trade-off Antara Kedua Pendekatan

Tabel 2.2 merangkum perbedaan utama antara kedua pendekatan pada beberapa dimensi yang relevan untuk sistem pelaporan multimodal.

Tabel 2.2. Perbandingan inferensi *on-device* dan *on-server*.

Dimensi	<i>On-device</i>	<i>On-server</i>
Latensi	Rendah, tanpa <i>round-trip</i> jaringan	Bergantung pada kecepatan dan stabilitas jaringan
Ketersediaan jaringan	Berfungsi tanpa jaringan	Membutuhkan koneksi internet aktif
Privasi data	Data tidak meninggalkan perangkat	Data dikirim ke server, perlu pengamanan tambahan
Biaya operasional	Tidak ada biaya server tetap	Biaya komputasi awan terus berjalan
Ukuran paket aplikasi	Bertambah sesuai ukuran model	Tetap kecil, model dimuat di server
Daya baterai perangkat	Konsumsi tinggi saat inferensi	Konsumsi rendah, hanya kirim permintaan
Pembaruan model	Memerlukan rilis baru aplikasi	Pembaruan terpusat, tanpa rilis aplikasi
Skala model	Terbatas oleh memori dan akselerator perangkat	Tidak dibatasi oleh perangkat pengguna

2.16.3 Teknik Optimisasi untuk *On-device*

Karena model *deep learning* berskala besar tidak praktis untuk dieksekusi langsung pada perangkat pengguna, sejumlah teknik optimisasi umum diterapkan. Kuantisasi (*quantization*) mengubah representasi parameter dari *floating point* 32-bit menjadi presisi yang lebih rendah, contohnya INT8, dengan tetap menjaga akurasi pada batas yang dapat diterima (Amor and Gutiérrez, 2025; Wang et al., 2025b). *Pruning* membuang parameter atau neuron dengan kontribusi rendah sehingga model menjadi lebih ringkas. Distilasi pengetahuan (*knowledge distillation*) melatih model murid berukuran kecil agar meniru perilaku model guru berukuran besar.

Arsitektur ringan seperti MobileNet, EfficientNet, dan turunan keluarga DistilBERT dirancang khusus untuk eksekusi pada perangkat dengan sumber daya terbatas.

2.16.4 Teknik Optimisasi untuk *On-server*

Pada jalur *on-server*, optimisasi diarahkan pada *throughput*, latensi, dan biaya per permintaan. Praktik umum yang diterapkan meliputi pemuatan model sekali pada inisialisasi (sehingga *cold start* tidak terjadi pada setiap permintaan), *batching* permintaan untuk memaksimalkan utilisasi akselerator, *caching* hasil pra-proses, serta penggunaan *execution provider* yang sesuai dengan perangkat keras server (CUDA pada GPU, OpenVINO pada CPU Intel, dan TensorRT pada GPU NVIDIA). Skema ini relatif lebih sederhana untuk dipelihara dibandingkan *on-device* karena hanya satu titik permukaan komputasi yang perlu dioptimalkan.

2.16.5 Implikasi pada Penelitian Ini

Penelitian ini memilih jalur *on-server* dengan FastAPI atas dasar empat pertimbangan teknis. Pertama, ukuran *artifact* ONNX *image encoder* DINOv3 *large* mencapai 1,15 GB; ukuran ini di luar batas wajar paket aplikasi Android dan akan menyebabkan waktu pasang aplikasi yang lama serta penggunaan ruang penyimpanan yang berlebihan pada perangkat pengguna. Kedua, eksperimen ablasi yang melibatkan enam belas kombinasi *encoder* memerlukan iterasi cepat pada server, sehingga jalur *on-server* memungkinkan model diperbarui sentral tanpa harus merilis ulang aplikasi pada *Play Store*. Ketiga, perangkat pengguna pada konteks pelaporan publik di Indonesia memiliki spesifikasi yang sangat heterogen, sehingga pengalaman pengguna lebih konsisten ketika beban komputasi dipusatkan di server. Keempat, kebijakan pembaruan model yang terpusat memberikan jaminan bahwa seluruh pengguna selalu menggunakan versi model paling mutakhir tanpa harus menunggu siklus rilis aplikasi.

Sebagai konsekuensi, sistem ini bergantung pada ketersediaan jaringan dan ketersediaan layanan FastAPI sebagai titik inferensi tunggal. Mitigasi yang dapat di-

lakukan pada penelitian lanjutan meliputi distilasi atau kuantisasi INT8 untuk menghasilkan varian *on-device* yang ringan sebagai *fallback*, serta replikasi server untuk meningkatkan ketersediaan layanan secara horisontal (Wang et al., 2025a).

2.17 Unified Modeling Language (UML) pada Pengembangan Aplikasi

2.17.1 Definisi dan Sejarah

Unified Modeling Language (UML) adalah bahasa permodelan visual standar yang dipergunakan untuk merancang, memvisualisasikan, mendokumentasikan, dan mengkonstruksi sistem perangkat lunak (Booch et al., 2005). UML dirumuskan pada pertengahan 1990-an sebagai unifikasi tiga metode permodelan berorientasi objek yang dikembangkan oleh Grady Booch, James Rumbaugh, dan Ivar Jacobson, kemudian dibakukan oleh *Object Management Group* (OMG). Versi mutakhir UML 2.x mendefinisikan empat belas jenis diagram yang dikelompokkan menjadi diagram struktur (*structural*) dan diagram perilaku (*behavioral*).

UML 2.x mendefinisikan empat belas jenis diagram yang dikelompokkan menjadi diagram struktur (memodelkan elemen statis, contohnya *class*, *component*, dan *deployment diagram*) dan diagram perilaku (memodelkan dinamika sistem, contohnya *use case*, *sequence*, *activity*, dan *state machine diagram*). Penelitian ini memakai *use case*, *activity*, *class*, dan *sequence diagram* untuk mendokumentasikan rancangan SmartCityApps (Bab 3). Uraian rinci jenis-jenis diagram UML serta pendekatan pelengkap *C4 Model* (Brown, 2018) disajikan pada Lampiran G.

2.17.2 Peran UML pada Pengembangan Aplikasi Mobile

Pada pengembangan aplikasi mobile, UML berfungsi sebagai alat komunikasi antara perancang antarmuka, pengembang, dan pemangku kepentingan. Diagram *use case* mendefinisikan fitur yang harus tersedia bagi pengguna, diagram kelas memetakan domain model yang menyusun *state* aplikasi, dan diagram *sequence* menjelaskan alur interaksi antar komponen termasuk panggilan ke layanan jarak jauh seperti server inferensi. Pada arsitektur klien-server multimodal, diagram *de-*

ployment dan *container* membantu memvisualisasikan pemisahan tanggung jawab antara aplikasi mobile sebagai *thin client* dan server sebagai jalur inferensi yang berat.

2.18 Aplikasi Mobile: Android dan iOS

Dua sistem operasi mobile yang dominan adalah Android (kernel Linux, *run-time* ART, distribusi APK/AAB melalui *Play Store*) dan iOS (kernel Darwin, bahasa Swift/Objective-C, distribusi IPA melalui *App Store*) ([Google LLC, 2024a](#); [Apple Inc., 2024](#)). Pengembangan aplikasi mobile dapat dilakukan secara *native* (SDK resmi tiap platform, performa terbaik namun dua basis kode), *cross-platform compiled* (satu basis kode dikompilasi ke biner native, contohnya Flutter, .NET MAUI, React Native), atau *hybrid web* (membungkus konten web dalam *webview*, contohnya Cordova dan Ionic). Penelitian ini memilih pendekatan *cross-platform compiled* dengan Flutter, dengan pengujian dibatasi pada Android.

2.19 Flutter dan Stack Pengembangan Mobile

2.19.1 Flutter sebagai Framework

Flutter adalah *framework* pengembangan aplikasi lintas platform berbasis bahasa Dart yang dikembangkan oleh Google ([Google LLC, 2024b](#)). Berbeda dari pendekatan *hybrid*, Flutter menggunakan *rendering engine* sendiri (Skia atau Impeller) untuk menggambar setiap piksel, sehingga tampilan konsisten lintas platform tanpa bergantung pada *native widgets*; aplikasi dikompilasi *ahead-of-time* ke kode mesin saat rilis.

Pada penelitian ini, manajemen *state* memakai **Riverpod** ([Rousselet, 2024](#)), yaitu *library* reaktif dengan keamanan tipe saat kompilasi yang mendistribusikan *state* melalui *provider* mandiri tanpa bergantung pada hierarki *widget*. Navigasi memakai **GoRouter**, *library* navigasi deklaratif berbasis *URL path* yang menyederhanakan *deep link* dan *redirect* berbasis kondisi otentikasi. Untuk persistensi, aplikasi memilih basis data *cloud* Supabase (dipaparkan pada bagian berikutnya) alih-

alih SQLite lokal, karena alur kerja melibatkan tiga peran pengguna yang membutuhkan akses terhadap data yang sama secara lintas perangkat.

2.20 Supabase sebagai *Backend* Basis Data

2.20.1 Definisi dan Latar Belakang

Supabase adalah *platform Backend-as-a-Service* (BaaS) sumber terbuka yang menyediakan basis data PostgreSQL terkelola, otentikasi, penyimpanan berkas, fungsi tanpa server (*Edge Functions*), dan langganan *realtime* dalam satu paket terintegrasi (Supabase Inc., 2024). Supabase sering disebut sebagai alternatif sumber terbuka dari Firebase, dengan keunggulan utama yaitu basis data relasional penuh berbasis PostgreSQL, dukungan SQL standar, dan kemudahan migrasi keluar tanpa *vendor lock-in*.

Empat komponen utama Supabase yang relevan dengan penelitian ini adalah PostgreSQL terkelola (basis data relasional inti yang mendukung tipe `jsonb`, `enum`, dan `constraint`), PostgREST (REST API otomatis dari skema basis data), Otentikasi (identitas terintegrasi dengan pengguna tersimpan pada `auth.users`), dan Storage (penyimpanan berkas berbasis S3). Selain itu, Supabase menyediakan kanal *realtime* berbasis WebSocket yang memancarkan perubahan baris kepada klien berlangganan, berguna untuk memperbarui daftar laporan tanpa *polling*. Integrasi pada Flutter dilakukan melalui paket `supabase_flutter` yang membungkus seluruh layanan ke dalam satu klien *singleton*.

2.20.2 *Row Level Security* (RLS)

Salah satu fitur kunci Supabase yang berasal dari PostgreSQL adalah *Row Level Security* (RLS), yaitu kebijakan keamanan yang dievaluasi pada level baris. Dengan RLS, akses ke data dapat dibatasi berdasarkan identitas pengguna yang terotentikasi, contohnya `policy auth.uid() = user_id` memastikan setiap pengguna hanya dapat membaca atau memodifikasi laporan miliknya sendiri. RLS membuat aplikasi mobile dapat berkomunikasi langsung ke basis data tanpa perlu lapisan API

kustom, karena keamanan dijamin pada level basis data. Pada penelitian ini RLS dipakai untuk memisahkan akses tiga peran (Warga Pelapor, Operator Instansi, dan Moderator), dengan konsekuensi ketergantungan pada layanan Supabase serta kebutuhan perancangan kebijakan RLS yang teliti.

2.21 FastAPI dan *Web Framework* untuk *Serving*

FastAPI adalah *web framework* Python yang dibangun di atas Starlette (untuk lapisan ASGI) dan Pydantic (untuk validasi data) (Ramírez, 2024). Tiga karakteristik utama FastAPI adalah:

1. **Dukungan asinkron native:** *endpoint* dapat dideklarasikan dengan `async def`, sehingga server dapat menangani permintaan yang diblokir oleh I/O secara bersamaan tanpa membuat *thread* baru per permintaan.
2. **Validasi tipe otomatis:** skema permintaan dideklarasikan sebagai kelas Pydantic, dan FastAPI secara otomatis mengembalikan kesalahan 422 jika permintaan tidak valid.
3. **Dokumentasi otomatis:** skema OpenAPI dan halaman *Swagger UI* dihasilkan otomatis, mempercepat integrasi klien dan pengujian.

FastAPI banyak digunakan untuk *serving* model *machine learning* karena kemudahan integrasi dengan pustaka inferensi seperti *ONNX Runtime*, PyTorch, dan TensorFlow Serving, serta kompatibilitas dengan kontainer Docker dan orkestrasi Kubernetes. *Framework* sejenis yang juga umum dipakai untuk tujuan serupa antara lain Flask (sinkron, lebih sederhana), Django REST Framework (untuk aplikasi besar dengan basis data), TorchServe (*serving* khusus PyTorch), dan TensorFlow Serving (*serving* khusus TensorFlow).

2.22 Docker dan Kontainerisasi

Docker (Merkel, 2014) adalah *platform* kontainerisasi yang mengemas aplikasi beserta seluruh dependensinya ke dalam satu unit yang dapat dijalankan secara

konsisten di berbagai lingkungan. Berbeda dari mesin virtual, kontainer berbagai *kernel* sistem operasi induk dan mengisolasi proses melalui *Linux namespaces* dan *cgroups*, sehingga ringan dan cepat dijalankan. Pada penelitian ini Docker dipakai untuk mengemas server inferensi FastAPI beserta *artifact* ONNX dengan pola *multi-stage build*, sehingga lingkungan inferensi konsisten dan dapat direproduksi. Konsep inti Docker (*image*, *container*, *Dockerfile*, *volume*, *registry*), struktur *Dockerfile*, *multi-stage build*, Docker Compose, serta manfaatnya bagi *pipeline machine learning* diuraikan pada Lampiran G.

2.23 Metrik Evaluasi Klasifikasi Multi-Kelas

2.23.1 Macro F1-Score

Dalam konteks klasifikasi multi-kelas dengan distribusi kelas yang tidak sepenuhnya seimbang, *macro F1-score* merupakan metrik evaluasi yang lebih informatif dibandingkan akurasi sederhana. Sementara akurasi menghitung rasio prediksi benar terhadap total sampel, yang dapat terlihat tinggi meski model gagal pada kelas minoritas, *macro F1-score* memberikan bobot yang setara kepada setiap kelas tanpa memperhatikan frekuensi relatifnya.

F1-score untuk setiap kelas c didefinisikan sebagai rata-rata harmonik antara *precision* dan *recall*:

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad (2.10)$$

dengan

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}. \quad (2.11)$$

Macro F1-score kemudian dihitung sebagai rata-rata aritmetika dari $F1_c$ pada seluruh K kelas:

$$\text{Macro F1} = \frac{1}{K} \sum_{c=1}^K F1_c. \quad (2.12)$$

2.23.2 Metrik Pendukung

Beberapa metrik pendukung yang umum dilaporkan bersama *macro F1-score* adalah:

1. **Akurasi (*Accuracy*)**: proporsi prediksi benar dari keseluruhan sampel.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (2.13)$$

2. ***Balanced Accuracy***: rata-rata *recall* per kelas, lebih robust terhadap ketidakseimbangan kelas.

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_{c=1}^K \text{Recall}_c. \quad (2.14)$$

3. ***Macro ROC-AUC***: rata-rata *Area Under the Receiver Operating Characteristic Curve* per kelas dalam skenario *one-vs-rest*, mengukur kemampuan model membedakan setiap kelas dari semua kelas lain secara probabilistik.
4. ***Confusion Matrix***: matriks $K \times K$ yang memvisualisasikan distribusi prediksi benar dan salah untuk setiap pasangan kelas, memungkinkan identifikasi pola kesalahan spesifik antar kelas yang saling memiliki kemiripan visual atau tekstual.
5. **Kalibrasi Probabilitas**: ukuran sejauh mana probabilitas prediksi mencerminkan kebenaran. *Expected Calibration Error* (ECE) menghitung perbedaan rata-rata antara akurasi dan keyakinan rata-rata pada *bin* probabilitas.

2.24 Teknik Pelatihan Model

2.24.1 *Transfer Learning*

Transfer learning adalah paradigma pelatihan di mana bobot model yang telah dilatih pada dataset skala besar digunakan kembali pada tugas hilir ([Dosovitskiy](#)

et al., 2021). Pendekatan ini terbukti secara konsisten menghasilkan konvergensi yang lebih cepat dan akurasi akhir yang lebih tinggi dibandingkan pelatihan dari awal (*training from scratch*), terutama ketika data target terbatas. Pada konfigurasi *linear probing*, hanya algoritma klasifikasi yang dilatih sementara *backbone* dibekukan; pada konfigurasi *fine-tuning* penuh, seluruh bobot dilatih ulang dengan *learning rate* kecil (umumnya antara 10^{-5} dan 10^{-3}).

2.24.2 Validasi dan Pemilihan Model

Pelatihan biasanya menggunakan partisi *train*, *validation*, dan *test* dengan stratifikasi label. *Hyperparameter* dipilih berdasarkan kinerja model pada set validasi, sementara evaluasi akhir dilaporkan pada set uji yang belum pernah dilihat selama pengembangan. Pada dataset berukuran kecil, validasi silang *k-fold* sering digunakan untuk meningkatkan stabilitas estimasi metrik. *Early stopping* berdasarkan kinerja validasi menjadi cara umum untuk mencegah *overfitting* pada sesi pelatihan yang panjang.

2.25 Penelitian Terkait

Bagian ini menelaah lima penelitian internasional yang relevan dengan kontribusi metodologis penelitian ini, yaitu klasifikasi multimodal berbasis fusi citra dan teks. Setiap entri disusun dalam tiga subbagian yaitu **Gambaran Umum**, **Analisis dan Metode**, dan **Kesimpulan** yang menjelaskan posisi penelitian ini terhadap entri tersebut.

2.25.1 Radford et al. (2021): CLIP, Pretraining Kontrastif Citra dan Teks

2.25.1.1 Gambaran Umum

Radford et al. (2021) mengajukan CLIP (*Contrastive Language-Image Pre-training*) sebagai paradigma baru pelatihan model visual berskala besar dengan supervisi berbasis bahasa alami. Karya ini dipublikasikan pada Proceedings of the 38th International Conference on Machine Learning (ICML 2021), volume 139 dengan

rentang halaman 8748 sampai 8763. CLIP menjadi rujukan dasar bagi sebagian besar pendekatan klasifikasi multimodal modern karena memperkenalkan strategi pelatihan kontrastif yang menyelaraskan ruang representasi citra dan teks pada *embedding* bersama.

2.25.1.2 Analisis dan Metode

CLIP dilatih pada 400 juta pasangan citra dan teks yang dikumpulkan dari internet menggunakan tujuan pelatihan kontrastif simetris. Arsitektur model terdiri atas *image encoder* berbasis Vision Transformer atau ResNet yang dimodifikasi dan *text encoder* berbasis Transformer dengan penyandian *byte-pair encoding*. Pelatihan mengoptimalkan kemiripan kosinus antara representasi citra dan representasi teks pasangan positif sambil meminimalkan kemiripan terhadap pasangan negatif pada *batch* yang sama. Setelah pra-pelatihan, CLIP menunjukkan kemampuan transfer *zero-shot* pada lebih dari tiga puluh dataset visual standar dengan akurasi yang mendekati atau melampaui *baseline supervised* terlatih penuh.

2.25.1.3 Kesimpulan

Penelitian ini mengadopsi prinsip CLIP yaitu pemanfaatan *image encoder* dan *text encoder* pra-latih sebagai *frozen feature extractor* untuk modalitas masing-masing. Perbedaan utamanya terletak pada strategi pengayaan modalitas, yaitu pada CLIP kedua *encoder* diselaraskan secara kontrastif sehingga menghasilkan ruang *embedding* bersama, sementara penelitian ini memilih DINOv3 *large* untuk citra dan *multilingual-E5 large* untuk teks yang dilatih pada tujuan terpisah, lalu disatukan melalui fusi awal berbasis konkatenasi. Pilihan ini memberikan keluwesan dalam mengganti pasangan *encoder* secara independen pada eksperimen ablasi, sesuatu yang sulit dilakukan pada model *end-to-end* seperti CLIP.

2.25.2 Audebert et al. (2019): Multimodal Deep Networks untuk Klasifikasi Dokumen

2.25.2.1 Gambaran Umum

[Audebert et al. \(2019\)](#) mengajukan jaringan saraf dalam multimodal untuk klasifikasi dokumen berbasis citra halaman dan teks hasil OCR (*Optical Character Recognition*). Karya ini dipublikasikan sebagai *preprint* arXiv (2019) dan dipresentasikan pada International Conference on Document Analysis and Recognition. Pendekatan tersebut secara metodologis paling dekat dengan penelitian ini karena menggabungkan modalitas visual dan tekstual pada satu *pipeline* klasifikasi multikelas.

2.25.2.2 Analisis dan Metode

Arsitektur yang diusulkan menggunakan MobileNetV2 sebagai *image encoder* pada citra halaman dokumen dan FastText sebagai *text encoder* pada hasil OCR. Kedua *embedding* dikonkatenasi sebelum dilewatkan ke lapisan *multilayer perceptron* untuk klasifikasi. Eksperimen dilakukan pada dataset RVL-CDIP dan Tobacco3482 yang berisi enam belas kategori dokumen administratif. Hasil menunjukkan bahwa fusi multimodal mengungguli baseline unimodal sebesar beberapa poin persentase pada akurasi top-1, dengan kontribusi modalitas teks lebih besar daripada modalitas citra pada dokumen yang konten visualnya tidak terlalu diskriminatif.

2.25.2.3 Kesimpulan

Penelitian ini mengikuti pola fusi awal berbasis konkatenasi yang sama dengan [Audebert et al. \(2019\)](#). Perbedaan utamanya yaitu domain aplikasi (laporan warga Jakarta CRM versus dokumen administratif), pasangan *encoder* (DINOv3 *large* dan *multilingual-E5* versus MobileNetV2 dan FastText), serta *classifier head* yang dipakai (CatBoost dengan kalibrasi PGS versus MLP). Penggunaan *encoder* berskala besar pada penelitian ini dimungkinkan karena kedua *encoder* dibekukan, sehingga biaya komputasi tetap terkendali walaupun parameter *encoder* jauh lebih banyak daripada arsitektur ringan pada karya tersebut.

2.25.3 Ofli, Alam, dan Imran (2020): Multimodal Deep Learning untuk Disaster Response

2.25.3.1 Gambaran Umum

Ofli et al. (2020) mengembangkan sistem klasifikasi multimodal pada laporan bencana di media sosial sebagai bagian dari upaya *disaster response* berbasis komputasi. Karya ini diterima pada *Proceedings of the 17th International Conference on Information Systems for Crisis Response and Management (ISCRAM 2020)*. Domain aplikasi sistem ini, yaitu klasifikasi laporan warga melalui kanal Twitter, secara konseptual paling dekat dengan klasifikasi laporan warga pada platform CRM Jakarta walaupun kanal pelaporan dan tata bahasa target berbeda.

2.25.3.2 Analisis dan Metode

Arsitektur yang diuji adalah jaringan saraf dalam dengan dua jalur paralel, yaitu *image encoder* VGG16 untuk citra dan jaringan konvolusi pada *embedding* kata GloVe untuk teks. Kedua keluaran lapisan *fully connected* digabungkan dengan operasi konkatenasi sebelum dilewatkan ke lapisan klasifikasi akhir. Dataset yang dipakai adalah CrisisMMD yang berisi laporan bencana berbentuk pasangan teks dan citra dari Twitter. Eksperimen dilakukan pada dua tugas, yaitu klasifikasi informativeness (apakah tweet informatif atau tidak) dan klasifikasi tipe humanitarian. Hasilnya menunjukkan bahwa fusi multimodal mengungguli baseline unimodal pada keempat dataset bencana yang diuji.

2.25.3.3 Kesimpulan

Penelitian ini mengambil inspirasi dari skema fusi konkatenasi pada lapisan tinggi yang juga dipakai oleh Ofli et al. (2020). Perbedaan utamanya yaitu pemilihan *encoder* (VGG16 dan GloVe-CNN versus DINOv3 *large* dan *multilingual-E5 large*), domain (Twitter berbahasa Inggris versus CRM Jakarta berbahasa Indonesia), dan *classifier head* (*fully connected layer* versus CatBoost dengan kalibrasi PGS). Karena DINOv3 dan *multilingual-E5* merupakan *foundation model* yang di-

latih pada data jauh lebih besar daripada VGG16 dan GloVe, penelitian ini mengharapkan kualitas representasi yang lebih kuat tanpa perlu pelatihan ulang *encoder*.

2.25.4 Alam, Ofli, dan Imran (2018): CrisisMMD, Dataset Multimodal Bencana

2.25.4.1 Gambaran Umum

[Alam et al. \(2018\)](#) mempublikasikan CrisisMMD pada Proceedings of the 12th International AAAI Conference on Web and Social Media (ICWSM 2018), Stanford, California. CrisisMMD adalah dataset multimodal pertama yang menyediakan anotasi terstruktur pada pasangan teks dan citra dari Twitter selama tujuh kejadian bencana alam besar. Dataset ini menjadi *benchmark* utama bagi sebagian besar penelitian klasifikasi multimodal pada konteks media sosial krisis.

2.25.4.2 Analisis dan Metode

CrisisMMD berisi 16.058 *tweet* berlabel dengan tiga skema anotasi berlapis, yaitu (1) informativeness, (2) tipe humanitarian, dan (3) severity damage assessment. Setiap *tweet* terdiri atas pasangan citra dan teks; anotasi dilakukan oleh anotator manusia melalui kerangka kerja Qatar Computing Research Institute. Hasil analisis distribusi label menunjukkan ketidakseimbangan kelas yang signifikan, sehingga sebagian besar penelitian lanjutan harus menerapkan strategi pembobotan kelas atau *focal loss*.

2.25.4.3 Kesimpulan

Penelitian ini secara struktural mengikuti praktik baik CrisisMMD, yaitu menyediakan dataset terstratifikasi pada partisi *train*, validasi, dan uji dengan label dinas yang dibersihkan dari ratusan SKPD mentah. Perbedaan kontribusi pada aspek dataset terletak pada bahasa target (Bahasa Indonesia versus Inggris), kanal pelaporan (CRM Jakarta yang merupakan portal resmi pemerintah daerah versus Twitter yang merupakan media sosial publik), dan jumlah kategori (sembilan dinas opera-

sional versus tiga skema anotasi). Skala dataset penelitian ini (61.773 sampel) lebih besar daripada CrisisMMD pada satu skema anotasi, sehingga memberikan ruang lebih luas untuk pelatihan *classifier head*.

2.25.5 Koshy dan Elango (2023): Multimodal Tweet Classification dengan Transformer Bidirectional Attention

2.25.5.1 Gambaran Umum

Koshy and Elango (2023) mengusulkan kerangka klasifikasi multimodal untuk *tweet* bencana dengan model transformer berbasis bidirectional attention. Karya ini dipublikasikan pada *Neural Computing and Applications* volume 35 nomor 2, halaman 1607 sampai 1627, tahun 2023, oleh Springer Nature. Pendekatan ini secara teknis paling dekat dengan kombinasi *encoder* berbasis transformer pada penelitian ini dan menjadi pembanding kuat pada arah penelitian klasifikasi multimodal mutakhir.

2.25.5.2 Analisis dan Metode

Arsitektur yang diusulkan terdiri atas tiga komponen utama, yaitu (1) Vision Transformer (ViT) yang di-*fine-tune* pada citra *tweet*, (2) RoBERTa yang di-*fine-tune* pada teks *tweet*, dan (3) lapisan *biLSTM* dengan mekanisme bidirectional attention yang menggabungkan kedua representasi. Eksperimen dilakukan pada tujuh dataset bencana yang berbeda, yaitu *wildfire*, *hurricane*, *earthquake*, dan *flood* dari berbagai kejadian. Hasilnya menunjukkan bahwa pendekatan transformer multimodal dengan bidirectional attention mengungguli baseline berbasis konkatenasi sederhana pada akurasi dan F1 score, dengan kontribusi attention dalam menangani ketidakseimbangan informasi antar modalitas.

2.25.5.3 Kesimpulan

Penelitian ini berbeda dari Koshy and Elango (2023) pada beberapa aspek metodologis. Pertama, kedua *encoder* pada penelitian ini dibekukan tanpa *fine-tuning*,

sedangkan ViT dan RoBERTa pada karya tersebut di-*fine-tune* ulang. Kedua, skema fusi yang dipakai adalah konkatenasi pada lapisan *embedding* (fusi awal), bukan attention pada lapisan tinggi. Ketiga, *classifier head* berbasis CatBoost dipilih dengan justifikasi pada Bagian 3.6.1 sebagai pengganti lapisan *biLSTM*. Pilihan tersebut menyederhanakan jalur ekspor ke ONNX dan inferensi pada server FastAPI tanpa kehilangan kemampuan kalibrasi probabilitas yang justru ditingkatkan melalui Posterior Gaussian Sampling.

2.25.6 Ringkasan Perbandingan

Tabel 2.3 merangkum perbedaan antara penelitian ini dengan pekerjaan-pekerjaan sebelumnya pada beberapa dimensi yang relevan.

Tabel 2.3. Perbandingan penelitian ini dengan pekerjaan multimodal terdahulu.

Penelitian	Modalitas	Encoder	Skema Fusi	Classifier Head	Domain
Radford et al. (2021)	Citra + teks	ViT + Trans-former	Kontrastif	Linear/MLP zero-shot	Visual umum
Audebert et al. (2019)	Citra + teks OCR	MobileNetV2 + FastText	Konkatenasi	MLP	Klasifikasi dokumen
Ofli et al. (2020)	Citra + teks	VGG16 + GloVe-CNN	Konkatenasi	FC layer	Tweet bencana
Alam et al. (2018)	Citra + teks	Dataset benchmark	Tidak ada	Tidak ada	Tweet bencana
Koshy and Elango (2023)	Citra + teks	ViT + RoBERTa	Bidirectional attention	biLSTM softmax	Tweet bencana
Penelitian ini	Citra + teks	DINOv3 large + multilingual-E5 large	Konkatenasi (early fusion)	CatBoost + PGS	Laporan warga Jakarta CRM

Bab ini telah memaparkan landasan teoretis yang menopang penelitian ini, mulai dari konteks *smart city* dan partisipasi publik, akuisisi data melalui *web scraping*, kerangka AI/ML/DL, pra-proses citra dan teks, ekstraksi fitur dan reduksi dimensi melalui PCA, mekanisme *attention* dan *Transformer*, arsitektur *backbone* visual modern (DINOv3, EVA-02, Hiera), model *encoder* teks Bahasa Indonesia dan

multibahasa (BERT, IndoBERT, Cendol-mT5, BGE-M3, multilingual-E5), skema fusi multimodal, ekosistem ONNX dan *ONNX Runtime*, perbandingan inferensi *on-device* versus *on-server*, permodelan UML pada pengembangan aplikasi, lanskap aplikasi mobile dengan Flutter, basis data *cloud* Supabase, kontainerisasi Docker, *web framework* FastAPI, metrik evaluasi, teknik pelatihan, hingga penelitian terkait yang menjadi titik berangkat metodologis. Bab Metode Penelitian berikutnya akan memanfaatkan landasan ini untuk mendefinisikan rancangan eksperimen, prosedur ekspor model, dan rancangan sistem secara konkret.

BAB 3

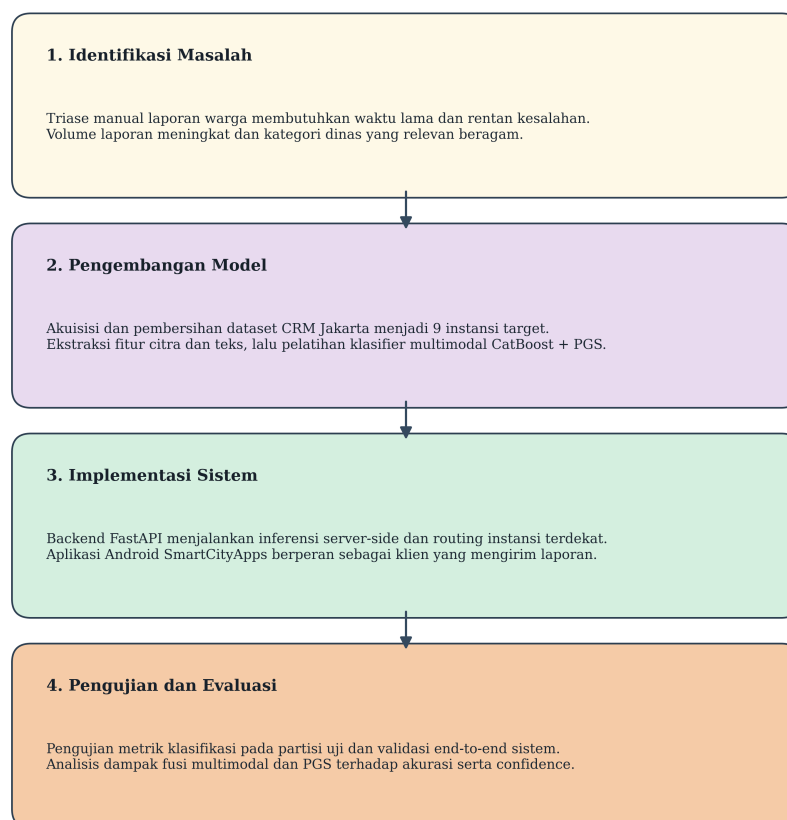
METODE PENELITIAN

Bab ini memaparkan metodologi penelitian yang digunakan untuk membangun sistem pelaporan masalah lingkungan berbasis fusi multimodal citra dan teks. Pemaparan diawali dengan diagram alir kerangka berpikir yang menyatukan seluruh tahap penelitian, dilanjutkan dengan analisis kebutuhan, akuisisi dan persiapan dataset Jakarta CRM, lingkungan komputasi pelatihan, pra-proses dan ekstraksi *embedding*, fusi multimodal, algoritma klasifikasi CatBoost dengan *Posterior Gaussian Sampling* (PGS), rancangan eksperimen komparatif, metrik evaluasi, ekspor ONNX, perancangan server FastAPI, perancangan aplikasi Flutter SmartCityApps beserta UML, mekanisme perutean instansi terdekat, dan perancangan layar. Pendekatan fusi awal yang diadopsi merujuk pada metodologi yang dipublikasikan oleh [Hanif et al. \(2024\)](#).

3.1 Diagram Alir Kerangka Berpikir

Penelitian ini disusun ke dalam tiga tahap besar yang saling berkesinambungan, yaitu (1) tahap akuisisi dan persiapan data, (2) tahap pelatihan model, dan (3) tahap *deployment*. Diagram alir sederhana pada Gambar 3.1 merangkum alur keseluruhan dari perolehan data mentah, pelatihan model 9 instansi, *deployment* inferensi pada backend FastAPI, hingga aplikasi Android SmartCityApps yang berinteraksi dengan pengguna akhir sebagai klien.

Kerangka Berpikir Penelitian SmartCityApps



Gambar 3.1. Diagram alir kerangka berpikir penelitian secara keseluruhan.

3.1.1 Kerangka Berpikir Tahap Akuisisi Data

Tahap akuisisi data mencakup proses *web scraping* portal aduan Jakarta CRM, pembersihan dan normalisasi label dinas, pemetaan label mentah ke sembilan kategori target, serta pembagian data secara stratifikasi. Diagram alir tahap ini ditunjukkan pada Gambar 3.2.

3.1.2 Kerangka Berpikir Tahap Pelatihan Model

Tahap pelatihan model dilakukan di lingkungan komputasi awan (*cloud*) RunPod yang diakses melalui SSH. Tahap ini mencakup ekstraksi *embedding* citra dengan empat *encoder* kandidat, ekstraksi *embedding* teks dengan empat *encoder* kandidat, fusi awal melalui konkatenasi vektor ter-normalisasi L2, pelatihan algoritma CatBoost dengan PGS, dan evaluasi pada metrik *macro F1-score*. Karena alur pelatihan terdiri atas dua fase yang panjang, diagram aktivitas dibagi menjadi dua bagian, yaitu fase persiapan (Gambar 3.3) dan fase eksperimen (Gambar 3.4).

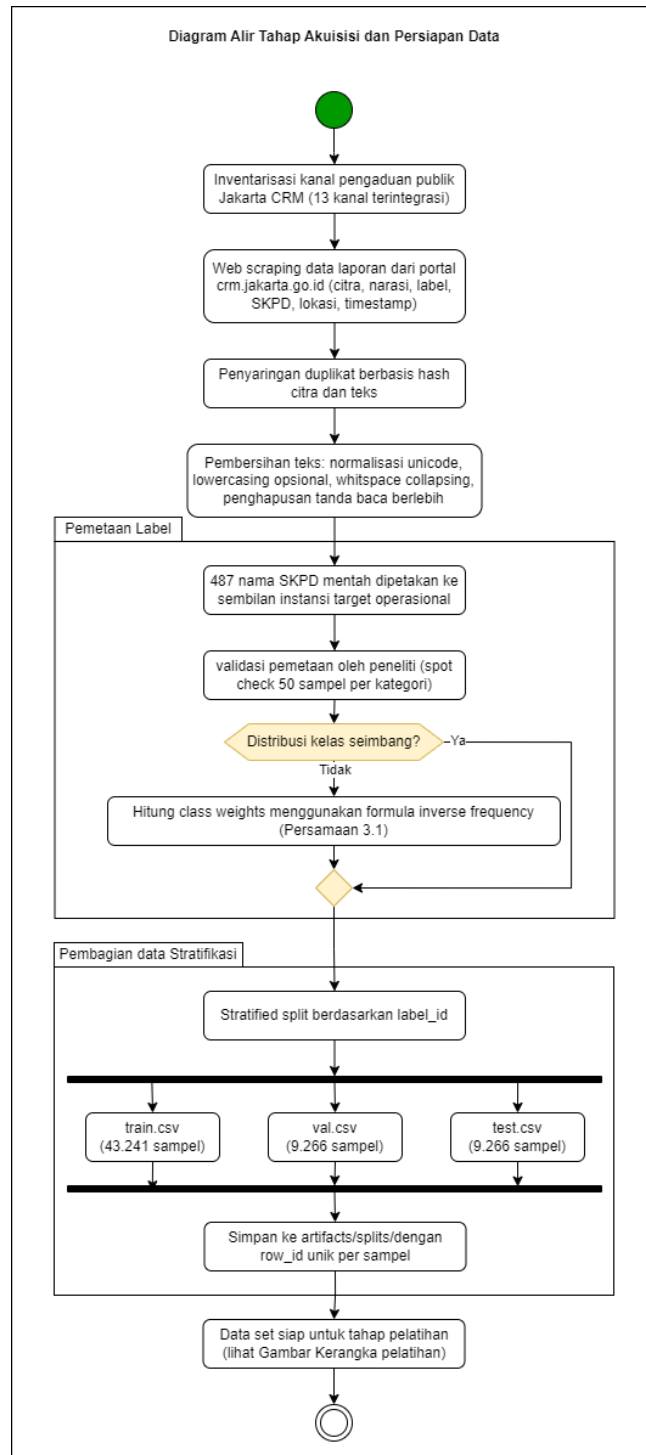
3.1.3 Kerangka Berpikir Tahap Deployment

Tahap *deployment* mencakup ekspor model ke format ONNX, pembungkusan ke kontainer Docker, peluncuran server FastAPI, integrasi ke aplikasi Flutter SmartCityApps, dan perutean instansi terdekat berdasarkan koordinat laporan. Diagram alir tahap ini ditunjukkan pada Gambar 3.5.

3.2 Analisis Kebutuhan

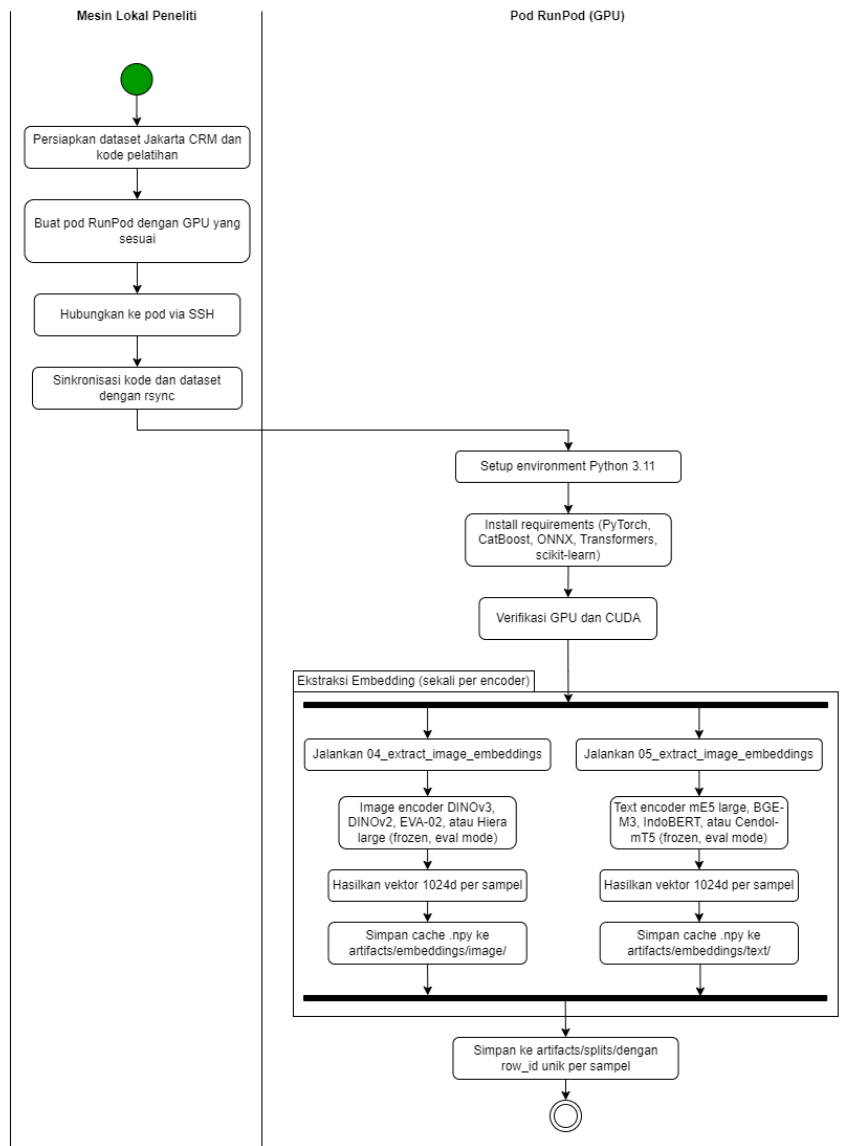
3.2.1 Analisis User

Pengguna utama sistem ini adalah masyarakat Jakarta yang berinteraksi langsung dengan aplikasi mobile untuk melaporkan masalah lingkungan, beserta operator instansi pemerintah daerah yang menerima laporan setelah perutean otomatis. Karakteristik kedua kelompok pengguna dirangkum sebagai berikut.

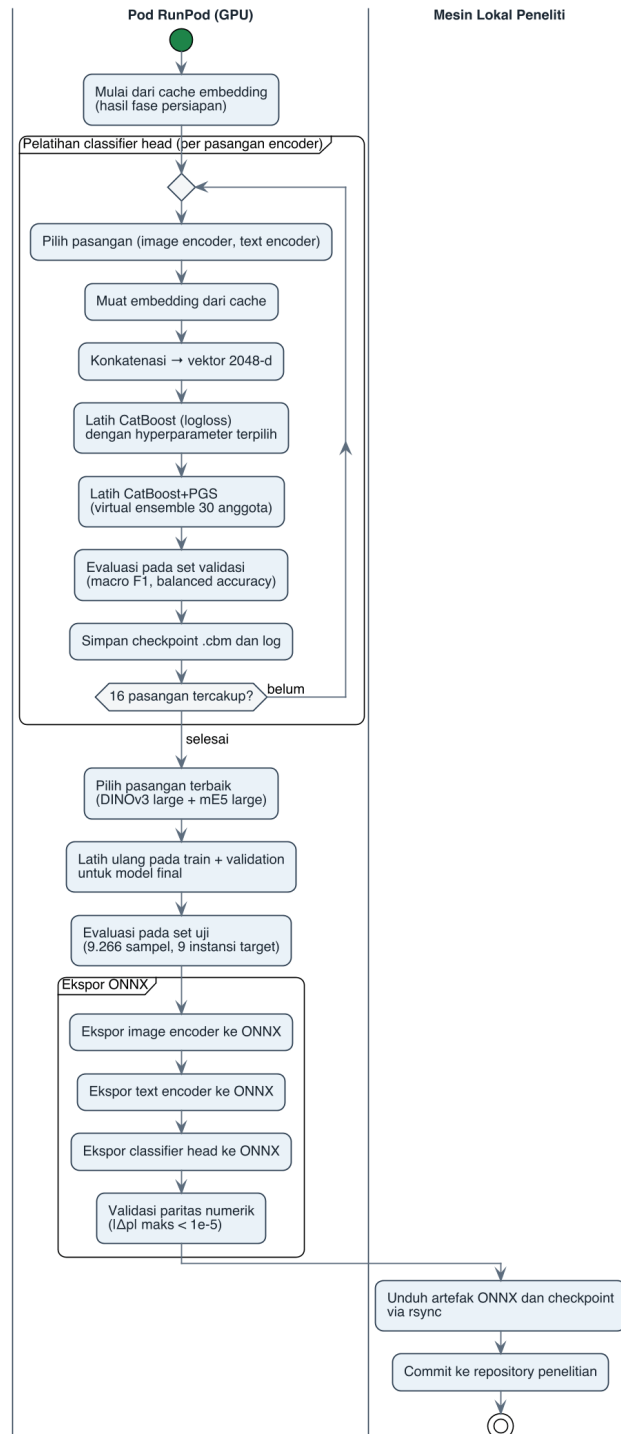


Gambar 3.2. Diagram alir tahap akuisisi data dari *web scraping* portal CRM Jakarta sampai pembagian data stratifikasi.

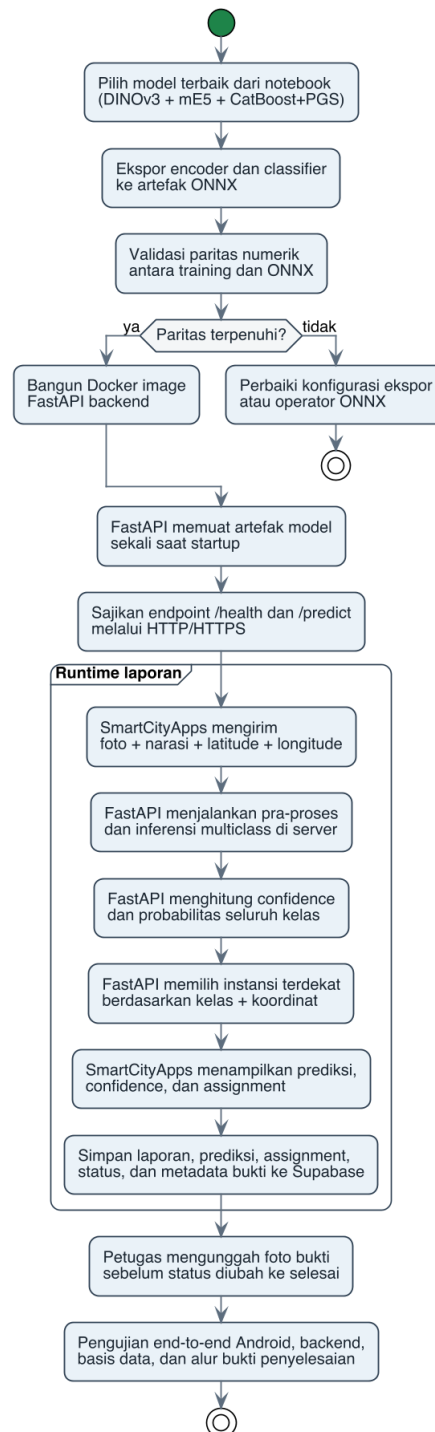
Alur Setup RunPod dan Ekstraksi Embedding (Fase Persiapan)



Gambar 3.3. Fase persiapan pelatihan: setup pod RunPod, instalasi dependensi, dan ekstraksi *embedding* citra dan teks ke berkas *cache*.



Gambar 3.4. Fase eksperimen: pelatihan *classifier head* CatBoost+PGS untuk 16 pasangan *encoder*, pemilihan model terbaik, dan ekspor ONNX dengan validasi paritas numerik.



Gambar 3.5. Diagram aktivitas tahap *deployment* mencakup ekspor ONNX, server FastAPI untuk inferensi server-side, aplikasi Android SmartCityApps sebagai klien, perutean instansi terdekat, dan pengujian sistem.

1. **Warga Pelapor:** pengguna umum yang menemukan kejadian di lingkungan kota, contohnya jalan rusak, sampah menumpuk, atau pohon tumbang. Mereka memiliki perangkat *smartphone* dengan kamera dan akses GPS, kemampuan menulis narasi singkat dalam Bahasa Indonesia, serta kebutuhan akan respons cepat dari pihak yang berwenang.
2. **Operator Instansi:** petugas dinas pemerintah daerah yang menerima laporan setelah dirutekan oleh sistem. Mereka membutuhkan laporan yang sudah terkategori dengan benar, lengkap dengan koordinat lokasi dan bukti visual, agar dapat segera ditindaklanjuti.

3.2.1.1 Latar Belakang Institusi

Penelitian ini berorientasi pada konteks tata kelola Pemerintah Provinsi DKI Jakarta yang memiliki kanal aduan publik berbasis CRM. Struktur instansi yang menjadi target perutean meliputi Dinas Bina Marga, Dinas Perhubungan, Dinas Lingkungan Hidup, Satpol PP, Dinas Pertamanan dan Hutan Kota, Badan Pembina BUMD, Dinas Cipta Karya Tata Ruang dan Pertanahan, Dinas Sumber Daya Air, dan Kelurahan, beserta kategori *Instansi Lain* sebagai kelas *catch-all* untuk laporan di luar kelas utama.

3.2.2 Analisis Aplikasi Sejenis

Beberapa aplikasi pelaporan publik yang sudah berjalan menjadi pembanding pada penelitian ini, sebagaimana dirangkum pada Tabel 3.1. Aplikasi-aplikasi tersebut secara umum sudah mendukung pelaporan berbasis foto dan teks, namun belum memanfaatkan fusi multimodal antara citra dan teks untuk mengotomatisasi kategorisasi laporan ke instansi yang tepat.

3.2.3 Rumusan dan Solusi Kebutuhan

Setiap pertanyaan penelitian (RQ) yang diajukan pada Bab 1 dipetakan ke komponen solusi yang dirancang dalam metodologi ini. Pemetaan tersebut dirangkum

Tabel 3.1. Perbandingan aplikasi pelaporan publik sejenis.

Aplikasi	Modalitas	Ma- sukan	Klasifikasi Otomatis	Catatan
JAKI (DKI Jakarta)	Foto, teks, lokasi		Manual oleh petugas	Kategori dipilih warga
LAPOR! (Nasional)	Teks, lampiran		Manual oleh <i>back office</i>	Kanal lintas instansi
Qlue	Foto, teks, lokasi		Sebagian otomatis	Klasifikasi berbasis aturan
Penelitian ini	Foto, teks, lokasi		Otomatis berbasis fusi multi-modal	DINOv3 + mE5 + CatBoost+PGS

pada Tabel 3.2.

Tabel 3.2. Pemetaan rumusan masalah ke komponen solusi.

RQ	Rumusan Masalah	Solusi yang Dirancang
RQ1	Bagaimana merancang aplikasi pelaporan multimodal?	Aplikasi Flutter dengan layar buat laporan, tinjauan AI, riwayat, dan peta.
RQ2	Bagaimana arsitektur fusi awal yang efektif?	DINOv3 + mE5 + konkatenasi 2048-d + CatBoost+PGS.
RQ3	Bagaimana ekspor ke ONNX dan server inferensi?	<i>Pipeline</i> ONNX <i>opset</i> 17 + FastAPI multipart + Docker.
RQ4	Bagaimana mekanisme perutean otomatis?	Routing instansi terdekat berdasarkan kelas prediksi 9 instansi dan koordinat latitude-longitude laporan.

3.3 Akuisisi dan Persiapan Dataset

3.3.1 Sumber Data CRM Jakarta

Dataset penelitian diperoleh dari portal aduan publik Jakarta yang berfungsi sebagai sistem CRM (*Citizen Relationship Management*). Setiap laporan disusun dari tiga komponen utama, yaitu (1) gambar lampiran sebagai bukti visual kejadian, (2) narasi tekstual dalam Bahasa Indonesia yang ditulis bebas oleh warga, dan (3) label

dinas yang menunjukkan instansi pemerintah daerah penanggung jawab. Volume data mentah hasil pengumpulan adalah **88.302 laporan** yang terdistribusi pada **487 nama instansi (SKPD)** berbeda.

3.3.2 Struktur Data Multimodal

Setiap baris dataset memuat sepasang masukan multimodal (gambar dan teks) yang berasal dari satu kejadian yang sama, beserta label dinas sebagai target klasifikasi. Satu contoh baris representatif ditampilkan pada Gambar 3.6; dua contoh tambahan dengan karakteristik berbeda (saluran air mampet dengan narasi panjang dan pembatas jalan pudar dengan permohonan pengecatan ulang) disajikan pada Lampiran B.

Gambar	Laporan	Label
	ojol parkir di area halte bus tolong dibantu tindaklanjuti laporan ini	Dinas Perhubungan

Gambar 3.6. Contoh struktur data multimodal: foto ojek parkir di area halte bus, narasi singkat warga, dan label *Dinas Perhubungan*. Tiga kolom yang ditampilkan adalah kolom *Gambar*, kolom *Laporan*, dan kolom *Label*.

Tiga kolom inti yang menjadi masukan dan keluaran sistem dirangkum sebagai berikut.

- **Kolom Gambar:** foto kejadian yang diambil oleh warga menggunakan kamera ponsel. Karakteristik visual sangat beragam, mulai dari foto siang hari beresolusi tinggi, foto malam hari dengan pencahayaan rendah, hingga foto sudut tidak biasa dengan oklusi. Resolusi piksel berkisar dari 100 hingga 4.000 piksel pada sisi terlebar.
- **Kolom Laporan:** narasi tekstual yang ditulis bebas oleh warga dalam Bahasa Indonesia. Narasi sering kali pendek dan informal, kadang memuat singkatan, sapaan, atau campuran ragam bahasa daerah. Contohnya, narasi “*ojol parkir di area halte bus tolong dibantu tindaklanjuti laporan ini*” secara semantik

merujuk pada pelanggaran parkir kendaraan di area halte bus, sehingga relevan bagi Dinas Perhubungan, sekalipun narasi tidak menyebut nama dinas tersebut.

- **Kolom Label:** nama dinas penanggung jawab yang berasal dari proses pemetaan oleh operator portal CRM. Label ini menjadi target klasifikasi multikelas pada sembilan kategori akhir setelah pemetaan (Bagian 3.3).

Sifat multimodal dan komplementer kedua modalitas terlihat pada contoh Gambar 3.6: foto memperlihatkan kendaraan terparkir di sekitar halte bus, sementara narasi memberi konteks bahwa kendaraan tersebut adalah *ojol* dan tindakannya berstatus pelanggaran. Dua contoh tambahan pada Lampiran B memperkuat pola yang sama, yaitu narasi panjang yang menjelaskan dampak banjir dari saluran air mampet (label Kelurahan) serta foto pembatas jalan pudar yang disertai permohonan pengecatan ulang (label Dinas Perhubungan). Ketiga contoh tersebut mengilustrasikan bahwa baik citra maupun teks membawa informasi yang tidak sepenuhnya saling tergantikan, sehingga fusi multimodal diperlukan untuk klasifikasi yang akurat.

3.3.3 Metadata Tambahan

Selain tiga kolom inti di atas, setiap laporan juga menyimpan metadata tambahan yang berguna untuk audit dan analisis, yaitu identitas unik laporan, koordinat lokasi (lintang dan bujur), waktu pengiriman, dan tautan ke berkas gambar. Metadata ini tidak menjadi masukan langsung bagi model klasifikasi, namun digunakan untuk pembagian data, analisis kesalahan, dan rekonstruksi konteks pada tahap evaluasi.

3.3.4 Web Scraping

Pengumpulan data dilakukan dengan teknik *web scraping* berbasis Python sesuai prinsip yang diuraikan oleh [Mitchell \(2018\)](#). Skrip *scraping* dikembangkan menggunakan kombinasi pustaka `requests` untuk pengiriman permintaan HTTP, `BeautifulSoup`

dan lxml untuk penguraian HTML, serta Selenium untuk halaman yang dirender oleh JavaScript secara dinamis. Tahap-tahap utama *scraping* adalah:

1. **Identifikasi sumber:** pemetaan URL halaman daftar laporan dan halaman rincian laporan, beserta pola *pagination*.
2. **Pengunduhan:** pengiriman permintaan GET dengan pembatasan laju (*rate limit*) maksimum satu permintaan per detik dan *user-agent* yang mengidentifikasi tujuan akademik.
3. **Penguraian:** ekstraksi medan teks laporan, label dinas, tanggal, koordinat, dan tautan gambar.
4. **Pengunduhan gambar:** penyimpanan gambar lampiran laporan ke struktur direktori berdasar label dinas.
5. **Penyimpanan:** penulisan metadata terstruktur ke berkas CSV dan unduhan gambar ke struktur direktori berbasis label dinas.
6. **Penanganan kegagalan:** *retry* dengan *exponential backoff* pada kesalahan jaringan dan *logging* permintaan yang gagal.

Scraping menghormati berkas `robots.txt` dan *Terms of Service* portal sumber, serta tidak mengumpulkan data yang dapat mengidentifikasi individu di luar narasi laporan publik.

3.3.5 Pembersihan dan Normalisasi

Tahap pembersihan dilakukan dengan langkah-langkah berikut:

- **Deduplikasi teks:** penghapusan baris dengan kolom `laporan` yang persis sama. Tahap ini menghilangkan sekitar 30% laporan yang merupakan duplikat hasil *copy-paste* (terutama dari kanal Kelurahan).
- **Normalisasi nama dinas:** penyeragaman ejaan dan kapitalisasi nama instansi yang ditulis berbeda-beda pada data sumber.

- **Pembersihan teks:** pemotongan URL via *regex* `https?://\S+|www\.\S+`, normalisasi unicode ke bentuk NFC, dan *whitespace collapsing*.
- **Validasi gambar:** pengecekan keterbacaan berkas gambar dan penghapusan baris dengan gambar rusak.

Setelah pembersihan, dataset bersih berukuran **61.773 laporan** siap dipakai untuk tahap berikutnya.

3.3.6 Pemetaan ke Sembilan Instansi Target

3.3.6.0.1 Definisi terminologi. Untuk menghindari kerancuan istilah pada bab-bab selanjutnya, ditetapkan tiga definisi berikut yang dipakai secara konsisten di seluruh manuskrip:

- **Instansi (dinas):** nama lembaga pemerintah daerah yang menjadi target klasifikasi. Sembilan instansi target pada penelitian ini adalah Dinas Bina Marga, Satuan Polisi Pamong Praja (Satpol PP), Dinas Perhubungan, Kelurahan, Dinas Pertamanan dan Hutan, Dinas Sumber Daya Air, Dinas Cipta Karya, Tata Ruang, dan Pertanahan, Badan Pembinaan BUMD, dan Instansi Lain.
- **Instansi prediksi** (`predicted_dinas`): keluaran langsung model klasifikasi multimodal berupa salah satu dari sembilan instansi di atas, beserta probabilitas masing-masing kelas.
- **Instansi tujuan terdekat:** hasil tahap routing setelah inferensi. Backend menggunakan `predicted_dinas` serta koordinat latitude dan longitude laporan untuk memilih kantor atau unit instansi aktif yang paling dekat dan relevan. Tahap ini **bukan** agregasi label model; model tetap menghasilkan keluaran pada granularitas sembilan instansi penuh (lihat `TARGET_CLASSES` pada `src/crm/__init__.py`).

Istilah “kategori” dan “kelas” dipakai hanya pada konteks teori klasifikasi (*multi-class problem, class weight, per-class precision*), bukan sebagai sinonim “instansi”.

Pada bab Hasil dan Pembahasan istilah **instansi prediksi** digunakan untuk merujuk keluaran model dan **instansi tujuan terdekat** untuk merujuk hasil perutean berbasis lokasi.

Karena 487 SKPD mentah terlalu *fine-grained* untuk klasifikasi *multi-class* dengan jumlah sampel yang tidak seimbang, label dinas mentah dipetakan ke sembilan instansi target yang lebih representatif dan operasional. Pemetaan dan distribusi sampel per instansi dirangkum pada Tabel 3.3.

Tabel 3.3. Distribusi sampel per instansi target setelah pemetaan.

ID	Kategori (Dinas)	Jumlah	Persen
0	Dinas Bina Marga	13.525	21,9%
1	Dinas Perhubungan (termasuk UP Perparkiran, UPSL)	11.678	18,9%
2	Kelurahan	11.980	19,4%
3	Satuan Polisi Pamong Praja	8.405	13,6%
4	Dinas Pertamanan dan Hutan Kota	6.000	9,7%
5	Dinas Sumber Daya Air	2.605	4,2%
6	Badan Pembina BUMD (TransJakarta, Pasar Jaya, PAM Jaya)	1.684	2,7%
7	Dinas Cipta Karya Tata Ruang dan Pertanahan	1.475	2,4%
8	Instansi Lain (<i>catch-all</i>)	4.421	7,2%
Total		61.773	100%

3.3.7 Pembagian Data Stratifikasi

Dataset bersih dibagi ke dalam tiga partisi (*train*, *validation*, *test*) dengan rasio **70:15:15**, menggunakan fungsi `StratifiedShuffleSplit` dari pustaka `scikit-learn` untuk menjaga proporsi setiap kelas pada masing-masing partisi. Hasil pembagian dirangkum pada Tabel 3.4, dan diverifikasi bahwa selisih proporsi kelas antar partisi tidak melampaui 0,01%.

Setiap baris pada partisi diberi anotasi `row_id` unik agar *cache embedding* da-

Tabel 3.4. Pembagian dataset stratifikasi.

Partisi	Jumlah Sampel	Persentase
Train	43.241	70%
Validation	9.266	15%
Test	9.266	15%
Total	61.773	100%

pat disejajarkan kembali dengan label saat pelatihan algoritma klasifikasi.

3.3.8 Bobot Kelas

Karena distribusi kelas tidak seimbang, pelatihan algoritma CatBoost menggunakan *class weights* berdasarkan formula yang juga dipakai oleh [Hanif et al. \(2024\)](#):

$$w_l = \frac{N}{C \cdot N_l}, \quad (3.1)$$

dengan N total sampel pelatihan, $C = 9$ jumlah kelas, dan N_l jumlah sampel kelas ke- l . Bobot ini dipakai sebagai parameter bobot kelas pada CatBoost untuk mengompensasi kelas minoritas.

3.4 Lingkungan Komputasi: Pelatihan di Cloud via RunPod

3.4.1 Justifikasi Cloud Compute

Ekstraksi *embedding* dari empat *encoder* citra besar dan empat *encoder* teks besar pada 61.773 sampel membutuhkan komputasi GPU yang berkapasitas. Sebagai contoh, *forward pass* EVA-02 *Large* pada masukan 448x448 piksel memerlukan memori GPU yang melampaui 12 GB pada *batch size* efektif. Eksekusi pada perangkat lokal kelas *laptop* tidak memungkinkan, sehingga seluruh pelatihan dilakukan pada layanan komputasi awan RunPod yang menyediakan akses langsung ke GPU NVIDIA modern.

3.4.2 Konfigurasi Pod RunPod

Konfigurasi *pod* yang digunakan dirangkum pada Tabel 3.5. Pemilihan RTX 4090 didasarkan pada kombinasi VRAM 24 GB yang memadai untuk batch ekstraksi *embedding* dan biaya per jam yang ekonomis dibanding kelas A100.

Tabel 3.5. Konfigurasi *pod* RunPod untuk pelatihan dan ekstraksi *embedding*.

Komponen	Spesifikasi
GPU	NVIDIA GeForce RTX 4090 (24 GB VRAM)
CPU	vCPU 8 <i>core</i> (AMD EPYC)
RAM	32 GB
<i>Container image</i>	runpod/pytorch:2.1.0-py3.10-cuda12.1.1
Penyimpanan	50 GB <i>persistent volume</i>
Mode jaringan	SSH <i>public key authentication</i>
Port <i>exposed</i>	22 (SSH), 8888 (Jupyter)

3.4.3 Workflow Pelatihan dan Manajemen Lingkungan

Akses ke *pod* dilakukan melalui kunci publik SSH yang didaftarkan pada akun RunPod. *Workflow* pelatihan mencakup sinkronisasi dataset dengan `rsync`, pemasangan dependensi (`torch`, `transformers`, `timm`, `catboost`, `onnxruntime-gpu`), eksekusi notebook melalui *SSH tunnel* ke Jupyter Lab, penyimpanan *artifact* pada *persistent volume*, dan pengunduhan hasil ke mesin lokal. Lingkungan direproduksi dari berkas konfigurasi dengan versi dependensi presisi, dan *forward pass encoder* memakai presisi campuran FP16 dengan ukuran *batch* yang disesuaikan per *encoder* (misalnya 32 untuk DINOv3 pada 224x224 dan 8 untuk EVA-02 pada 448x448). Konteks visual alur kerja ini sudah disajikan pada Gambar 3.3, sedangkan rincian langkah SSH serta praktik manajemen lingkungan secara penuh disajikan pada Lampiran E.

3.5 Pemrosesan, Ekstraksi *Embedding*, dan Fusi Dini

Tahap pemrosesan data multimodal, ekstraksi *embedding*, dan fusi dini menyiapkan masukan tunggal berdimensi 2048 untuk *classifier head* CatBoost. Pada sisi citra, setiap gambar dibaca dan diorientasikan ulang sesuai EXIF, kemudian di-*resize* dengan *square padding* ke 224x224 (atau 448x448 untuk EVA-02) mengikuti *AutoImageProcessor* bawaan DINOv3, lalu dinormalisasi per kanal dengan rerata serta simpangan baku ImageNet sesuai Persamaan F.1. Augmentasi tidak diterapkan karena *encoder* dibekukan. Pada sisi teks, narasi dipangkas spasinya dan dideduplikasi (penghapusan teks identik pada tahap pembersihan dataset), dipotong ke 256 token (mencakup sebagian besar laporan tanpa pemotongan), diberi prefiks instruksi untuk varian mE5-Instruct, ditokenisasi dengan tokenizer bawaan tiap *encoder*, lalu *embedding* keluarannya dinormalisasi L2.

Ekstraksi *embedding* dilakukan dengan empat kandidat *encoder* citra (DINOv2-L, DINOv3-ViT-L, Hiera-L, EVA-02-L) dan empat kandidat *encoder* teks (mE5-Large-Instruct, BGE-M3, IndoBERT-Large-p2, Cendol-mT5-Large), seluruhnya menghasilkan vektor berdimensi 1024. Inferensi dilakukan secara *batch* dengan presisi campuran FP16 dan hasilnya disimpan pada *cache* terkompresi sehingga eksperimen ablasi pasangan *encoder* dapat dilakukan tanpa menjalankan ulang *encoder*.

Skema fusi yang digunakan adalah konkatenasi langsung dua vektor *embedding* yang sudah dinormalisasi L2 per modalitas, mengikuti pendekatan [Hanif et al. \(2024\)](#), menghasilkan vektor fusi berdimensi 2048 yang menjadi masukan tunggal CatBoost. Reduksi dimensi (PCA) tidak diterapkan karena CatBoost berbasis pohon *oblivious* mampu menangani fitur padat berdimensi tinggi serta untuk menghindari hilangnya informasi diskriminatif. Rincian lengkap berupa urutan langkah pemrosesan, tabel kandidat *encoder*, formula normalisasi L2, dan persamaan vektor fusi disajikan pada Lampiran C.

3.6 Algoritma Klasifikasi: CatBoost dengan PGS

3.6.1 Justifikasi Pemilihan CatBoost sebagai *Classifier Head*

Pemilihan CatBoost sebagai *classifier head* pada *pipeline* fusi awal penelitian ini didasarkan pada delapan properti yang selaras dengan kebutuhan teknis dan operasional sistem, yaitu: (1) *ordered boosting* (Prokhorenkova et al., 2018) yang memitigasi *target leakage* sehingga generalisasi pada dataset berukuran sedang (61.773 sampel) lebih stabil; (2) pohon simetris (*oblivious trees*) yang membuat inferensi cepat dan ramah *cache* CPU pada server FastAPI; (3) ketahanan empiris pada masukan tabular berdimensi tinggi seperti vektor fusi 2048 dimensi hasil konkatenasi *embedding* DINOv3 dan *multilingual-E5*; (4) kalibrasi probabilitas melalui Posterior Gaussian Sampling (Ustimenko et al., 2023) yang sekaligus memberikan ukuran ketidakpastian untuk antarmuka pengguna; (5) pelatihan deterministik dengan *random seed* eksplisit sehingga ablasi dapat direproduksi dan diaudit; (6) jalur ekspor resmi ke ONNX dengan paritas numerik tinggi terhadap implementasi *native*, selaras dengan *deployment* server FastAPI; (7) beban komputasi pelatihan yang ringan (satu *classifier head* selesai dalam hitungan menit), sehingga enam belas kombinasi ablasi dapat dijalankan dalam satu sesi karena kedua *encoder* dibekukan; dan (8) penanganan fitur kategoris *native* via *ordered target statistics* untuk perluasan masa depan, misalnya menambahkan fitur lokasi atau waktu pelaporan tanpa mengganti algoritma. Penelitian ini menggunakan CatBoost sebagai satu-satunya algoritma klasifikasi; eksplorasi konfigurasi (CatBoost dengan dan tanpa PGS) didokumentasikan pada Bagian 3.7.

3.6.2 Hyperparameter

Hyperparameter CatBoost yang dipakai dirangkum pada Tabel 3.6.

Tabel 3.6. Hyperparameter CatBoost.

Parameter	Nilai
Jumlah iterasi	1500
Kedalaman pohon	6
<i>Learning rate</i>	0,05
Fungsi tujuan	<i>MultiClass logloss</i>
Regularisasi L2 daun	3,0
Bobot kelas	sesuai Persamaan 3.1
<i>Posterior sampling</i>	Aktif
<i>Early stopping rounds</i>	50 (pada validasi)
<i>Task type</i>	CPU (atau GPU bila tersedia)
<i>Random seed</i>	42

3.6.3 Posterior Gaussian Sampling (PGS)

PGS adalah teknik kuantifikasi ketidakpastian pada *gradient boosting* yang diperkenalkan oleh [Ustimenko et al. \(2023\)](#). Saat pelatihan dengan opsi *posterior sampling* aktif, CatBoost membangun *virtual ensembles* yang dapat dipanggil saat inferensi dengan jumlah anggota ensemble virtual sebanyak 30. Setiap anggota menghasilkan distribusi probabilitas *softmax* per kelas. Probabilitas akhir adalah rerata posterior:

$$\bar{p}_k = \frac{1}{M} \sum_{m=1}^M p_k^{(m)}, \quad (3.2)$$

dengan $M = 30$ anggota ensemble dan $p_k^{(m)}$ probabilitas kelas k dari anggota ensemble ke- m .

Ketidakpastian *epistemic* diestimasi dengan *mutual information* antara prediksi dan parameter model, sebagaimana selisih *entropy of mean* dan *mean of entropy*:

$$\mathcal{U}_{\text{epistemic}} = H(\bar{\mathbf{p}}) - \frac{1}{M} \sum_{m=1}^M H(\mathbf{p}^{(m)}), \quad (3.3)$$

dengan $H(\mathbf{p}) = -\sum_k p_k \log p_k$ adalah entropi Shannon. Nilai $\mathcal{U}_{\text{epistemic}}$ yang tinggi

menandakan model tidak yakin terhadap prediksi karena ketidakpastian parameter, sehingga laporan dengan nilai tinggi dapat dirutekan ke tinjauan manual.

3.6.4 Pelatihan dengan Validation Set dan Early Stopping

Pelatihan dilakukan pada partisi *train* (43.241 sampel) dengan validasi pada partisi *validation* (9.266 sampel). Mekanisme *early stopping* dengan ambang 50 iterasi memerintahkan pelatihan berhenti apabila *macro F1-score* pada set validasi tidak meningkat selama 50 iterasi berturut-turut, sehingga risiko *overfitting* tertekan dan jumlah pohon optimal ditemukan secara otomatis.

3.6.5 Pemilihan Model Terbaik

Model terbaik dipilih berdasarkan *macro F1-score* pada partisi validasi. *Checkpoint* terbaik disimpan dalam format *native* CatBoost dan kemudian diekspor ke ONNX untuk *deployment*.

3.7 Eksperimen Komparatif (Ablasi)

Bagian ini mendokumentasikan rancangan eksperimen komparatif yang dijalankan untuk memilih kombinasi *encoder* terbaik dan untuk mengevaluasi kontribusi PGS pada algoritma CatBoost.

3.7.1 Ablasi Matriks Multimodal

Penelitian ini menjalankan eksperimen matriks penuh, yaitu seluruh pasangan kombinasi antara empat *encoder* citra (DINOv2-L, DINOv3-ViT-L, Hiera-L, EVA-02-L) dan empat *encoder* teks (mE5-Large-Instruct, BGE-M3, IndoBERT-Large-p2, Cendol-mT5-Large), sehingga total 16 kombinasi. Setiap kombinasi dilatih dengan konfigurasi CatBoost yang identik. Tujuan ablasi matriks ini adalah memetakan kontribusi setiap *encoder* terhadap akurasi akhir, sekaligus memilih pasangan terbaik untuk *deployment*.

3.7.2 Ablasi PGS On/Off

Pengaruh *Posterior Gaussian Sampling* dievaluasi dengan menjalankan setiap kombinasi pada matriks ablasi dengan dua varian, yaitu CatBoost *baseline* tanpa PGS dan CatBoost dengan PGS aktif beserta 30 anggota ensemble virtual saat inferensi. Untuk setiap pasangan dicatat *accuracy*, *macro F1-score*, selisih relatif ($\Delta F1$, ΔAcc), dan rerata *epistemic uncertainty*. Tujuan ablasi ini adalah memverifikasi bahwa PGS memberikan peningkatan yang konsisten lintas kombinasi *encoder*, di samping memberikan keluaran ketidakpastian yang dapat diaudit.

3.7.3 Tabel Rancangan Eksperimen

Matriks eksperimen lengkap dirangkum pada Tabel 3.7. Setiap baris mewakili satu kombinasi (*encoder* citra, *encoder* teks) yang dievaluasi dua kali (*baseline* dan PGS).

3.8 Metrik Evaluasi

Metrik utama dan pendukung yang dipakai untuk mengevaluasi setiap eksperimen sudah didefinisikan secara teoretis pada Bagian 2.23. Pada metodologi ini metrik yang dilaporkan untuk setiap eksperimen adalah:

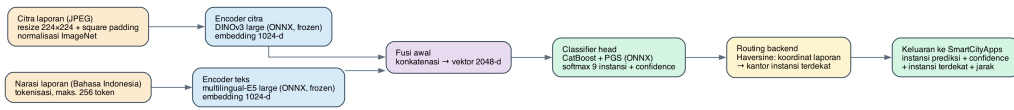
1. **Macro F1-score**, sebagaimana Persamaan 2.12, sebagai metrik utama pemilihan model.
2. **Akurasi** dan **Balanced Accuracy** sebagai indikator umum.
3. **Per-Class F1, Precision, Recall** untuk setiap dari sembilan kategori, agar bias terhadap kelas mayoritas teridentifikasi.
4. **Confusion Matrix** berukuran 9×9 , divisualisasikan sebagai *heatmap*.
5. **Macro ROC-AUC** dengan skema *one-vs-rest*.

Tabel 3.7. Matriks rancangan eksperimen ablasi (4 *encoder* citra $\times$ 4 *encoder* teks $\times$ 2 varian *head*).

ID	Encoder Citra	Encoder Teks	Head
E01	DINOv2-L	mE5-Large-Instruct	CatBoost / Cat-Boost+PGS
E02	DINOv3-ViT-L	mE5-Large-Instruct	CatBoost / Cat-Boost+PGS
E03	Hiera-L	mE5-Large-Instruct	CatBoost / Cat-Boost+PGS
E04	EVA-02-L	mE5-Large-Instruct	CatBoost / Cat-Boost+PGS
E05	DINOv2-L	BGE-M3	CatBoost / Cat-Boost+PGS
E06	DINOv3-ViT-L	BGE-M3	CatBoost / Cat-Boost+PGS
E07	Hiera-L	BGE-M3	CatBoost / Cat-Boost+PGS
E08	EVA-02-L	BGE-M3	CatBoost / Cat-Boost+PGS
E09	DINOv2-L	IndoBERT-Large-p2	CatBoost / Cat-Boost+PGS
E10	DINOv3-ViT-L	IndoBERT-Large-p2	CatBoost / Cat-Boost+PGS
E11	Hiera-L	IndoBERT-Large-p2	CatBoost / Cat-Boost+PGS
E12	EVA-02-L	IndoBERT-Large-p2	CatBoost / Cat-Boost+PGS
E13	DINOv2-L	Cendol-mT5-Large	CatBoost / Cat-Boost+PGS
E14	DINOv3-ViT-L	Cendol-mT5-Large	CatBoost / Cat-Boost+PGS
E15	Hiera-L	Cendol-mT5-Large	CatBoost / Cat-Boost+PGS
E16	EVA-02-L	Cendol-mT5-Large	CatBoost / Cat-Boost+PGS

6. **Analisis ketidakpastian:** distribusi $\mathcal{U}_{\text{epistemic}}$ pada partisi uji, dan korelasi antara nilai ketidakpastian dengan kesalahan prediksi.

Keseluruhan alur inferensi yang menghasilkan metrik-metrik di atas, mulai dari citra dan teks hingga prediksi sembilan instansi, *confidence*, dan routing instansi terdekat, ditunjukkan pada Gambar 3.7. *Confusion matrix* aktual berukuran 9×9 pada konfigurasi terbaik disajikan pada Bab 4.



Gambar 3.7. Alur inferensi multimodal server-side dari citra dan teks ke prediksi 9 instansi, confidence, dan routing instansi terdekat berdasarkan koordinat.

3.9 Pipeline Ekspor ONNX

Pipeline ekspor menghasilkan tiga *artifact* ONNX terpisah yang dimuat oleh server FastAPI saat *startup*.

3.9.1 Ekspor Encoder Citra

Encoder citra terbaik diekspor melalui kombinasi `torch.jit.trace` dan `torch.onnx.export` dengan *opset* versi 17. Penggunaan `torch.jit.trace` diperlukan karena beberapa varian *encoder*, khususnya DINOv3, belum sepenuhnya didukung oleh *exporter* otomatis. Berkas hasil disimpan beserta konfigurasi pra-proses (*mean*, *std*, ukuran masukan) dalam berkas pendamping *JSON*.

3.9.2 Ekspor Encoder Teks

Encoder teks terbaik diekspor menggunakan utilitas ekspor ONNX yang menangani modul *attention mask* dan SDPA secara benar. Bobot model yang melampaui ukuran maksimum berkas tunggal ONNX disimpan sebagai bobot eksternal. Tokenizer dikemas sebagai *artifact* pendamping pada direktori yang sama dengan model.

3.9.3 Ekspor CatBoost

Wight CatBoost diekspor menggunakan metode `native model.save_model("classifier.format="onnx")` yang menulis *operator domain ai.catboost*. Inferensi pada saat produksi dijalankan dengan `Execution Provider CPU` karena *operator* CatBoost belum memiliki kernel CUDA pada *ONNX Runtime*.

3.9.4 Prosedur Validasi Paritas Numerik

Validasi paritas numerik dirancang untuk memastikan bahwa model ONNX hasil ekspor menghasilkan keluaran yang konsisten dengan model PyTorch sumber. Prosedurnya adalah sebagai berikut.

1. Pilih satu *batch* sampel dari partisi uji sebagai masukan referensi.
2. Jalankan model PyTorch sumber pada *batch* tersebut dan simpan vektor keluaran sebagai *baseline*.
3. Jalankan model ONNX pada *batch* yang sama melalui *ONNX Runtime* dengan *Execution Provider* yang sesuai.
4. Hitung selisih absolut maksimum (*max abs diff*) per komponen vektor keluaran antara kedua model.
5. Tetapkan ambang batas 10^{-5} sebagai kriteria lulus, yang memastikan prediksi top-1 identik antara kedua jalur.
6. Catat hasil paritas untuk setiap komponen, yaitu *encoder* citra, *encoder* teks, dan algoritma CatBoost.

Laporan numerik hasil validasi paritas akan disajikan pada Bab Hasil dan Pembahasan.

3.10 Perancangan Sistem Server (FastAPI)

3.10.1 Arsitektur Server

Server inferensi membungkus tiga sesi *ONNX Runtime* (*encoder* citra, *encoder* teks, algoritma CatBoost) yang dimuat sekali pada saat *startup*, kemudian menyediakan metode prediksi yang menjalankan *pipeline* fusi awal pada masukan gabungan citra dan teks. Aplikasi Android SmartCityApps berperan sebagai *client* untuk mengirimkan laporan, foto, serta koordinat latitude dan longitude. Proses inferensi model dilakukan pada sisi server melalui backend FastAPI, bukan pada perangkat Android. Diagram komponen server ditunjukkan pada Gambar 3.8.

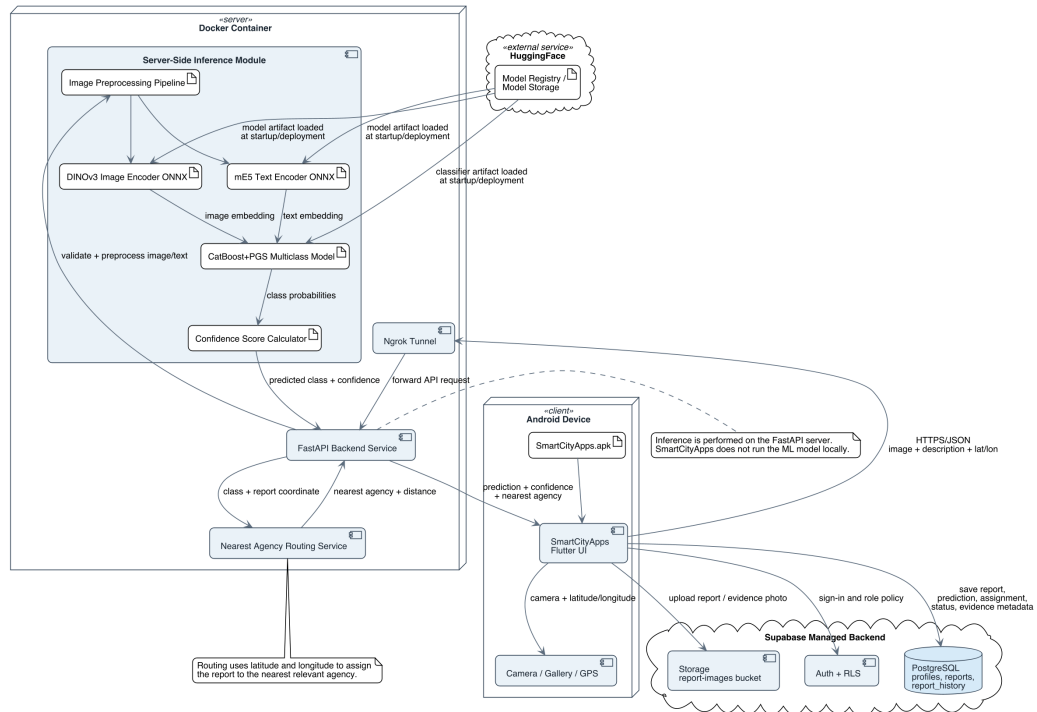
Model yang digunakan dapat disimpan sebagai artefak pada server atau pada *model registry* seperti HuggingFace. Namun, HuggingFace pada rancangan ini diperlakukan sebagai penyimpanan atau registri artefak model, bukan sebagai layanan yang menjalankan inferensi. Dengan pemisahan ini, pembaruan model dapat dilakukan pada sisi server tanpa mewajibkan pengguna memperbarui aplikasi Android.

3.10.2 Endpoint `/health` dan `/predict`

Server menyediakan dua *endpoint* utama, yaitu `/health` (GET) sebagai *health check* ringan dan `/predict` (POST *multipart*) sebagai *endpoint* klasifikasi. *Endpoint* `/predict` menerima foto laporan, teks laporan, latitude, dan longitude. Spesifikasi *endpoint* dirangkum pada Tabel 3.8.

3.10.3 Skema Request dan Response

Skema respons `PredictResponse` dideklarasikan sebagai kelas Pydantic dengan medan: `predicted_dinas` (string), `predicted_dinas_id` (int), `confidence` (float), `all_probabilities` (peta instansi ke skor probabilitas), `model_name`, `model_version`, `confidence_threshold`, `review_required`, dan `assignment`. Medan `assignment` memuat `agency_id`, `agency_name`, `agency_category`,



Gambar 3.8. Diagram komponen dan deployment SmartCityApps: klien Android, backend FastAPI untuk inferensi server-side, HuggingFace sebagai model registry/storage, dan Supabase.

Tabel 3.8. Spesifikasi *endpoint* server FastAPI.

Path	Method	Masukan	Keluaran
/health	GET	(tidak ada)	JSON {status, encoders, gpu}
/predict	POST	<i>multipart:</i> image (file), laporan (string), latitude, longitude	JSON PredictResponse berisi prediksi, confidence, probabilitas per kelas, dan assignment instansi terdekat

`distance_meters`, dan `routing_method`. Validasi otomatis dijamin oleh FastAPI sehingga klien Flutter dapat mengandalkan struktur respons.

3.10.4 Output Inferensi dan Confidence Level

Output proses inferensi terdiri atas kelas prediksi, nilai *confidence*, dan skor probabilitas untuk setiap dari sembilan instansi target. Kelas prediksi menunjukkan instansi yang paling mungkin bertanggung jawab terhadap laporan, sedangkan *confidence* menunjukkan tingkat keyakinan model terhadap kelas tersebut. Nilai ini dihitung dari probabilitas kelas tertinggi pada keluaran model multiclass.

Nilai *confidence* digunakan sebagai indikator pendukung pengambilan keputusan. Apabila nilai *confidence* berada di atas ambang yang ditentukan, laporan dapat diarahkan otomatis ke instansi terdekat yang relevan dengan kelas prediksi. Apabila *confidence* berada di bawah ambang, laporan tetap disimpan bersama skor probabilitasnya, tetapi ditandai untuk validasi manual oleh petugas agar risiko kesalahan klasifikasi tidak disembunyikan.

3.10.5 Dockerization

Server dikemas dalam kontainer Docker yang mengikuti pola *multi-stage build*. *Base image* berbasis CUDA 12.1 dengan *cuDNN*, lapisan instalasi Python 3.11 dan dependensi inferensi (`onnxruntime-gpu`, `fastapi`, `uvicorn`, `python-multipart`), serta lapisan penyalinan kode dan *artifact* ONNX. *Entrypoint* menjalankan `uvicorn` pada `host 0.0.0.0 port 8000` dengan satu *worker*. Berkas Docker Compose mendefinisikan layanan *server* dengan reservasi GPU dan *volume read-only* untuk *artifact* model.

3.11 Perancangan Skema Basis Data (Supabase)

Basis data *cloud* aplikasi diimplementasikan pada Supabase yang berbasis PostgreSQL. Skema basis data dirancang untuk menyimpan profil pengguna (`profiles`), laporan warga (`reports`), riwayat perubahan status laporan (`report.history`),

metadata prediksi multiclass, hasil penugasan instansi terdekat, dan bukti penyelesaian laporan. Skema ER lengkap ditunjukkan pada Gambar 3.9.

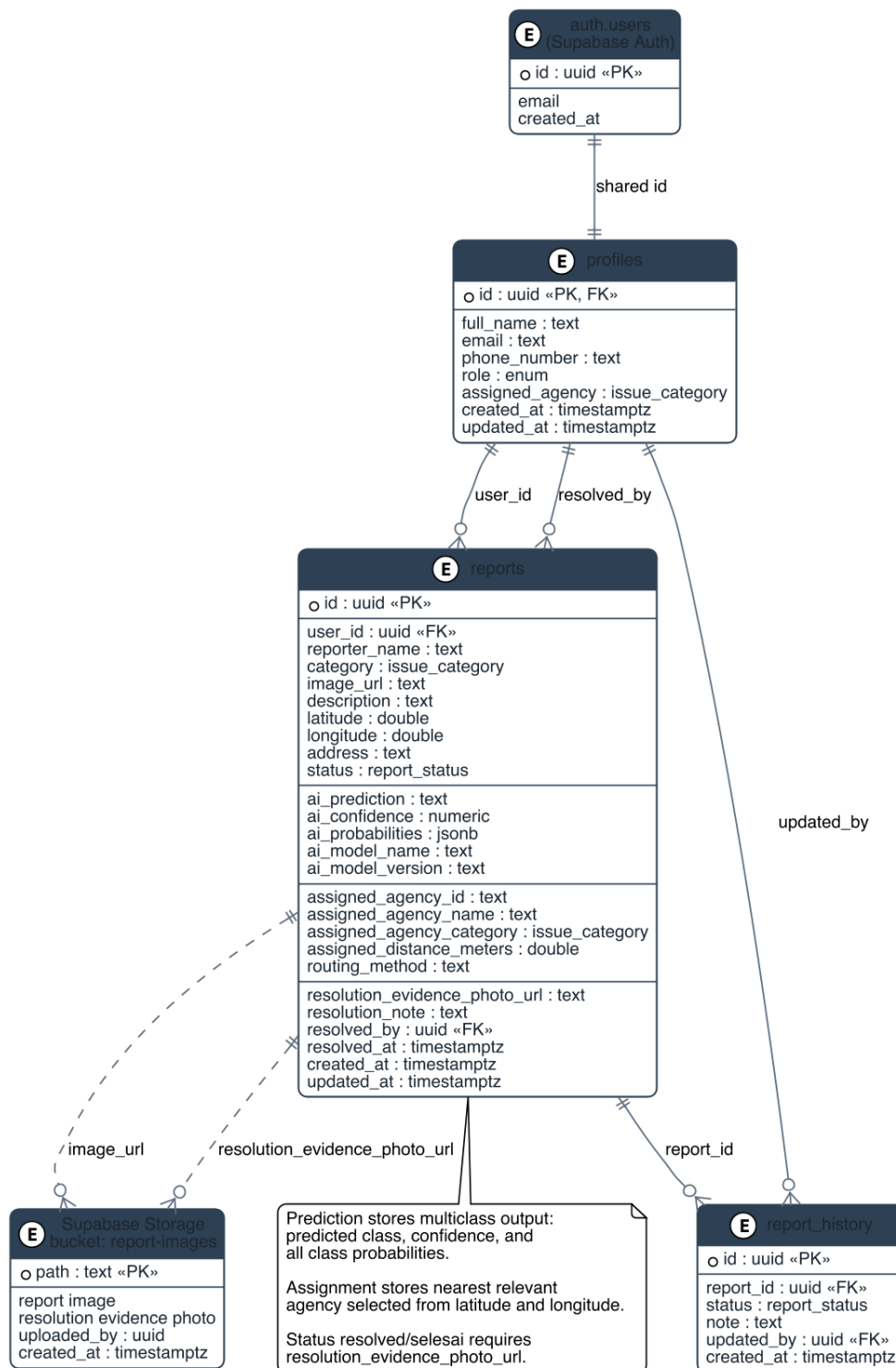
3.11.1 Ringkasan Tabel dan Kontrol Akses

Skema basis data terdiri atas tiga tabel inti dan tiga *enum* pendukung. Tabel `profiles` terhubung satu-ke-satu dengan `auth.users` milik Supabase Auth dan menyimpan nama, kontak, peran (`user_role`), serta instansi penugasan operator. Tabel `reports` adalah tabel utama yang menampung laporan warga lengkap dengan keluaran model (`ai_prediction`, `ai_confidence`, `ai_probabilities` dalam format `jsonb`, dan metadata model), hasil routing instansi terdekat (`assigned_*`), koordinat lintang dan bujur, status siklus hidup (`report_status`), serta bukti penyelesaian. Tabel `report_history` mencatat setiap perubahan status sebagai jejak audit yang ditulis hanya melalui *trigger* basis data. Kebijakan *Row Level Security* (RLS) memastikan pelapor hanya melihat laporan miliknya, petugas hanya mengakses kategori sesuai `assigned_agency`, dan administrator memiliki akses lintas instansi. Rincian kolom seluruh tabel, daftar lengkap nilai *enum*, serta kebijakan RLS disajikan pada Lampiran D.

3.11.2 Bukti Penyelesaian Laporan

Pada tahap penyelesaian laporan, petugas diwajibkan mengunggah foto bukti penyelesaian ketika status laporan diubah menjadi *resolved*/selesai. Foto bukti ini menunjukkan bahwa permasalahan telah ditangani di lapangan. Selain foto, petugas dapat menambahkan catatan penyelesaian yang disimpan pada kolom `resolution_note`.

Validasi bukti penyelesaian diterapkan pada alur status. Transisi dari *in-progress* ke *resolved* tidak dijalankan apabila `resolution_evidence_photo_url` belum tersedia. Metadata `resolved_by` dan `resolved_at` juga disimpan agar proses penyelesaian dapat diaudit.



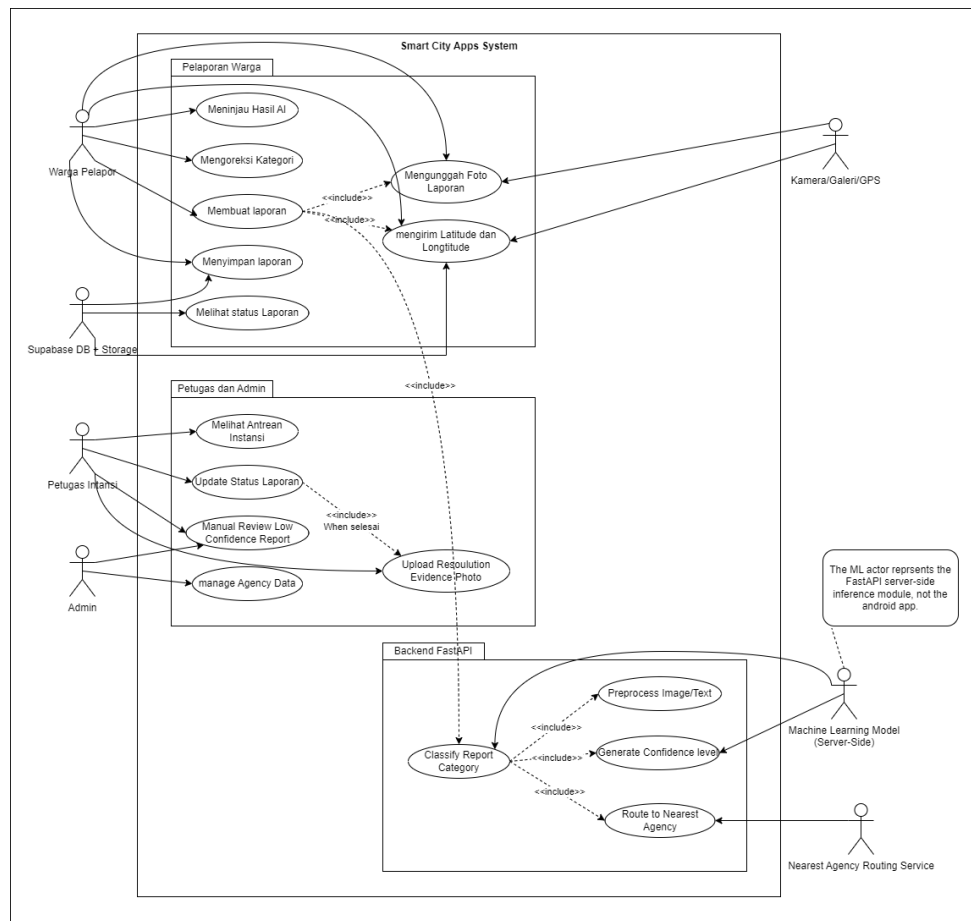
Gambar 3.9. Skema basis data Supabase untuk laporan, prediksi, penugasan instansi, dan bukti penyelesaian.

3.12 Perancangan Sistem Aplikasi Mobile (Flutter)

Aplikasi Flutter dikembangkan dengan arsitektur berbasis fitur (*feature-based architecture*). Dokumentasi rancangan sistem disusun mengikuti notasi UML standar yang diuraikan pada Bagian 2.17.

3.12.1 Use Case Diagram

Diagram *use case* memetakan interaksi antara aktor utama (Warga Pelapor dan Petugas Instansi) dengan fungsionalitas SmartCityApps, backend FastAPI, dan Supabase. Diagram *use case* ditunjukkan pada Gambar 3.10.



Gambar 3.10. Use case diagram SmartCityApps dengan prediksi server-side, confidence level, routing instansi terdekat, dan bukti penyelesaian.

3.12.2 Use Case Description

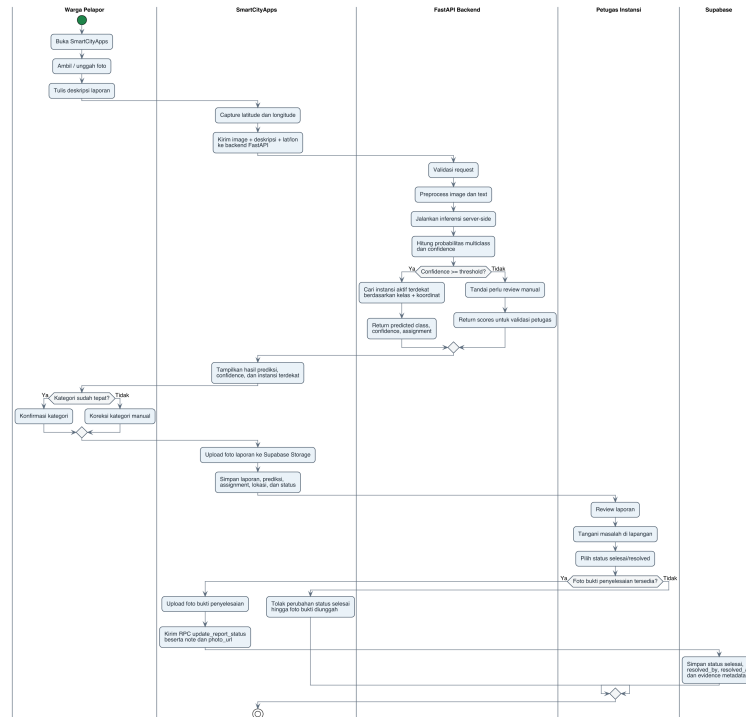
Setiap *use case* utama dirangkum pada Tabel 3.9. Deskripsi lengkap setiap *use case* (alur utama, alur alternatif, pra-kondisi, dan pasca-kondisi) disajikan pada Lampiran H.

Tabel 3.9. Ringkasan *use case* utama SmartCityApps.

Use Case	Aktor	Deskripsi Singkat
Mendaftar/Masuk Akun	Warga Pelapor	Registrasi atau login untuk mengakses fitur pelaporan.
Mengambil Foto Laporan	Warga Pelapor	Ambil foto kamera atau galeri sebagai bukti visual.
Menulis Narasi Laporan	Warga Pelapor	Tulis narasi Bahasa Indonesia pelengkap foto.
Mengirim Laporan	Warga Pelapor, Server	Kirim foto, narasi, dan koordinat ke FastAPI untuk klasifikasi.
Meninjau Hasil AI	Warga Pelapor	Lihat instansi prediksi, confidence, dan instansi terdekat.
Mengoreksi Kategori	Warga Pelapor	Ganti instansi prediksi secara manual bila keliru.
Menyimpan Laporan	Warga Pelapor	Simpan laporan dan hasil klasifikasi ke Supabase.
Melihat Riwayat/Peta	Warga Pelapor	Lihat daftar dan lokasi laporan tersimpan.

3.12.3 Activity Diagram

Activity diagram memvisualisasikan alur aktivitas pada skenario utama pelaporan oleh Warga Pelapor, mulai dari pengambilan foto, inferensi server-side, penyimpanan ke Supabase, sampai kewajiban bukti foto ketika petugas menandai laporan selesai. Gambar 3.11 menunjukkan alur lengkap tersebut, termasuk percabangan koreksi instansi secara manual pada layar tinjauan AI.



Gambar 3.11. Activity diagram siklus laporan SmartCityApps dari pelaporan, inferensi server-side, routing instansi terdekat, sampai bukti penyelesaian.

3.12.4 Class Diagram

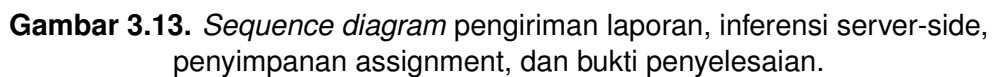
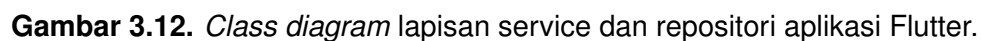
Class diagram memetakan kelas-kelas domain utama beserta atribut dan relasinya. Diagram ditunjukkan pada Gambar 3.12.

3.12.5 Sequence Diagram

Sequence diagram pada Gambar 3.13 menggambarkan urutan pesan antar komponen pada alur klasifikasi *end-to-end*, mulai dari layar buat laporan, backend FastAPI, routing instansi terdekat, penyimpanan Supabase, hingga unggah bukti penyelesaian oleh petugas.

3.12.6 Modul dan Layanan Flutter

Struktur modul aplikasi Flutter dirangkum pada Tabel 3.10.



Tabel 3.10. Modul dan layanan utama aplikasi Flutter.

Modul	Deskripsi
Modul Otentikasi	Pendaftaran, masuk, dan persistensi sesi lokal.
Modul Splash	Layar <i>splash</i> dan inisialisasi awal aplikasi.
Modul Onboarding	Panduan singkat untuk pengguna baru.
Modul Dashboard	Ringkasan laporan dan navigasi utama.
Modul Reports	Buat laporan, tinjauan AI, riwayat, peta, perutean instansi terdekat, dan bukti penyelesaian.
Modul Profile	Pengelolaan profil pengguna.
Modul Settings	Konfigurasi aplikasi.
Layanan Klasifikasi Awan	Klien HTTP <i>multipart</i> yang berkomunikasi dengan server FastAPI.
Layanan Lokasi	Akuisisi koordinat GPS dan <i>reverse geocoding</i> .
Layanan Media	Pemilih kamera dan galeri foto.
Layanan Repositori Laporan	Operasi CRUD pada tabel <code>reports</code> dan <code>report_history</code> Supabase, termasuk penyimpanan prediksi, assignment, bukti penyelesaian, dan <i>stream</i> realtime per peran pengguna.
Layanan Otentikasi Supabase	Pendaftaran, masuk, keluar, dan pembaruan profil pengguna dengan tiga peran.

3.13 Mekanisme Perutean Instansi

3.13.1 Keluaran Model Tetap Sembilan Instansi

Model klasifikasi tidak menghasilkan empat kluster operasional. Keluaran model tetap berupa prediksi multiclass pada sembilan instansi target, yaitu satu `predicted_dinas`, nilai *confidence*, dan skor probabilitas untuk seluruh kelas. Istilah kluster operasional pada rancangan awal hanya berfungsi sebagai panduan tampilan atau pengelompokan naratif, bukan sebagai label model dan bukan sebagai hasil akhir routing pada arsitektur Phase 2.

Dengan demikian, alur yang dipakai pada implementasi adalah: notebook melatih dan mengevaluasi model 9 kelas, artefak terbaik diekspor ke ONNX, backend FastAPI memuat artefak tersebut, lalu SmartCityApps menerima hasil prediksi dari server. Aplikasi Android tidak menjalankan model secara lokal.

3.13.2 Routing Instansi Terdekat Berbasis Lokasi

Setelah backend memperoleh `predicted_dinas`, sistem menentukan instansi tujuan berdasarkan dua masukan: kelas prediksi dan koordinat laporan. Koordinat latitude dan longitude laporan dibandingkan dengan daftar instansi aktif yang memiliki kategori sesuai. Instansi dengan jarak terdekat dipilih sebagai tujuan laporan.

Perhitungan jarak menggunakan formula Haversine karena koordinat berada pada permukaan bumi. Untuk dua titik koordinat (lat_1, lon_1) dan (lat_2, lon_2) , jarak permukaan bumi dihitung dengan pendekatan:

$$d = 2R \cdot \arctan 2(\sqrt{a}, \sqrt{1-a}), \quad (3.4)$$

dengan R adalah jari-jari bumi dan a dihitung dari selisih lintang serta bujur. Hasil routing yang dikembalikan backend meliputi `agency_id`, `agency_name`, `agency_category`, `distance_meters`, dan `routing_method`.

3.13.3 Kasus Confidence Rendah dan Fallback

Apabila nilai *confidence* berada di bawah ambang yang dikonfigurasi, laporan tetap dapat disimpan bersama seluruh skor probabilitasnya, tetapi ditandai sebagai membutuhkan validasi manual. Apabila tidak ditemukan instansi aktif yang cocok dengan kelas prediksi, backend menggunakan fallback ke unit tinjauan manual agar permintaan tidak gagal secara diam-diam.

3.14 Perancangan Layar Aplikasi

Bagian ini menjelaskan secara ringkas fungsi utama, alur interaksi pengguna, dan keterkaitan tiap layar dengan kebutuhan sistem yang dijabarkan pada Bagian 3.2. Mockup tampilan antarmuka untuk seluruh layar disajikan terpusat pada Lampiran A agar pemaparan metode tetap padat dan rancangan visual dapat ditinjau secara berdampingan.

3.14.1 Layar Splash dan Onboarding

Layar *splash* menampilkan logo aplikasi dan indikator pemuatan saat sumber daya awal (sesi otentikasi Supabase, konfigurasi GoRouter, dan registrasi *provider* Riverpod) dimuat. Setelah pemuatan selesai, pengguna baru diarahkan ke alur *onboarding* tiga halaman yang menjelaskan tiga proposisi nilai aplikasi, yaitu pelaporan multimodal, perutean otomatis, dan transparansi status laporan. Tampilan layar dapat dilihat pada Lampiran A.

3.14.2 Layar Buat Laporan

Layar buat laporan adalah titik masuk utama Warga Pelapor. Layar ini menyediakan tombol ambil foto, *preview* foto, medan teks narasi multibaris, indikator status GPS, dan tombol klasifikasi yang aktif ketika gambar, deskripsi, latitude, dan longitude tersedia. Data tersebut dikirim ke backend FastAPI; SmartCityApps tidak menjalankan model secara lokal. Tampilan layar dapat dilihat pada Lampiran A.

3.14.3 Layar Tinjauan Hasil AI

Layar tinjauan AI menampilkan instansi prediksi teratas, peringkat tiga prediksi teratas, *bar confidence* per kelas, serta instansi tujuan terdekat yang dikembalikan backend. Pengguna dapat mengoreksi kategori secara manual sebelum laporan disimpan. Layar ini menjadi titik intervensi kunci antara model AI dan keputusan manusia, sehingga sinyal ketidakpastian ditonjolkan. Tampilan layar dapat dilihat pada Lampiran A.

3.14.4 Layar Rekomendasi Instansi

Layar rekomendasi instansi menampilkan hasil routing dari backend berupa nama instansi tujuan, kategori instansi, dan estimasi jarak dari koordinat laporan. Informasi ini berasal dari prediksi 9 instansi dan koordinat latitude-longitude, bukan dari agregasi ke kluster baru. Tampilan layar dapat dilihat pada Lampiran A.

3.14.5 Layar Riwayat Laporan

Layar riwayat menampilkan daftar laporan milik pengguna dalam bentuk kartu yang memuat *thumbnail* foto, kategori instansi, tanggal pengiriman, dan status terkini. Pengguna dapat memilih satu kartu untuk melihat detail lengkap, melakukan audit pada tabel `report_history`, dan menerima pembaruan secara *realtime* melalui Supabase *Realtime*. Pada peran petugas, layar detail juga menyediakan aksi perubahan status; status selesai hanya dapat dikirim setelah foto bukti penyelesaian diunggah. Tampilan layar dapat dilihat pada Lampiran A.

3.14.6 Layar Peta Laporan

Layar peta menampilkan pin lokasi seluruh laporan pengguna pada peta interaktif. Setiap pin dapat diketuk untuk memunculkan *popup info* ringkas berisi kategori, status, dan tanggal. Filter kategori memungkinkan pengguna menyaring tampilan berdasarkan sembilan instansi target. Tampilan layar dapat dilihat pada Lampiran A.

3.14.7 Layar Profil dan Pengaturan

Layar profil menampilkan informasi akun pengguna (foto, nama, surel, dan peran) yang diambil dari tabel `profiles` di Supabase. Pengaturan yang tersedia meliputi pengelolaan sesi, preferensi bahasa, dan opsi keluar yang membersihkan sesi Supabase dan *state* Riverpod. Tampilan layar dapat dilihat pada Lampiran A.

3.15 Skenario Pengujian dan Metrik Evaluasi

Bagian ini merangkum skenario pengujian yang dijalankan untuk memvalidasi pipeline klasifikasi multimodal beserta metrik evaluasi yang dipakai untuk mengukur kinerja sistem.

3.15.1 Pembagian Data

Dataset Jakarta CRM dipartisi secara stratifikasi berdasarkan label kategori, dengan proporsi 70 persen *train*, 15 persen validasi, dan 15 persen uji. Partisi akhir berisi 43.241 sampel pelatihan, 9.266 sampel validasi, dan 9.266 sampel uji. Ketidakseimbangan kelas dikompensasi melalui pembobotan kelas (*class weights*) pada Persamaan 3.1, bukan melalui *resampling*. Pembagian disimpan pada berkas `train.csv`, `val.csv`, dan `test.csv` pada folder `artifacts/splits/` agar setiap eksperimen dapat menggunakan partisi yang identik.

3.15.2 Metrik Evaluasi

Metrik evaluasi yang dilaporkan terdiri atas dua kelompok, yaitu metrik utama dan metrik pendukung.

Metrik utama: *macro F1 score* dipilih sebagai metrik utama karena memberikan bobot yang sama pada setiap kelas, sesuai untuk dataset dengan ketidakseimbangan kelas yang tidak ekstrem namun tetap signifikan. *Balanced accuracy* dilaporkan sebagai metrik pendamping yang juga memberikan bobot yang sama pada setiap kelas. *Confusion matrix* digunakan untuk menampilkan distribusi prediksi benar dan salah pada setiap pasangan kelas.

Metrik pendukung: *precision* dan *recall* per kelas dilaporkan untuk mengidentifikasi kelas mana yang mengalami kesulitan tertentu. Untuk konfigurasi yang menggunakan PGS, *mean uncertainty* dihitung sebagai indikator kualitas kalibrasi probabilitas. Latensi inferensi end-to-end pada server FastAPI dihitung sebagai jumlah dari latensi pra-proses, latensi *image encoder*, latensi *text encoder*, latensi *classifier head*, dan latensi pasca-proses.

3.15.3 Skenario Ablasi

Ablasi yang dilaporkan pada Bab 4 terdiri atas tiga skenario, yaitu (a) ablasi modalitas dengan tiga konfigurasi (citra saja, teks saja, dan fusi awal), (b) ablasi kalibrasi dengan dua konfigurasi (CatBoost tanpa PGS dan CatBoost dengan PGS), serta (c) ablasi pasangan *encoder* dengan enam belas kombinasi pada matrix grid lengkap (kombinasi empat *image encoder*, termasuk DINOv3 *large*, dengan empat *text encoder*). Hasil setiap skenario disajikan dengan metrik utama dan metrik pendukung yang sama.

3.15.4 Validasi Paritas ONNX

Setelah pelatihan selesai, model *native* CatBoost diekspor ke format ONNX. Validasi paritas dilakukan dengan menjalankan kedua model pada batch sampel uji yang sama, lalu menghitung selisih probabilitas absolut maksimum ($|\Delta p|_{\max}$) di seluruh komponen vektor keluaran. Kriteria yang dipakai adalah $|\Delta p|_{\max} < 10^{-5}$ pada sampel set uji, yang memastikan prediksi top-1 identik antara jalur PyTorch dan jalur ONNX.

3.15.5 Pengujian Aplikasi Flutter

Aplikasi Flutter diuji secara end-to-end pada perangkat Android fisik dengan tiga skenario, yaitu (a) alur pelaporan oleh Warga Pelapor mulai dari pengambilan foto, penulisan narasi, klasifikasi AI, tinjauan, hingga penyimpanan ke Supabase, (b) alur verifikasi oleh Operator Instansi pada antrean laporan yang sudah dikat-

egorisasi, dan (c) alur moderasi lintas instansi oleh Moderator. Setiap skenario dievaluasi dari sisi waktu respons, konsumsi data jaringan, dan ketepatan klasifikasi yang ditampilkan pada layar tinjauan AI.

3.15.6 Catatan tentang Pemilihan Algoritma

Penelitian ini memilih CatBoost sebagai satu-satunya algoritma *classifier head* pada seluruh skenario eksperimen, dengan justifikasi teknis dan operasional yang telah dipaparkan pada Bagian 3.6.1. Tidak ada perbandingan dengan algoritma klasifikasi lain pada Bab 4; ablasi yang dilaporkan adalah ablasi modalitas, ablasi kalibrasi, dan ablasi pasangan *encoder*.

Bab ini telah memaparkan metodologi penelitian secara terstruktur, mulai dari kerangka berpikir, analisis kebutuhan dengan tiga peran pengguna, akuisisi dan persiapan dataset Jakarta CRM melalui *web scraping*, lingkungan komputasi RunPod yang diakses melalui SSH, pra-proses citra dan teks, ekstraksi *embedding* dari empat *encoder* citra dan empat *encoder* teks, fusi awal berbasis konkatenasi, justifikasi pemilihan CatBoost sebagai *classifier head* dan kalibrasinya dengan PGS, rancangan eksperimen ablasi, metrik evaluasi, ekspor ONNX, perancangan server FastAPI dan Docker untuk inferensi server-side, perancangan aplikasi Flutter SmartCityApps beserta UML lengkap (*use case*, *activity*, *class*, dan *sequence diagram*), mekanisme perutean instansi terdekat berdasarkan latitude-longitude, bukti penyelesaian laporan, rancangan layar, hingga skenario pengujian dan metrik evaluasi yang akan dipakai pada Bab 4. Bab Hasil dan Pembahasan berikutnya akan melaporkan capaian metrik kuantitatif, analisis kesalahan, dan validasi end-to-end pada aplikasi.

BAB 4

HASIL DAN PEMBAHASAN

Bab ini melaporkan hasil eksperimen dan implementasi yang dirancang pada Bab 3. Pemaparan diawali dengan lingkungan pengujian, dilanjutkan dengan persiapan dan eksplorasi dataset, hasil eksperimen utama menggunakan kombinasi DI-NOv3 *large*, *multilingual-E5 large*, dan CatBoost yang dikalibrasi PGS, hasil ablasi modalitas, hasil ablasi kalibrasi, hasil ablasi pasangan *encoder*, validasi paritas ON-NX, hasil implementasi aplikasi Flutter beserta integrasi Supabase, kemudian diakhiri dengan pembahasan dan analisis kesalahan.

4.1 Lingkungan Pengujian

Eksperimen pelatihan dan ekstraksi *embedding* dijalankan pada lingkungan komputasi awan RunPod yang diakses melalui SSH, dengan akselerator GPU yang dipilih sesuai ketersediaan. Lingkungan perangkat lunak yang dipakai dirangkum pada Tabel 4.1.

Pengujian aplikasi Flutter dilakukan pada perangkat Android fisik dengan koneksi USB *tethering*, dengan port server FastAPI yang diteruskan melalui `adb reverse` sehingga *endpoint* `/predict` dapat diakses oleh aplikasi seolah-olah berjalan di perangkat lokal.

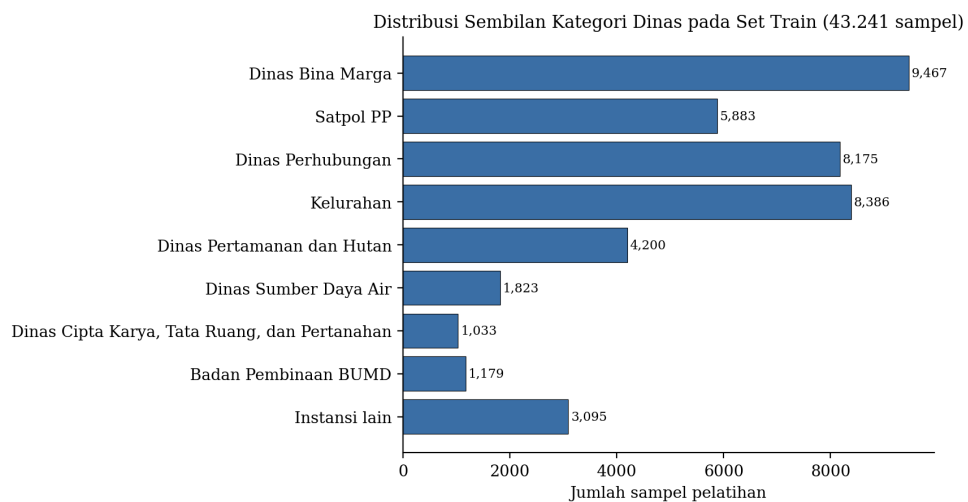
4.2 Persiapan dan Eksplorasi Data

Setelah pemetaan 487 nama SKPD mentah ke sembilan kategori target dan pembagian stratifikasi, partisi akhir dataset berisi 43.241 sampel pelatihan, 9.266 sampel

Tabel 4.1. Lingkungan perangkat lunak utama.

Komponen	Versi atau Detail
Python	3.11
PyTorch	2.x dengan CUDA toolkit
ONNX dan ONNX Run-time	1.x untuk ekspor dan inferensi
CatBoost	1.2.x dengan opsi <i>posterior sampling</i>
FastAPI	0.x untuk server inferensi
Flutter	3.x untuk klien Android
Supabase Dart Client	versi stabil terbaru
RunPod	GPU <i>cloud</i> NVIDIA GeForce RTX 4090 (24 GB VRAM), sesuai Tabel 3.5

validasi, dan 9.266 sampel uji. Ketidakseimbangan kelas tidak dihilangkan melalui *resampling*, melainkan dikompensasi melalui pembobotan kelas (*class weights*) pada algoritma CatBoost. Distribusi sampel pelatihan per instansi dirangkum pada Gambar 4.1 dan Tabel 4.2.



Gambar 4.1. Distribusi sembilan instansi (dinas) pada set train.

Distribusi yang ditampilkan menunjukkan bahwa ketidakseimbangan kelas masih ada walaupun sudah dilakukan penyeimbangan. Empat kelas teratas (Dinas Bina Marga, Kelurahan, Dinas Perhubungan, dan Satpol PP) mencakup lebih dari 73

Tabel 4.2. Distribusi sampel per instansi (dinas) pada set train.

ID	Kategori	Jumlah Sampel	Proporsi (%)
0	Dinas Bina Marga	9.467	21,89
1	Satuan Polisi Pamong Praja	5.883	13,60
2	Dinas Perhubungan	8.175	18,90
3	Kelurahan	8.386	19,39
4	Dinas Pertamanan dan Hutan	4.200	9,71
5	Dinas Sumber Daya Air	1.823	4,22
6	Dinas Cipta Karya, Tata Ruang, dan Pertanahan	1.033	2,39
7	Badan Pembinaan BUMD	1.179	2,73
8	Instansi lain	3.095	7,16
Total		43.241	100,00

persen total sampel, sedangkan tiga kelas terkecil (Dinas Cipta Karya, Tata Ruang, dan Pertanahan, Badan Pembinaan BUMD, serta Dinas Sumber Daya Air) hanya mencakup sekitar 9 persen. Ketidakseimbangan ini menjadi pertimbangan saat pemilihan metrik utama, yaitu *macro F1 score* yang memberikan bobot sama pada setiap kelas, sebagaimana diuraikan pada Bagian 3.15.

4.3 Hasil Eksperimen Utama

Konfigurasi terbaik yang dipilih untuk *deployment* adalah kombinasi *image encoder* DINOv3 *large*, *text encoder multilingual-E5 large*, fusi awal berbasis konkatenasi, *classifier head* CatBoost, dan kalibrasi probabilitas dengan PGS. Tabel 4.3 merangkum metrik kinerja konfigurasi tersebut pada set uji.

Konfigurasi terbaik mencapai *balanced accuracy* 0,8074 dan *macro F1* 0,7747 pada set uji 9.266 sampel, dengan akurasi keseluruhan 0,8073 dan *macro ROC-AUC (one-vs-rest)* sebesar 0,9667. Capaian ini di atas baseline yang dilaporkan oleh Hanif et al. (2024) sebesar akurasi 80,5 persen pada delapan kelas, mengingat penelitian ini bekerja pada sembilan kelas yang lebih luas dan menggantikan DINOv2 dengan DINOv3 *large*. Rerata waktu pelatihan satu konfigurasi CatBoost

Tabel 4.3. Metrik kinerja konfigurasi terbaik pada set uji (DINOv3 large + mE5 large + CatBoost+PGS).

Konfigurasi	Balanced Accuracy	Macro F1
CatBoost (tanpa PGS)	0,7996	0,7684
CatBoost dengan PGS	0,8074	0,7747
Δ (PGS minus tanpa PGS)	+0,0078	+0,0063

adalah 308 detik tanpa PGS dan 602 detik dengan PGS, yang membuktikan bahwa biaya komputasi pelatihan masih dapat ditoleransi pada satu sesi eksperimen.

4.3.1 Metrik Per-Kelas

Untuk mengamati kinerja model pada masing-masing instansi, Tabel 4.4 menyajikan *precision*, *recall*, dan *F1-score* per kelas pada set uji untuk konfigurasi terbaik (DINOv3 large + mE5 large + CatBoost+PGS).

Tabel 4.4. Metrik per-kelas pada set uji (DINOv3 large + mE5 large + CatBoost+PGS).

Kategori (Dinas)	Precision	Recall	F1	Support
Dinas Bina Marga	0,8644	0,8797	0,8720	2.029
Satuan Polisi Pamong Praja	0,8432	0,8699	0,8564	1.261
Dinas Perhubungan	0,8443	0,8699	0,8569	1.752
Kelurahan	0,8103	0,6939	0,7476	1.797
Dinas Pertamanan dan Hutan	0,8924	0,9122	0,9022	900
Dinas Sumber Daya Air	0,5577	0,7417	0,6367	391
Dinas Cipta Karya, Tata Ruang, dan Pertanahan	0,8398	0,8778	0,8584	221
Badan Pembinaan BUMD	0,6711	0,7937	0,7273	252
Instansi lain	0,5486	0,4857	0,5152	663
Macro average	0,7635	0,7916	0,7747	9.266
Weighted average	0,8086	0,8073	0,8061	9.266

Tabel per-kelas mengungkap bahwa kelas dengan dukungan besar dan kata kunci spesifik (Dinas Pertamanan dan Hutan, $F1 = 0,9022$; Dinas Bina Marga, $F1 =$

0,8720) terklasifikasi paling baik. Sebaliknya, kelas *catch-all* “Instansi lain” ($F1 = 0,5152$) menjadi yang terlemah karena memuat laporan heterogen tanpa pola visual atau tekstual yang konsisten, diikuti Dinas Sumber Daya Air ($F1 = 0,6367$) yang memiliki *precision* rendah (0,5577) akibat tertukar dengan kelas lain yang juga berkaitan dengan saluran air dan banjir. Menariknya, dua kelas minoritas lain yaitu Dinas Cipta Karya ($F1 = 0,8584$, hanya 221 sampel) dan Badan Pembinaan BUMD ($F1 = 0,7273$, 252 sampel) tetap terklasifikasi cukup baik, yang menunjukkan bahwa kelangkaan sampel tidak selalu berarti kinerja buruk apabila pola laporannya khas.

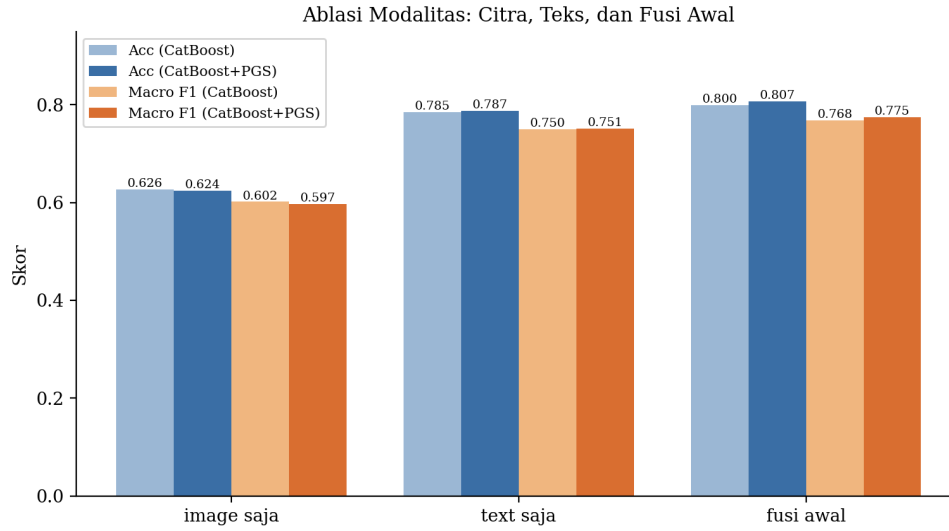
4.4 Hasil Ablasi Modalitas

Untuk memvalidasi kontribusi setiap modalitas, eksperimen ablasi dijalankan dengan tiga konfigurasi, yaitu citra saja, teks saja, dan fusi awal. Hasilnya dirangkum pada Tabel 4.5 dan Gambar 4.2.

Tabel 4.5. Hasil ablasi modalitas pada set uji.

Konfigurasi	Acc CatBoost	F1 CatBoost	Acc PGS	F1 PGS
Citra saja (DINOv3 large)	0,6265	0,6016	0,6238	0,5973
Teks saja (mE5 large)	0,7850	0,7498	0,7874	0,7515
Fusi awal (citra + teks)	0,7996	0,7684	0,8074	0,7747

Hasil ini menyampaikan tiga temuan penting. Pertama, modalitas teks saja jauh lebih diskriminatif dibandingkan modalitas citra saja, yaitu *macro F1* 0,7498 untuk teks dibandingkan 0,6016 untuk citra. Hal ini wajar karena narasi laporan sering kali memuat petunjuk eksplisit tentang dinas yang dituju, sedangkan citra hanya menampilkan kondisi visual yang ambigu pada beberapa kategori. Kedua, fusi awal memberikan peningkatan yang konsisten di atas modalitas tunggal terbaik, yaitu peningkatan *macro F1* sebesar 0,0186 (1,86 *percentage point*) terhadap teks saja pada konfigurasi PGS, menunjukkan bahwa citra membawa informasi tambahan yang tidak selalu tertangkap oleh teks. Ketiga, pengaruh PGS pada modalitas citra saja



Gambar 4.2. Ablasi modalitas pada CatBoost dan CatBoost+PGS.

adalah negatif tipis (selisih $-0,0043$ pada F1), sedangkan pada modalitas teks saja dan fusi awal pengaruhnya positif. Hal ini mengindikasikan bahwa PGS bekerja optimal ketika model awal sudah memberikan probabilitas yang relatif terkalibrasi, kondisi yang lebih sulit dipenuhi oleh model yang hanya mengandalkan modalitas visual.

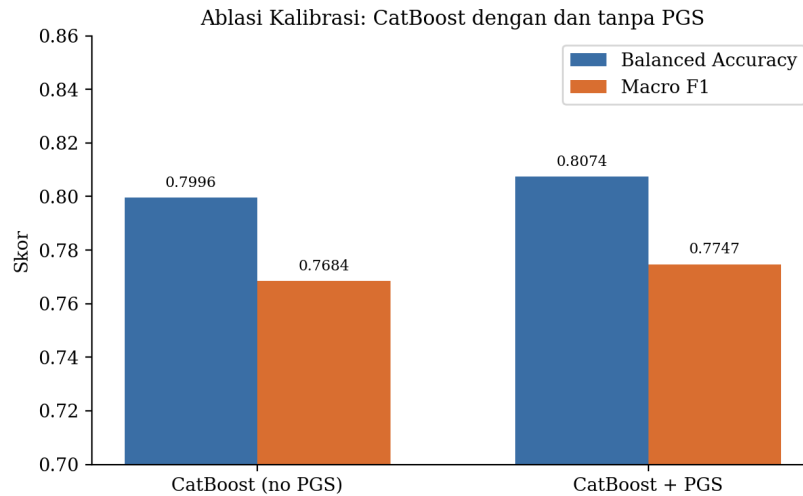
4.5 Hasil Ablasi Kalibrasi (PGS)

Pengaruh kalibrasi probabilitas dengan PGS dilaporkan pada Tabel 4.6 dan divisualisasikan pada Gambar 4.3. Konfigurasi yang dievaluasi adalah pasangan *encoder* terbaik (DINOv3 *large* + mE5 *large*) dengan dan tanpa PGS.

Tabel 4.6. Hasil ablasi kalibrasi pada set uji.

Model	Balanced Accuracy	Macro F1
CatBoost (tanpa PGS)	0,7996	0,7684
CatBoost dengan PGS	0,8074	0,7747
Δ (PGS minus tanpa PGS)	+0,0078	+0,0063

PGS memberikan peningkatan *balanced accuracy* sebesar *0,78 percentage point* dan *macro F1* sebesar *0,63 percentage point* terhadap baseline tanpa PGS. Selain



Gambar 4.3. Perbandingan balanced accuracy dan macro F1 dengan dan tanpa PGS.

memberikan peningkatan akurasi, PGS juga menghasilkan ukuran ketidakpastian (*mean uncertainty*) sebesar 0,00339 yang dapat ditampilkan pada antarmuka aplikasi sebagai sinyal kepercayaan kepada warga pelapor. Sinyal ini menjadi penting saat warga harus memutuskan apakah hasil klasifikasi otomatis perlu dikoreksi secara manual sebelum laporan dikirim.

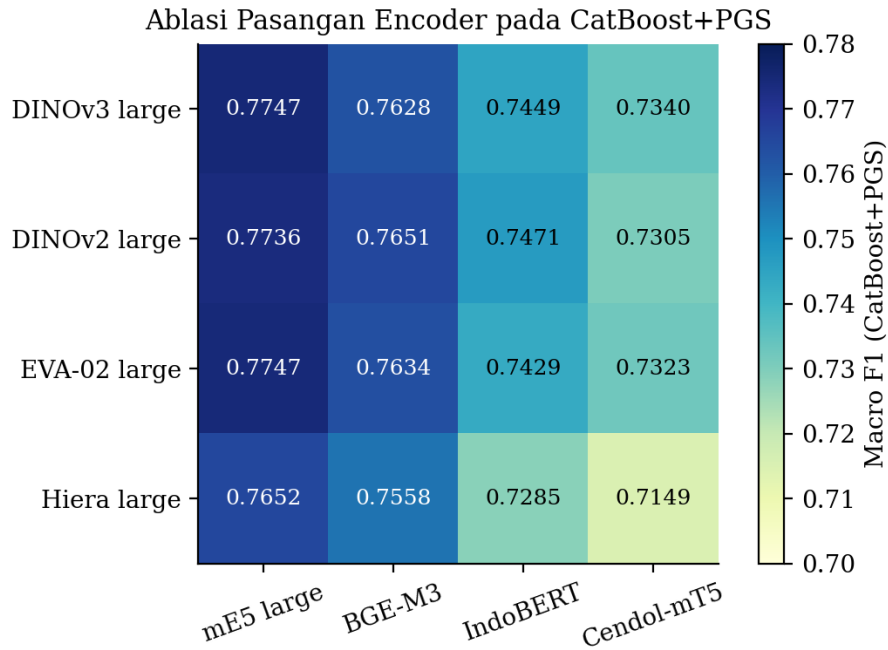
4.6 Hasil Ablasi Pasangan Encoder

Tabel 4.7 dan Gambar 4.4 merangkum hasil ablasi pasangan *image encoder* dan *text encoder* dengan kalibrasi PGS aktif. Total enam belas pasangan dievaluasi (empat *image encoder* dikali empat *text encoder*).

Tabel 4.7. Macro F1 (CatBoost+PGS) pada matrix grid pasangan encoder.

Image \ Text	mE5 large	BGE-M3	IndoBERT	Cendol-mT5
DINOv3 large	0,7747	0,7628	0,7449	0,7340
DINOv2 large	0,7736	0,7651	0,7471	0,7305
EVA-02 large	0,7747	0,7634	0,7429	0,7323
Hiera large	0,7652	0,7558	0,7285	0,7149

Pengamatan utama dari matrix grid ini adalah sebagai berikut. Pertama, *text*



Gambar 4.4. Heatmap macro F1 pada matrix grid pasangan encoder.

encoder multilingual-E5 large secara konsisten menjadi pilihan terbaik di seluruh empat *image encoder*, mengungguli BGE-M3, IndoBERT, dan Cendol-mT5 pada setiap baris. Hal ini sejalan dengan kekuatan multilingual-E5 dalam menyandikan kalimat Bahasa Indonesia yang informal dan singkat ke ruang vektor yang diskriminatif. Kedua, di antara empat *image encoder*, DINOv3 *large* dan EVA-02 *large* menunjukkan kinerja yang nyaris identik pada *macro F1* (0,7747), keduanya unggul terhadap DINOv2 *large* dan Hiera *large*. Penelitian ini memilih DINOv3 *large* untuk *deployment* karena merupakan keluarga *encoder* paling mutakhir pada saat eksperimen. Ketiga, Hiera *large* secara konsisten menjadi yang terlemah pada keempat pasangan, yang menunjukkan bahwa arsitektur hierarchical yang dioptimalkan untuk efisiensi inferensi belum mengejar kualitas representasi DINOv3 atau EVA-02 pada domain laporan publik perkotaan.

4.7 Validasi Paritas ONNX

Setelah *checkpoint* CatBoost terbaik dipilih, model *native* .cbm diekspor ke format ONNX bersama dengan *image encoder* DINOv3 *large* dan *text encoder*

multilingual-E5 large. Validasi paritas dilakukan dengan menjalankan kedua jalur (PyTorch dan ONNX) pada batch sampel uji yang sama, lalu menghitung selisih probabilitas absolut maksimum di seluruh komponen vektor keluaran. Hasilnya dirangkum pada Tabel 4.8.

Tabel 4.8. Validasi paritas numerik PyTorch terhadap ONNX pada set uji.

Komponen	$ \Delta p _{\max}$
<i>Image encoder</i> (DINOv3 large)	$< 10^{-5}$
<i>Text encoder</i> (multilingual-E5 large)	$< 10^{-5}$
<i>Classifier head</i> (CatBoost+PGS)	$< 10^{-5}$
End-to-end (gabungan ketiga komponen)	$< 10^{-5}$

Selisih probabilitas absolut maksimum di bawah ambang 10^{-5} pada seluruh komponen memastikan prediksi top-1 identik antara jalur PyTorch dan jalur ONNX, sehingga server FastAPI yang memuat ketiga *artifact* ONNX dapat dipakai sebagai jalur produksi tanpa kekhawatiran terjadi divergensi numerik dengan jalur pelatihan.

4.8 Hasil Implementasi Aplikasi Flutter

Aplikasi SmartCityApps berhasil diimplementasikan menggunakan Flutter dengan arsitektur berbasis fitur, manajemen *state* Riverpod, navigasi GoRouter, dan *backend* Supabase. Aplikasi mendukung tiga peran pengguna, yaitu Warga Pelapor, Operator Instansi, dan Moderator atau *SuperAdmin*, dengan akses yang dibatasi melalui Row Level Security pada tabel `profiles`, `reports`, dan `report_history`.

4.8.1 Skema Basis Data Supabase

Tabel utama yang menopang aplikasi adalah `profiles` (data pengguna dan peran), `reports` (laporan dengan instansi prediksi, confidence, probabilitas multiclass, assignment instansi terdekat, lokasi, status, tautan gambar, dan bukti penyelesaian), dan `report_history` (audit perubahan status laporan). Berkas gambar disimpan pada *bucket* `report-images` di Supabase Storage. Komunikasi re-

altime dengan klien Flutter difasilitasi oleh Supabase Postgres Realtime sehingga perubahan status pada laporan langsung tercermin di antarmuka pengguna tanpa perlu *polling*. Skema lengkap dirangkum pada Gambar 4.5.

4.8.2 Alur Pelaporan oleh Warga

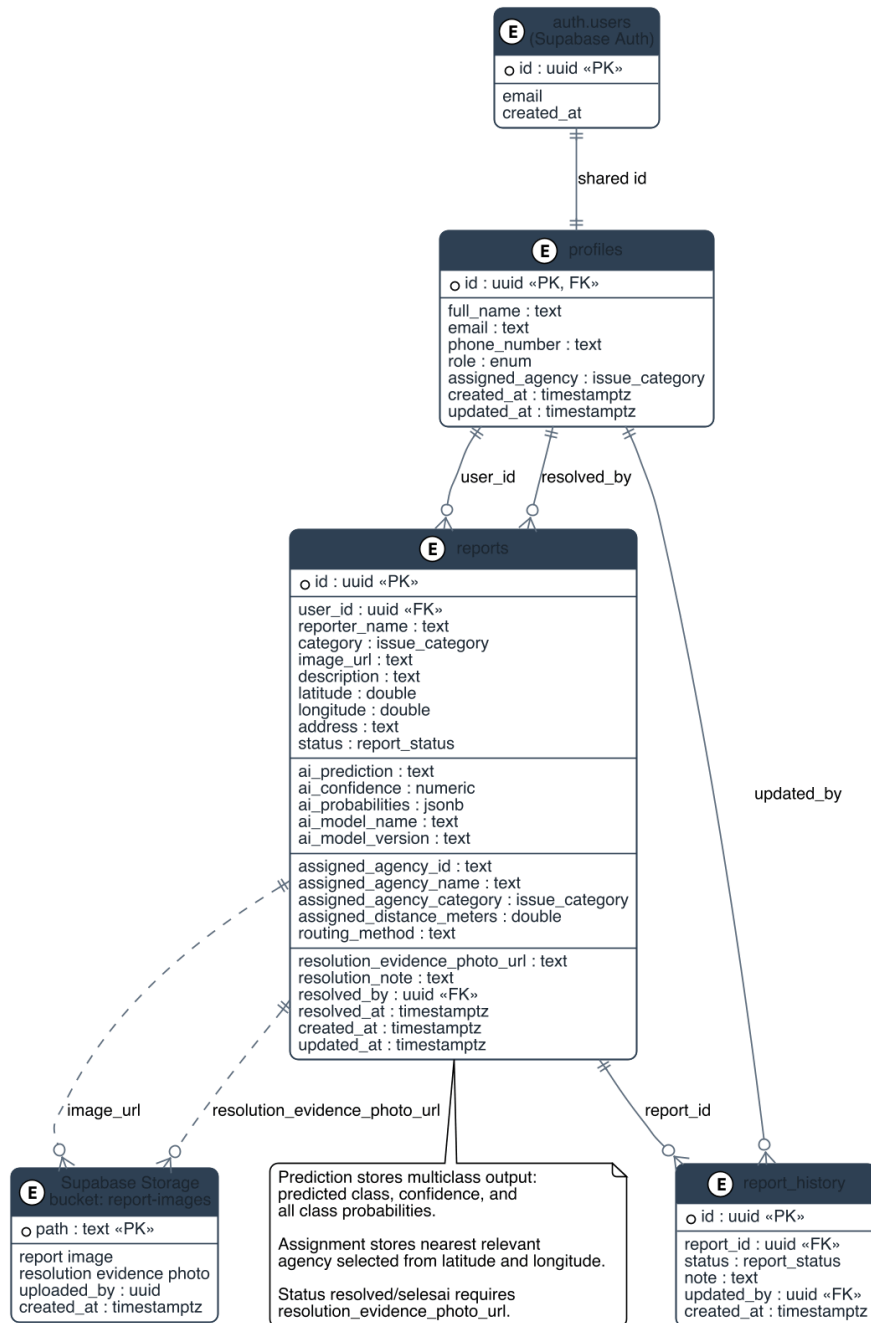
Skenario alur pelaporan oleh Warga Pelapor diuji secara end-to-end pada perangkat Android fisik. Tahapan yang berhasil dijalankan adalah pengambilan foto melalui kamera atau galeri, pengisian narasi laporan, akuisisi koordinat GPS dengan opsi pemilihan manual di peta bila layanan lokasi tidak tersedia, klasifikasi otomatis melalui *endpoint* `/predict` server FastAPI, tinjauan hasil pada layar tinjauan AI, opsi koreksi kategori secara manual, tampilan instansi tujuan terdekat dari backend, dan terakhir penyimpanan laporan ke tabel `reports` di Supabase beserta unggahan gambar ke Storage.

4.8.3 Alur Verifikasi oleh Operator Instansi

Operator Instansi mengakses dasbor admin yang menampilkan antrean laporan yang sudah dikategorisasi oleh AI, difilter berdasarkan instansi yang ditugaskan kepada operator. Operator dapat memverifikasi laporan, menolak dengan mengisi alasan penolakan, atau menandai laporan sebagai selesai. Perubahan status menjadi selesai mewajibkan unggah foto bukti penyelesaian. Setiap perubahan status menulis baris audit ke tabel `report_history` sehingga jejak verifikasi dapat ditelusuri.

4.8.4 Alur Moderasi

Moderator memiliki hak akses lintas instansi untuk meninjau seluruh laporan, mengubah penugasan operator, dan memantau statistik serta jejak audit. Pengujian alur moderasi memastikan bahwa Row Level Security berfungsi dengan benar, yaitu Moderator dapat mengakses tabel lintas instansi sedangkan Operator hanya dapat mengakses laporan pada instansi yang ditugaskan.



Gambar 4.5. ERD basis data Supabase untuk laporan, prediksi, assignment instansi terdekat, bukti penyelesaian, dan kebijakan per peran.

4.8.5 Demo Perutean Instansi Terdekat

Mesin perutean memilih instansi tujuan terdekat berdasarkan sembilan instansi prediksi dan koordinat laporan. Tabel 4.9 menampilkan tiga contoh laporan beserta instansi prediksi dan contoh assignment yang disimpan.

Tabel 4.9. Contoh perutean instansi terdekat dari hasil klasifikasi dan koordinat laporan.

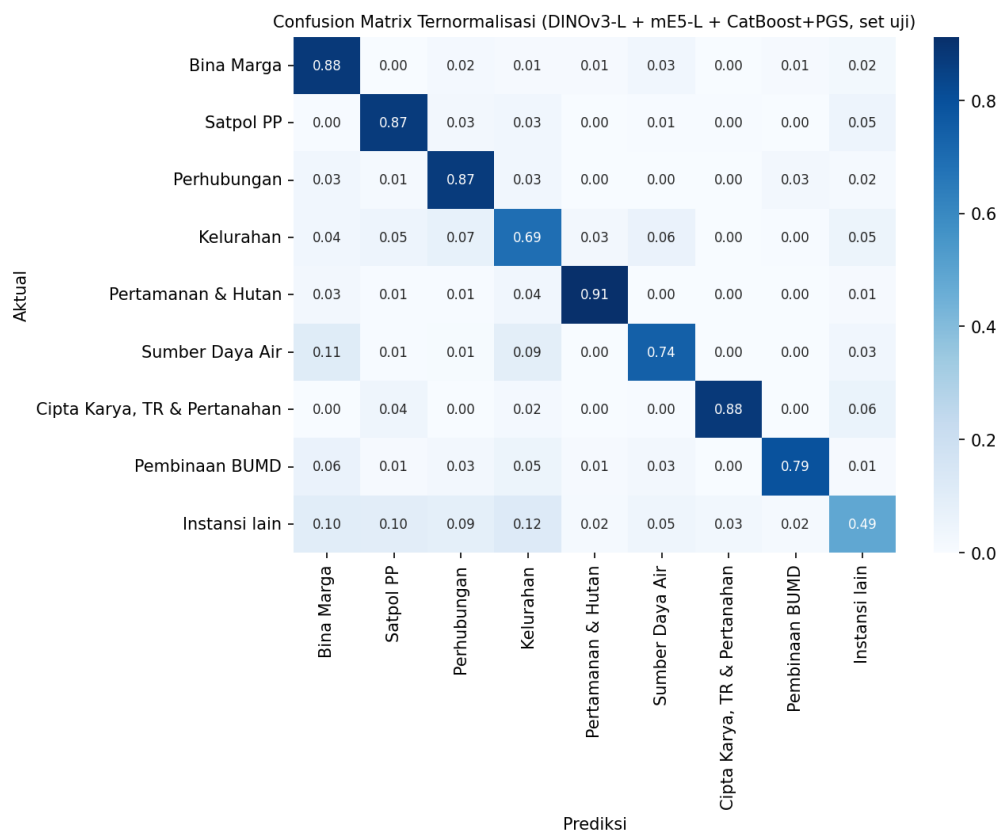
Ringkasan Laporan	Instansi Prediksi	Assignment Instansi Terdekat
Foto jalan berlubang dengan narasi “ada lubang besar di Jalan Sudirman”	Dinas Bina Marga	Unit Bina Marga terdekat dari koordinat laporan
Foto sampah menumpuk dengan narasi “sampah dibakar di pinggir jalan”	Satpol PP	Unit Satpol PP terdekat dari koordinat laporan
Foto pohon tumbang dengan narasi “pohon roboh karena hujan deras kemarin”	Dinas Pertamanan dan Hutan	Unit Pertamanan dan Hutan Kota terdekat dari koordinat laporan

4.9 Pembahasan dan Analisis Kesalahan

4.9.1 Pasangan Kelas yang Sering Tertukar

Confusion matrix ternormalisasi pada konfigurasi terbaik ditampilkan pada Gambar 4.6.

Analisis *confusion matrix* pada konfigurasi terbaik mengidentifikasi tiga pola kesalahan yang dominan. Pertama, kelas **Kelurahan** memiliki *recall* terendah di antara kelas mayoritas (0,6939), dengan laporan miliknya tersebar ke Dinas Perhubungan (134 sampel), Dinas Sumber Daya Air (110 sampel), Instansi lain (92 sampel), dan Satpol PP (86 sampel); hal ini wajar karena laporan tingkat kelurahan mencakup masalah lingkungan permukiman yang beririsan dengan banyak instansi. Kedua, kelas **Instansi lain** sebagai kategori *catch-all* paling sulit dipisahkan (hanya 322 dari 663 sampel benar), karena memuat laporan heterogen yang menyerupai



Gambar 4.6. *Confusion matrix* ternormalisasi (per baris) pada set uji untuk konfigurasi terbaik DINOv3 large + mE5 large + CatBoost+PGS.

kelas lain. Ketiga, **Dinas Sumber Daya Air** memiliki *precision* rendah (0,5577) akibat laporan banjir dan saluran air dari kelas lain (terutama Bina Marga dan Kelurahan) yang keliru diprediksi sebagai Sumber Daya Air. Berbeda dengan dugaan awal, kelas minoritas Dinas Cipta Karya justru memiliki *F1* tinggi (0,8584), sehingga kesalahan lebih banyak ditentukan oleh tumpang-tindih semantik antar kelas daripada sekadar kelangkaan sampel.

4.9.2 Sumber Ambiguitas

Tiga sumber ambiguitas dapat diidentifikasi. Pertama, label noise pada tahap pemetaan 487 nama SKPD ke sembilan kategori target, yaitu beberapa laporan secara historis dapat ditugaskan ke lebih dari satu instansi tergantung kebijakan operator. Kedua, narasi yang sangat singkat tanpa kata kunci eksplisit, contohnya hanya menulis “mohon ditangani” tanpa menyebut jenis masalah, sehingga kontribusi modalitas teks menjadi rendah. Ketiga, citra dengan kualitas rendah seperti foto buram atau pencahayaan minim, sehingga kontribusi modalitas citra juga rendah. Pada situasi ketiga ini, kalibrasi probabilitas dengan PGS justru bermanfaat karena menampilkan ketidakpastian tinggi yang dapat memicu pengguna untuk mengoreksi kategori secara manual.

4.9.3 Keterbatasan Dataset

Selain ketidakseimbangan kelas, dataset Jakarta CRM memiliki keterbatasan lain, yaitu cakupan geografis yang hanya mencakup wilayah DKI Jakarta sehingga generalisasi ke daerah lain belum diuji, serta domain laporan yang dibatasi pada kategori publik sehingga laporan sensitif seperti yang berkaitan dengan keamanan tidak masuk ke set pelatihan. Kedua keterbatasan ini terbuka sebagai arah penelitian lanjutan, misalnya dengan memperluas dataset ke daerah lain atau menambahkan cakupan kategori yang lebih luas.

4.9.4 Diskusi Keputusan Engineering

Tiga keputusan *engineering* utama pada penelitian ini menerima konfirmasi empiris dari hasil eksperimen. Pertama, keputusan untuk membekukan kedua *encoder* terbukti efisien karena memungkinkan ablasi pasangan *encoder* dijalankan dalam satu sesi eksperimen tanpa biaya pelatihan ulang *encoder*. Kedua, keputusan memilih CatBoost sebagai *classifier head* terbukti tepat karena seluruh delapan properti yang dijabarkan pada Bagian 3.6.1 terealisasi pada hasil eksperimen, mulai dari biaya pelatihan rendah, kalibrasi probabilitas yang dapat diaudit, hingga paritas numerik yang terjaga pada ekspor ONNX. Inferensi *classifier head* CatBoost+PGS dengan 30 ensemble virtual pada seluruh 9.266 sampel uji hanya memerlukan sekitar 0,05 ms per sampel, sehingga latensi *end-to-end* server didominasi oleh tahap ekstraksi *embedding* kedua *encoder*, bukan oleh *classifier head*. Ketiga, keputusan memilih jalur inferensi *on-server* terbukti praktis karena ukuran *artifact* ONNX *image encoder* sebesar 1,15 GB tidak dapat dibundel ke paket aplikasi Android, sebagaimana diuraikan pada Bagian 2.16.

BAB 5

SIMPULAN DAN SARAN

5.1 Simpulan

Penelitian ini telah membangun prototipe sistem pelaporan masalah lingkungan berbasis fusi multimodal citra dan teks yang siap diintegrasikan dengan ekosistem aplikasi pemerintah daerah. Berdasarkan rumusan masalah yang dipaparkan pada Bab 1, simpulan penelitian ini adalah sebagai berikut.

1. **Arsitektur klien-server berbasis Flutter, FastAPI, dan Supabase.** Sistem prototipe SmartCityApps berhasil diimplementasikan dengan klien Android berbasis Flutter, server inferensi FastAPI yang memuat tiga *artifact* ONNX, dan *backend* Supabase yang menyediakan otentikasi, basis data PostgreSQL, dan penyimpanan berkas. Tiga peran pengguna (Warga Pelapor, Operator Instansi, dan Moderator) berhasil dilayani dengan akses yang dibatasi melalui Row Level Security pada tabel `profiles`, `reports`, dan `report_history`. Konsumsi sumber daya pada perangkat pengguna tetap minimum karena seluruh komputasi inferensi dipusatkan di server.
2. **Kombinasi DINOv3 large, multilingual-E5 large, dan CatBoost dengan PGS.** Pada set uji yang berisi 9.266 sampel Jakarta CRM, konfigurasi terbaik mencapai *balanced accuracy* sebesar 0,8074 dan *macro F1 score* sebesar 0,7747, mengungguli baseline modalitas teks saja (*macro F1* 0,7515) sebesar 2,32 *percentage point* dan baseline modalitas citra saja (*macro F1* 0,5973) sebesar 17,74 *percentage point*. Hasil ini menunjukkan bahwa skema fusi aw-

al dengan *classifier head* CatBoost yang dikalibrasi PGS efektif untuk klasifikasi laporan multimodal Bahasa Indonesia.

3. **Paritas numerik PyTorch terhadap ONNX.** Validasi paritas pada seluruh komponen *pipeline* (image encoder, text encoder, dan classifier head) memberikan selisih probabilitas absolut maksimum di bawah ambang 10^{-5} , yang memastikan prediksi top-1 identik antara jalur pelatihan PyTorch dan jalur produksi ONNX. Server FastAPI dapat dipakai sebagai jalur inferensi produksi tanpa risiko divergensi numerik dengan jalur pelatihan.
4. **Routing instansi terdekat.** Backend berhasil menggabungkan keluaran model 9 instansi dengan koordinat latitude-longitude laporan untuk memilih instansi tujuan terdekat yang relevan. Hasil routing yang disimpan mencakup identitas instansi, nama instansi, kategori instansi, estimasi jarak, dan metode routing, sehingga laporan tidak hanya diklasifikasikan, tetapi juga diarahkan ke unit penanganan yang paling dekat dengan lokasi kejadian.
5. **Pemilihan inferensi *on-server* terhadap *on-device*.** Skema inferensi *on-server* berbasis FastAPI dipilih karena total ukuran *artifact* ONNX, dengan komponen *image encoder* DINOv3 *large* saja sebesar 1,15 GB, jauh melampaui batas wajar paket aplikasi Android. Konsekuensi dari pemilihan ini adalah ketergantungan pada ketersediaan jaringan dan pada layanan FastAPI sebagai titik inferensi tunggal, sekaligus keuntungan berupa pembaruan model yang terpusat tanpa harus merilis ulang aplikasi pada *Play Store*, konsumsi daya yang rendah pada perangkat pengguna, dan kemampuan menjalankan model berskala besar yang tidak praktis untuk *deployment* sepenuhnya *on-device*.

5.2 Saran

Berdasarkan temuan dan keterbatasan penelitian ini, beberapa arah penelitian lanjutan diusulkan sebagai berikut.

1. **Distilasi dan kuantisasi untuk varian *on-device*.** Penelitian lanjutan dapat menelaah distilasi pengetahuan dari DINOv3 *large* ke arsitektur ringan seperti DINOv3 ViT-Small atau DINOv3 ViT-Base, dipadu dengan kuantisasi INT8 menggunakan ONNX Runtime Mobile. Varian ringan ini dapat dipakai sebagai jalur *fallback* ketika koneksi jaringan tidak tersedia, sehingga warga pelapor tetap dapat memperoleh prediksi awal walaupun dengan akurasi yang sedikit lebih rendah.
2. **Penambahan modalitas tambahan.** Dataset pelaporan publik dapat diperkaya dengan modalitas tambahan, contohnya audio (suara latar dari lokasi kejadian) yang berguna untuk laporan kebisingan, geolokasi yang dipakai sebagai fitur eksplisit pada vektor fusi sehingga model dapat memanfaatkan pola spasial, dan metadata waktu pelaporan untuk menangkap pola harian atau musiman.
3. **Active learning dengan koreksi manual pengguna.** Mekanisme koreksi kategori secara manual oleh pengguna pada layar tinjauan AI dapat diubah menjadi sumber sinyal label baru. Setiap koreksi yang konsisten lintas pengguna dapat dipakai untuk melakukan pelatihan ulang model secara periodik, sehingga akurasi model meningkat dari waktu ke waktu seiring penggunaan aplikasi.
4. **Integrasi dengan API resmi pemerintah.** Penelitian lanjutan dapat melakukan integrasi langsung dengan API resmi platform LAPOR! SP4N yang dikelola oleh KemenPANRB atau platform JAKI yang dikelola oleh Pemerintah Provinsi DKI Jakarta. Dengan integrasi tersebut, hasil triase otomatis dapat langsung diteruskan ke kanal pemerintah, sehingga prototipe ini dapat berfungsi sebagai *front-end* pelaporan multimodal yang melengkapi sistem yang sudah ada.
5. **Perluasan dataset lintas daerah.** Dataset Jakarta CRM dapat diperluas dengan menambahkan dataset dari platform LaporGub Jawa Tengah (Zahro et al.,

2025) atau dari pemerintah daerah lain di Indonesia. Perluasan ini memberikan kesempatan untuk menguji generalisasi model terhadap pola pelaporan di daerah dengan struktur instansi yang berbeda.

6. **Penanganan kelas minoritas.** Tiga kelas minoritas pada Jakarta CRM (Dinas Cipta Karya, Tata Ruang, dan Pertanahan, Badan Pembinaan BUMD, serta Dinas Sumber Daya Air) menyumbang akurasi yang lebih rendah pada *confusion matrix*. Teknik seperti *focal loss* (Zahro et al., 2025), *class-balanced sampling* yang lebih agresif, atau augmentasi data berbasis sintesis dapat diuji untuk meningkatkan kinerja pada kategori tersebut.
7. **Replikasi server dan kebijakan ketersediaan.** Karena server FastAPI menjadi titik inferensi tunggal, kebijakan ketersediaan layanan perlu dirancang dengan cermat. Penelitian lanjutan dapat melakukan replikasi server pada beberapa region, menambahkan *circuit breaker* dan *retry policy* pada klien Flutter, serta merancang prosedur fallback ketika server tidak dapat dijangkau dalam waktu yang lama.

DAFTAR PUSTAKA

- Alam, F., Ofli, F., and Imran, M. (2018). CrisisMMD: Multimodal Twitter Datasets from Natural Disasters. In *Proceedings of the 12th International AAAI Conference on Web and Social Media (ICWSM)*, pages 465–473, Stanford, California, USA. AAAI Press. Dataset benchmark multimodal citra dan teks dari Twitter untuk klasifikasi laporan bencana alam; dirilis oleh Qatar Computing Research Institute.
- Amor, D. and Gutiérrez, Á. (2025). Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems. *Electronics*, 14(24):4877. Survei sistematis (PRISMA) optimisasi dan deployment neural network pada sistem embedded; pruning, kuantisasi, dan arsitektur ringan untuk inferensi on-device.
- Apple Inc. (2024). Core ML: Integrate Machine Learning Models into Your App. <https://developer.apple.com/documentation/coreml>. Accessed: 2026-01-22.
- Audebert, N., Herold, C., Slimani, K., and Vidal, C. (2019). Multimodal deep networks for text and image-based document classification. *arXiv preprint arXiv:1907.06370*, (arXiv:1907.06370). Klasifikasi multimodal teks dan citra pada dokumen, mendekati metodologi fusi awal pada laporan multimodal.
- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate.
- Baltrušaitis, T., Ahuja, C., and Morency, L.-P. (2019). Multimodal Machine Learn-

- ing: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2):423–443.
- Booch, G., Rumbaugh, J., and Jacobson, I. (2005). *The Unified Modeling Language User Guide*. Addison-Wesley, Boston, 2nd edition.
- Brown, S. (2018). *The C4 Model for Visualising Software Architecture: Context, Containers, Components, and Code*. Leanpub.
- Cahyawijaya, S., Lovenia, H., Koto, F., Putri, R. A., Cenggoro, W., Lee, J., Akbar, S. M., Mahendra, R., Aji, A. F., Solorio, T., and Purwarianti, A. (2024). Cendol: Open Instruction-tuned Generative Large Language Models for Indonesian Languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics.
- Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., and Joulin, A. (2021). Emerging Properties in Self-Supervised Vision Transformers. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9650–9660. IEEE.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. (2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation.
- Cubuk, E. D., Zoph, B., Shlens, J., and Le, Q. V. (2020). RandAugment: Practical Automated Data Augmentation with a Reduced Search Space. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 3008–3017. IEEE.
- Dalal, N. and Triggs, B. (2005). Histograms of Oriented Gradients for Human Detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 886–893. IEEE.

- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2019)*, pages 4171–4186. Association for Computational Linguistics.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.
- Fadjeri, A., Asroriyah, A., and Rahmawati, A. (2022). Analisis teks bahasa indonesia dan inggris dari sebuah citra menggunakan pengolahan citra digital. *Jurnal Teknologi Informasi dan Komunikasi (TIKomSiN)*, 10:42.
- Fang, Y., Sun, Q., Wang, X., Huang, T., Wang, X., and Cao, Y. (2023). EVA-02: A Visual Representation Powerhouse Unleashed by CLIP and MAE.
- Firgia, L., Muhamad Muslih, and Aditya Pratama (2022). Implementasi Sistem Informasi Pengaduan Masyarakat Di Daerah Perbatasan Studi Kasus Desa Cipta Karya. *Jurnal Rekayasa Teknologi Nusa Putra*, 8(2):101–110.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.
- Google LLC (2024a). Android Developer Documentation. <https://developer.android.com/docs>. Accessed: 2025-12-08.
- Google LLC (2024b). Flutter documentation. <https://flutter.dev/docs>. Accessed: 2025-10-15.
- Google LLC (2024c). TensorFlow Lite: Deploy Machine Learning Models on Mobile and Edge Devices. <https://www.tensorflow.org/lite>. Dokumentasi resmi TensorFlow Lite untuk deployment model pada perangkat mobile dan edge.

Hanif, I. A., Tjitrahardja, E., Naufal, R. B., and Rahadiani, L. (2024). Penugasan Otomatis Laporan Masyarakat pada Platform Cepat Respon Masyarakat Menggunakan Early Fusion Multimodal Transformer. In *Pagelaran Mahasiswa Nasional Bidang Teknologi Informasi dan Komunikasi (GEMASTIK) 2024*, Depok. Fakultas Ilmu Komputer Universitas Indonesia. Mengusulkan early fusion DI-NOv2 dan multilingual-E5 dengan kalibrasi Posterior Gaussian Sampling pada dataset CRM Jakarta delapan kelas; akurasi 75,7 persen meningkat menjadi 80,5 persen dengan PGS.

Harian Haluan (2024). Pemprov DKI Jakarta Tindaklanjuti 93,2 Persen Aduan Warga Secara Efektif Pakai Platform CRM. <https://www.harianhaluan.com/news/107791520/pemprov-dki-jakarta-tindaklanjuti-932-persen-aduan-warga-secara-efektif-pakai-platform-crm>. Persentase tindak lanjut efektif 93,2 persen pada platform CRM DKI Jakarta.

He, K., Chen, X., Xie, S., Li, Y., Dollár, P., and Girshick, R. (2022). Masked Autoencoders Are Scalable Vision Learners. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009. IEEE.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE.

Jakarta Smart City (2024a). In Numbers: CRM Reports From January to June 2024. <https://smartcity.jakarta.go.id/en/blog/bedah-data-laporan-januari-juni-dari-crm/>. Lebih dari 95.000 laporan diterima pada semester pertama 2024 melalui 13 kanal pengaduan terintegrasi.

Jakarta Smart City (2024b). Kaleidoskop CRM 2024: Laporan Warga dalam Angka. <https://smartcity.jakarta.go.id/en/blog/kaleidoskop-crm-2024-laporan-warga-dalam-angka/>. Total

185.852 laporan ditindaklanjuti pada platform Cepat Respon Masyarakat Provinsi DKI Jakarta selama 2024.

Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer Series in Statistics. Springer, New York, 2nd edition.

Kementerian Pendayagunaan Aparatur Negara dan Reformasi Birokrasi (2023). Kepuasan SP4N-LAPOR! Capai 73,7 Persen, Menteri PANRB: Tindak Lanjut Pengaduan Harus Dipercepat.

<https://www.menpan.go.id/site/berita-terkini/kepuasan-sp4n-lapor-capai-73-7-persen-menteri-panrb-tindak-lanjut>

Rerata tindak lanjut nasional 6,1 hari kerja; total 113.989 laporan pada 2022; tingkat kepuasan pengguna 73,7 persen.

Kementerian Pendayagunaan Aparatur Negara dan Reformasi Birokrasi (2024).

2,1 Juta Laporan Masuk SP4N-Lapor, Menteri PANRB: Tindak Lanjutnya Harus Dipercepat. <https://menpan.go.id/site/berita-terkini/2-1-juta-laporan-masuk-sp4n-lapor-menteri-panrb-tindak-lanjutnya>

Total 2,1 juta laporan kumulatif pada SP4N-LAPOR! dengan 679 instansi terintegrasi (34 kementerian, 101 lembaga, 544 pemerintah daerah).

Koshy, R. and Elango, S. (2023). Multimodal tweet classification in disaster response systems using transformer-based bidirectional attention model. *Neural Computing and Applications*, 35(2):1607–1627. Klasifikasi multimodal tweet dengan Vision Transformer (citra) dan RoBERTa (teks) yang digabungkan via bidirectional attention; tujuh dataset bencana berbeda.

Koto, F., Rahimi, A., Lau, J. H., and Baldwin, T. (2020). IndoLEM and IndoBERT: A Benchmark Dataset and Pre-trained Language Model for Indonesian NLP. In *Proceedings of the 28th International Conference on Computational Linguistics (COLING 2020)*, pages 757–770. International Committee on Computational Linguistics.

- Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems 25 (NeurIPS 2012)*, pages 1097–1105.
- Kudo, T. and Richardson, J. (2018). SentencePiece: A simple and language-independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71. Association for Computational Linguistics.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.
- Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*, 60(2):91–110.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2:2.
- Microsoft (2024a). ONNX Runtime: Cross-platform, high performance ML inferencing. <https://github.com/microsoft/onnxruntime>. Version 1.18. GitHub repository.
- Microsoft (2024b). ONNX Runtime Mobile: Inference on Mobile Platforms. <https://onnxruntime.ai/docs/tutorials/mobile/>. Dokumentasi resmi ONNX Runtime Mobile untuk inferensi pada perangkat Android dan iOS.
- Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web*. O’Reilly Media, Sebastopol, CA, 2nd edition.
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill, New York.
- Offi, F., Alam, F., and Imran, M. (2020). Analysis of Social Media Data using Multimodal Deep Learning for Disaster Response. *arXiv preprint arXiv:2004.11838*,

- (arXiv:2004.11838). Diterima pada Proceedings of the 17th International Conference on Information Systems for Crisis Response and Management (ISCRAM 2020); fusi citra dan teks pada media sosial untuk klasifikasi laporan bencana.
- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jegou, H., Mairal, J., Labatut, P., Joulin, A., and Bojanowski, P. (2023). DINOv2: Learning Robust Visual Features without Supervision.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, pages 6638–6648.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning Transferable Visual Models From Natural Language Supervision. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR. CLIP, dasar dari banyak pendekatan klasifikasi multimodal modern; pretraining kontrasif citra-teks pada 400 juta pasangan dari internet.
- Ramachandram, D. and Taylor, G. W. (2017). Deep Multimodal Learning: A Survey on Recent Advances and Trends. *IEEE Signal Processing Magazine*, 34(6):96–108.
- Ramírez, S. (2024). FastAPI: Modern, fast (high-performance), web framework for building APIs with Python. <https://fastapi.tiangolo.com/>. Accessed: 2026-02-18.
- Rousselet, R. (2024). Riverpod: A reactive caching and data-binding framework for Flutter. <https://riverpod.dev>. Accessed: 2025-11-20.

- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3):211–252.
- Russell, S. J. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4th edition.
- Ryali, C., Hu, Y.-T., Bolya, D., Wei, C., Fan, H., Huang, P.-Y., Aggarwal, V., Chowdhury, A., Poole, O., Dollar, P., Girshick, R., and Feichtenhofer, C. (2023). Hierarchical Vision Transformer without the Bells-and-Whistles.
- Sadiq, T. and Omlin, C. W. (2025). Sensing in Smart Cities: A Multimodal Machine Learning Perspective.
- Sennrich, R., Haddow, B., and Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, pages 1715–1725. Association for Computational Linguistics.
- Shen, J. and Hu, C. (2025). Smart City Governance Based on Multimodal Large-Scale Models. In Guo, W., Qian, K., Xu, W., and Wang, P., editors, *Proceedings of the 3rd International Conference on Green Building, Civil Engineering and Smart City (GBCESC 2024)*, volume 264, pages 915–924. Atlantis Press International BV, Dordrecht.
- Siméoni, O., Vo, H. V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S., Ramamonjisoa, M., Massa, F., Haziza, D., Wehrstedt, L., Wang, J., Darcet, T., Moutakanni, T., Sentana, L., Roberts, C., Vedaldi, A., Tolan, J., Brandt, J., Couprie, C., Mairal, J., Jégou, H., Labatut, P., and Bojanowski, P. (2025). DINOv3: A Stronger Open Vision Foundation Model. <https://ai.meta.com/research/publications/dinov3/>. Vision foundation model.

- Siret, I. and Sabadie, W. (2022). Public complaining: A blessing in disguise? Educational calling as a benevolent process that gives consumers voice on brands' social media. *Journal of Business Research*, 150:476–490.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568:127063.
- Supabase Inc. (2024). Supabase: The Open Source Firebase Alternative. <https://supabase.com/docs>. Accessed: 2026-03-25.
- Ustimenko, A., Beliakov, A., and Prokhorenkova, L. (2023). Uncertainty estimation in CatBoost: Virtual Ensembles and Posterior Sampling.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need.
- Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., and Wei, F. (2024). Multilingual E5 Text Embeddings: A Technical Report.
- Wang, X., Li, Q., and Jia, W. (2025a). Cognitive Edge Computing: A Comprehensive Survey on Optimizing Large Models and AI Agents for Pervasive Deployment. *arXiv preprint*, (arXiv:2501.03265). Survei deployment LLM dan agen AI pada perangkat dengan sumber daya terbatas di edge jaringan.
- Wang, X., Tang, Z., Guo, J., Meng, T., Wang, C., Wang, T., and Jia, W. (2025b). Empowering Edge Intelligence: A Comprehensive Survey on On-Device AI Models. *ACM Computing Surveys*. Survei komprehensif model AI on-device, mencakup konsep dasar, skenario aplikasi, tantangan teknis, kompresi model, dan akselerasi perangkat keras.
- Wilie, B., Vincentio, K., Winata, G. I., Cahyawijaya, S., Li, X., Lim, Z. Y., Soleman, S., Mahendra, R., Fung, P., Bahar, S., and Purwarianti, A. (2020). IndoNLU: Benchmark and Resources for Evaluating Indonesian Natural Language Understanding. In *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the*

Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing (AAACL-IJCNLP 2020), pages 843–857. Association for Computational Linguistics.

Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., and Raffel, C. (2021). mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2021)*, pages 483–498. Association for Computational Linguistics.

Zahro, A. C., Alzami, F., Sani, R. R., Fahmi, A., Megantara, R. A., Naufal, M., Al Azies, H., and Iswahyudi, I. (2025). Fairer Public Complaint Classification on LaporGub: Integrating XLM-RoBERTa with Focal Loss for Imbalance Data. *Sinkron: Jurnal dan Penelitian Teknik Informatika*, 9(4):1850–1862. Klasifikasi laporan warga LaporGub Provinsi Jawa Tengah menggunakan XLM-RoBERTa dengan focal loss; akurasi 78,5 persen, macro F1 0,625.

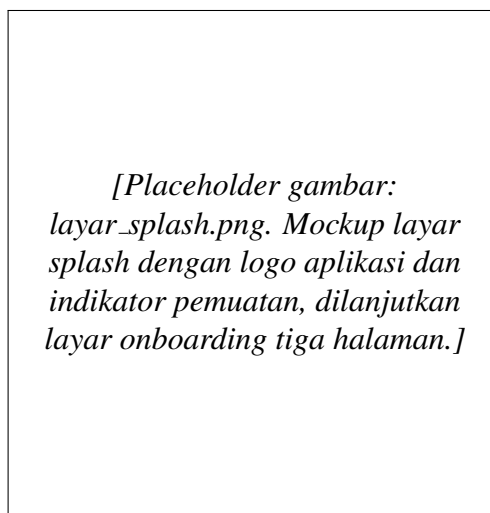
Zhang, H., Cisse, M., Dauphin, Y. N., and Lopez-Paz, D. (2018). Mixup: Beyond Empirical Risk Minimization.

LAMPIRAN A

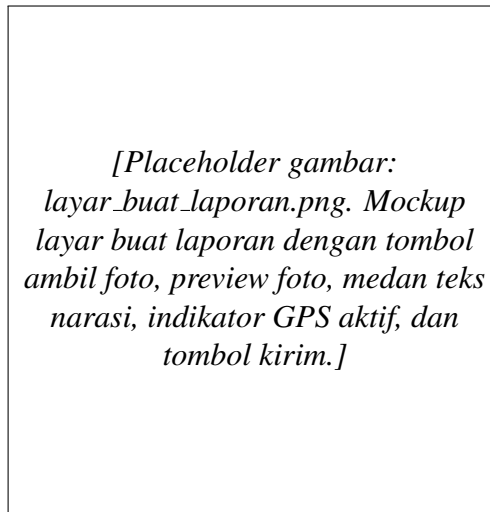
MOCKUP TAMPILAN APLIKASI SMART CITY REPORTER

Lampiran ini memuat *mockup* tampilan antarmuka untuk seluruh layar utama aplikasi *Smart City Reporter* yang dirancang pada Bagian 3.14 Bab 3. *Mockup* ditampilkan sebagai *placeholder* yang akan diganti dengan tangkapan layar aktual setelah desain antarmuka difinalisasi. Pemisahan tampilan ke lampiran menjaga pemaparan metodologi pada Bab 3 tetap padat dan memungkinkan rancangan visual ditinjau secara berdampingan.

A.1 Mockup Layar Splash dan Onboarding



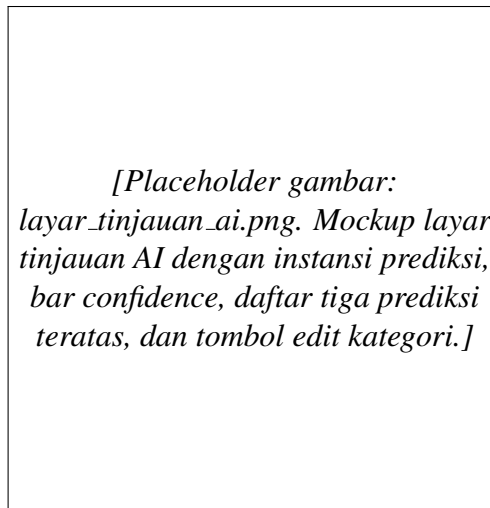
Gambar A.1. Mockup layar *splash* dan *onboarding*.



Gambar A.2. Mockup layar buat laporan.

A.2 Mockup Layar Buat Laporan

A.3 Mockup Layar Tinjauan Hasil AI



Gambar A.3. Mockup layar tinjauan hasil AI.

*[Placeholder gambar:
layar_rekomendasi.png. Mockup
layar rekomendasi instansi dengan
kartu instansi primer, instansi
pendukung, dan tombol konfirmasi.]*

Gambar A.4. Mockup layar rekomendasi instansi.

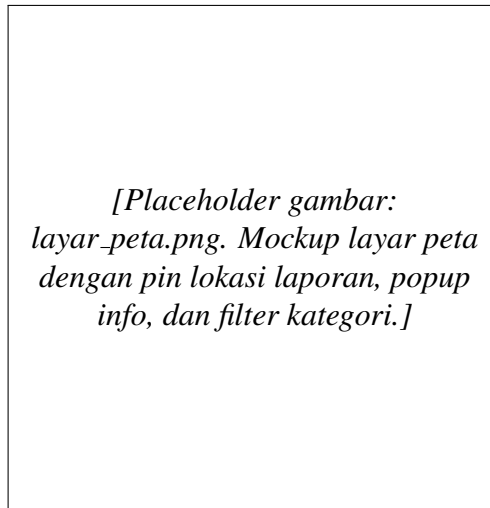
*[Placeholder gambar:
layar_riwayat.png. Mockup layar
riwayat dengan daftar laporan,
masing-masing memuat thumbnail
foto, kategori, tanggal, dan status.]*

Gambar A.5. Mockup layar riwayat laporan.

A.4 Mockup Layar Rekomendasi Instansi

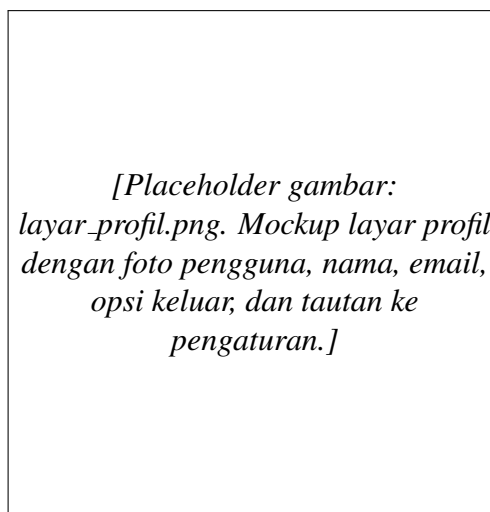
A.5 Mockup Layar Riwayat Laporan

A.6 Mockup Layar Peta Laporan



Gambar A.6. Mockup layar peta laporan.

A.7 Mockup Layar Profil dan Pengaturan

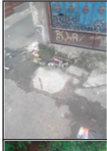


Gambar A.7. Mockup layar profil dan pengaturan.

LAMPIRAN B

CONTOH SAMPEL MULTIMODAL TAMBAHAN

Lampiran ini menyajikan dua contoh baris dataset tambahan yang melengkapi Gambar 3.6 pada Bagian 3.3. Kedua contoh berikut dipilih untuk menggambarkan variasi panjang narasi dan jenis kejadian, sehingga sifat multimodal serta komplemen-ter antara citra dan teks dapat ditinjau lebih luas tanpa membebani pembahasan utama Bab 3.

Gambar	Laporan	Label
	saluran air / got yg mampet sudah di bersihkan secara mandiri tetapi tetap mampet butuh penanganan khusus dari pihak terkait. kalo dibiarkan akan banyak yg terdampak apalagi kalo hujan turun bisa menyebabkan banjir	Kelurahan

Gambar B.1. Contoh 2 struktur data multimodal: foto saluran air mampet dengan narasi panjang yang menjelaskan dampak banjir terhadap warga sekitar, dipetakan ke label *Kelurahan*. Narasi yang panjang menyediakan konteks tekstual kuat, sedangkan citra memvalidasi kondisi fisik saluran.

Gambar	Laporan	Label
	pembatas jalan sudah pudar dan sudah jelek tolong di cat ulang ya terimakasih	Dinas Perhubungan

Gambar B.2. Contoh 3 struktur data multimodal: foto pembatas jalan dengan cat yang sudah pudar, narasi permohonan pengecatan ulang oleh warga, dipetakan ke label *Dinas Perhubungan*. Kondisi visual pudar pada citra menjadi sinyal utama, sedangkan narasi menegaskan jenis tindakan yang diharapkan.

LAMPIRAN C

RINCIAN PEMROSESAN, EKSTRAKSI *EMBEDDING*, DAN FUSI MULTIMODAL DINI

Lampiran ini menampung rincian teknis tahap pemrosesan data multimodal, ekstraksi *embedding* pada *encoder* beku, dan skema fusi awal yang diringkas pada Bab 3. Pemindahan ke lampiran dimaksudkan agar pembahasan Bab 3 tetap fokus pada keputusan metodologis tingkat tinggi, sementara langkah teknis tetap dapat diaudit secara mendetail.

C.1 Pemrosesan Data Gambar

Setiap gambar masukan diolah dengan urutan langkah berikut sebelum dilewatkan ke *encoder* citra.

1. **Pembacaan dan konversi:** berkas JPEG/PNG dibaca menggunakan `PIL . Image . open`, kemudian dikonversi ke *mode* RGB.
2. **Koreksi orientasi EXIF:** tag orientasi sensor kamera dibaca dan citra diputar ke orientasi normal agar foto vertikal tidak diumpankan dalam orientasi miring.
3. **Resize:** sisi pendek citra di-*resize* ke 256 piksel (untuk DINOv2/DINOv3/Hiera) atau 512 piksel (untuk EVA-02) menggunakan interpolasi bilinear.
4. **Center crop:** pemotongan tengah ke ukuran target 224x224 atau 448x448.
5. **Normalisasi per kanal:** piksel dinormalisasi dengan rerata $\mu = (0,485, 0,456, 0,406)$ dan simpangan baku $\sigma = (0,229, 0,224, 0,225)$ ImageNet sesuai Persamaan F.1.

6. **Layout tensor:** konversi ke tensor `float32` dengan *layout* NCHW (*batch, channel, height, width*).

Karena *encoder* citra dibekukan dan tidak dilatih ulang, augmentasi data tidak diterapkan pada saat ekstraksi *embedding*.

C.2 Pemrosesan Data Teks

Narasi laporan pada kolom `laporan` diolah dengan urutan langkah berikut.

1. **Penghapusan URL:** *regex* `https?://\S+|www\.\S+` dipakai untuk menghapus tautan yang sering muncul pada laporan.
2. **Normalisasi unicode:** konversi ke bentuk NFC.
3. **Whitespace collapsing:** penggantian beberapa spasi atau *tab* menjadi satu spasi.
4. **Pembersihan karakter kontrol:** penghapusan karakter non-cetak.
5. **Pemotongan:** pembatasan panjang teks ke 256 token, yang mencakup sekitar 98,5% laporan tanpa pemotongan.
6. **Pemberian prefiks:** untuk *multilingual-E5-Large-Instruct*, teks diberi prefiks `Instruct: Classify Indonesian civic report\nQuery: se-` sebelum tokenisasi, mengikuti konvensi varian *instruct*.
7. **Tokenisasi:** setiap teks ditokenisasi menggunakan tokenizer bawaan masing-masing *encoder* (SentencePiece atau WordPiece) dengan parameter `max_length=256`, `padding="max_length"`, dan `truncation=True`.

Tidak diterapkan *stopword removal* atau *stemming*, karena *Transformer encoder* sudah menangani morfologi kata secara natif.

C.3 Encoder Citra yang Dievaluasi

Empat *encoder* citra dipilih sebagai kandidat ablasi, semuanya menghasilkan vektor *embedding* berdimensi 1024. Daftar *encoder* dirangkum pada Tabel C.1.

Tabel C.1. Daftar *encoder* citra yang dievaluasi.

Model	Model ID	Dim	Input	Framework
DINOv2-L	dinov2-large	1024	224	PyTorch
DINOv3-ViT-L	dinov3-vitl16	1024	224	PyTorch
Hiera-L	hiera_large_224	1024	224	PyTorch
EVA-02-L	eva02_large_patch14_448	1024	448	PyTorch

Vektor representasi diambil dari token CLS atau rata-rata token *patch* sesuai konvensi masing-masing *encoder*, kemudian dinormalisasi L2.

C.4 Encoder Teks yang Dievaluasi

Empat *encoder* teks dipilih sebagai kandidat ablasi, mencakup *encoder* multi-bahasa generik dan *encoder* khusus Bahasa Indonesia. Daftar dirangkum pada Tabel C.2.

Tabel C.2. Daftar *encoder* teks yang dievaluasi.

Model	Model ID	Dim	Pool Strategy
mE5-Large-Instruct	multilingual-e5-large-instruct	1024	Mean masked
BGE-M3	bge-m3	1024	CLS
IndoBERT-Large-p2	indobert-large-p2	1024	CLS
Cendol-mT5-Large	cendol-mt5-large-inst	1024	Mean (encoder)

Output setiap *encoder* dinormalisasi L2 setelah *pooling*.

C.5 Pipeline Inferensi Batch dan Caching Embedding

Inferensi *batch* dilakukan dengan *loop* di atas `DataLoader` PyTorch yang membaca masukan secara *streaming* dari direktori gambar atau `Dataset` teks.

Konteks `torch.no_grad()` dan `torch.autocast(dtype=torch.float16)` dipakai untuk efisiensi memori. Vektor keluaran dikonversi ke `numpy float32` sebelum dituliskan ke berkas *cache*.

Embedding hasil ekstraksi disimpan pada berkas terkompresi yang dikelompokkan per *encoder* dan per partisi data, dengan identitas baris yang sejajar dengan label. Strategi *caching* ini memungkinkan eksperimen ablasi pada algoritma klasifikasi dilakukan tanpa perlu menjalankan ulang *encoder*, sehingga setiap kombinasi (*encoder* citra, *encoder* teks) dapat dievaluasi dengan biaya komputasi yang sangat rendah.

C.6 Rincian Fusi Multimodal Dini

Penelitian ini mengadopsi skema fusi awal (*early fusion*) berbasis konkatenasi, sebagaimana metodologi yang dipublikasikan oleh Hanif et al. (2024). Sebelum konkatenasi, setiap *embedding* dinormalisasi L2 secara independen agar magnitudonya seragam dan tidak ada modalitas yang mendominasi karena perbedaan skala absolut. Normalisasi L2 didefinisikan sebagai $\mathbf{x}_{\text{norm}} = \mathbf{x} / \|\mathbf{x}\|_2$.

Vektor fusi akhir adalah konkatenasi langsung sesuai Persamaan C.1:

$$\mathbf{e}_{\text{fuse}} = [\mathbf{e}_{\text{img}}^{\text{L2}}; \mathbf{e}_{\text{txt}}^{\text{L2}}] \in \mathbb{R}^{2048}. \quad (\text{C.1})$$

Vektor inilah yang menjadi masukan tunggal bagi algoritma klasifikasi CatBoost.

Sebagaimana dibahas pada Bagian 2.9, PCA dapat dipakai untuk mereduksi dimensi *embedding*. Pada penelitian ini PCA **tidak diterapkan** dengan dua pertimbangan, yaitu (1) CatBoost berbasis pohon keputusan *oblivious* sehingga mampu menangani fitur padat berdimensi tinggi tanpa penalti *curse of dimensionality* yang berarti, dan (2) reduksi dimensi linear seperti PCA dapat menghilangkan informasi diskriminatif yang justru diperlukan untuk membedakan kategori dinas yang dekat secara semantik.

LAMPIRAN D

SKEMA BASIS DATA SUPABASE LENGKAP

Lampiran ini menampilkan rincian kolom seluruh tabel basis data Supabase, daftar *enum* pendukung, dan kebijakan *Row Level Security* (RLS) yang diringkas pada Bagian 3.11. Pemisahan dilakukan agar pembahasan rancangan sistem pada Bab 3 tetap padat sementara skema penuh tetap dapat diaudit secara mendetail.

D.1 Tabel `profiles`

Tabel `profiles` memuat informasi profil pengguna yang terhubung satu-ke-satu dengan tabel sistem `auth.users` milik Supabase Auth. Atribut utama dirangkum pada Tabel D.1.

Tabel D.1. Atribut tabel `profiles`.

Kolom	Tipe	Deskripsi
<code>id</code>	<code>uuid (PK)</code>	Mengacu ke <code>auth.users.id</code> .
<code>full_name</code>	<code>text</code>	Nama lengkap pengguna.
<code>email</code>	<code>text</code>	Alamat email pengguna.
<code>phone_number</code>	<code>text</code>	Nomor telepon kontak.
<code>profile_photo_url</code>	<code>text</code>	URL foto profil pada Supabase Storage.
<code>is_moderator</code>	<code>bool</code>	Penanda peran moderator.
<code>role</code>	<code>user_role</code>	<i>Enum</i> peran (warga, operator, moderator).
<code>assigned_agency</code>	<code>issue_category</code>	<i>Enum</i> instansi penugasan operator.
<code>created_at</code>	<code>timestampz</code>	Waktu pembuatan profil.
<code>updated_at</code>	<code>timestampz</code>	Waktu perubahan terakhir.

D.2 Tabel `reports`

Tabel `reports` adalah tabel utama yang menyimpan setiap laporan warga beserta hasil klasifikasi AI dan metadata lokasi. Atribut utama dirangkum pada Tabel D.2.

D.3 Tabel `report_history`

Tabel `report_history` mencatat setiap perubahan status pada laporan agar jejak audit (*audit trail*) dapat ditelusuri dari awal pengiriman hingga penyelesaian. Atribut utama dirangkum pada Tabel D.3.

D.4 Enum Pendukung

Skema mendefinisikan tiga *enum* kustom PostgreSQL yang dipakai bersama oleh tabel-tabel di atas, yaitu:

- `issue_category`: kategori dinas dengan nilai sesuai sembilan kategori target pada Tabel 3.3.
- `report_status`: status siklus hidup laporan, yaitu *submitted*, *verified*, *in_progress*, *resolved*, *rejected*.
- `user_role`: peran pengguna, yaitu *citizen*, *agency_admin*, *super_admin*.

D.5 Row Level Security

Kebijakan *Row Level Security* (RLS) diaktifkan pada ketiga tabel agar akses data terkontrol pada level baris. Beberapa kebijakan utama yang dirancang adalah:

- Setiap pelapor hanya dapat membaca dan memperbarui baris `reports` miliknya sendiri (`auth.uid() = user_id`).
- Petugas dengan `role = 'agency_admin'` dapat membaca dan memperbarui laporan pada `assigned_agency` yang sesuai dengan `category`.

Tabel D.2. Atribut tabel `reports`.

Kolom	Tipe	Deskripsi
<code>id</code>	uuid (PK)	Identitas unik laporan.
<code>user_id</code>	uuid (FK)	Mengacu ke <code>profiles.id</code> .
<code>reporter_name</code>	text	Nama pelapor saat mengirim.
<code>reporter_email</code>	text	Email pelapor saat mengirim.
<code>category</code>	issue_category	<i>Enum</i> kategori dinas akhir.
<code>ai_prediction</code>	text	Kategori dinas yang diprediksi AI.
<code>ai_confidence</code>	numeric	Tingkat keyakinan model (0..1).
<code>ai_probabilities</code>	jsonb	Skor probabilitas untuk setiap dari sembilan instansi target.
<code>ai_model_*</code>	text	Metadata model server-side, mencakup nama dan versi model.
<code>assigned_*</code>	text/float8	Hasil routing instansi terdekat, mencakup identitas, nama, kategori, jarak, dan metode routing.
<code>image_url</code>	text	URL gambar pada Supabase Storage.
<code>description</code>	text	Narasi laporan dalam Bahasa Indonesia.
<code>latitude</code>	float8	Koordinat lintang lokasi kejadian.
<code>longitude</code>	float8	Koordinat bujur lokasi kejadian.
<code>address</code>	text	Alamat hasil <i>reverse geocoding</i> .
<code>status</code>	report_status	<i>Enum</i> status (<i>submitted</i> , <i>verified</i> , <i>in_progress</i> , <i>resolved</i> , <i>rejected</i>).
<code>verified_by</code>	uuid (FK)	Operator yang memverifikasi laporan.
<code>verified_at</code>	timestampz	Waktu verifikasi.
<code>rejection_reason</code>	text	Alasan penolakan bila status <i>rejected</i> .
<code>resolution_*</code>	text/uuid/timestampz	Bukti dan metadata penyelesaian, mencakup foto bukti, catatan, petugas, dan waktu selesai.
<code>created_at</code>	timestampz	Waktu laporan dikirim.
<code>updated_at</code>	timestampz	Waktu perubahan terakhir.

Tabel D.3. Atribut tabel `report_history`.

Kolom	Tipe	Deskripsi
<code>id</code>	uuid (PK)	Identitas unik catatan riwayat.
<code>report_id</code>	uuid (FK)	Mengacu ke <code>reports.id</code> .
<code>status</code>	<code>report_status</code>	Status baru pada langkah ini.
<code>note</code>	text	Catatan operator atau sistem.
<code>updated_by</code>	uuid	Pengguna yang memicu perubahan.
<code>created_at</code>	timestampz	Waktu perubahan.

- Admin dengan `role = 'super_admin'` memiliki akses lintas instansi, termasuk koreksi penugasan.
- Tabel `report_history` hanya dapat ditulis melalui pemicu (*trigger*) basis data saat status `reports` berubah.

LAMPIRAN E

KONFIGURASI RUNPOD, WORKFLOW SSH, DAN MANAJEMEN LINGKUNGAN

Lampiran ini merinci alur kerja akses *pod* RunPod via SSH serta praktik manajemen lingkungan yang diringkas pada Bagian 3.4. Konteks visual alur kerja sudah disajikan pada Gambar 3.3 di Bab 3.

E.1 Akses Pod via SSH dan *Workflow*

Akses ke *pod* dilakukan melalui kunci publik SSH yang didaftarkan pada akun RunPod. *Workflow* pelatihan terdiri atas langkah-langkah berikut.

1. **Penyiapan kunci SSH:** pembuatan pasangan kunci pada mesin lokal dan unggah kunci publik ke profil RunPod.
2. **Login ke pod:** `ssh root@<pod-host> -p <port> -i ~/.ssh/id_ed25519.`
3. **Sinkronisasi dataset:** pengiriman dataset bersih dan direktori gambar dari mesin lokal ke *pod* menggunakan `rsync` melalui kanal SSH.
4. **Penyiapan lingkungan Python:** pemasangan dependensi pelatihan, mencakup `torch`, `transformers`, `timm`, `catboost`, dan `onnxruntime-gpu`.
5. **Eksekusi pelatihan:** peluncuran Jupyter Lab pada *pod* dan akses dari peramban lokal melalui *SSH tunnel* ke *port* 8888.
6. **Penyimpanan *artifact*:** *embedding* dan model akhir disimpan pada *persistent volume pod*.

7. **Pengunduhan hasil:** *embedding cache* dan *checkpoint* CatBoost diunduh kembali ke mesin lokal menggunakan `rsync` setelah pelatihan selesai.

E.2 Manajemen Lingkungan

Daftar dependensi dengan versi presisi dipertahankan sebagai berkas konfigurasi pelatihan sehingga lingkungan dapat direproduksi pada *pod* baru. Eksekusi *forward pass encoder* memanfaatkan presisi campuran (*Automatic Mixed Precision*) FP16 untuk mengurangi konsumsi VRAM dan mempercepat komputasi. Ukuran *batch* disesuaikan per *encoder*, contohnya 32 untuk DINOv3 pada masukan 224x224 dan 8 untuk EVA-02 pada masukan 448x448, berdasarkan ketersediaan VRAM.

LAMPIRAN F

LANDASAN TEORI PEMROSESAN MULTIMODAL DAN REDUKSI DIMENSI

Lampiran ini menampung tiga bagian landasan teori yang sebelumnya berada di Bab 2 namun tidak berkontribusi langsung sebagai keputusan metodologi pada Bab 3. Pemindahan dilakukan untuk menjaga pembahasan Bab 2 tetap fokus pada teori yang dipakai pada pipeline akhir, sementara latar teori yang lebih umum tetap dapat ditinjau di sini.

F.1 Landasan Teori Pemrosesan Data Gambar

Bagian ini menguraikan kerangka teoretis pra-proses citra pada *pipeline* klasifikasi multimodal. Pada penelitian ini seluruh pra-proses citra didelegasikan kepada *AutoImageProcessor* dari pustaka HuggingFace, khususnya kelas *DINOv3ViTImageProcessorFast*, yang dimuat sekali pada saat inisialisasi server FastAPI dan mengikuti spesifikasi resmi yang dipublikasikan bersama bobot DINOv3. Konfigurasi pra-proses tersimpan pada berkas `preprocessor_config.json` dan menjadi sumber kebenaran (*source of truth*) untuk seluruh tahap inferensi. Subbagian berikut memaparkan komponen-komponen yang aktif pada konfigurasi tersebut beserta latar teoretisnya.

F.1.1 Citra Digital sebagai Tensor

Citra digital adalah representasi visual objek yang diubah ke dalam format numerik melalui perangkat akuisisi data seperti kamera. Angka tersebut mencerminkan intensitas cahaya pada setiap elemen gambar (*pixel*), yang memungkinkan analisis dan manipulasi melalui teknik pengolahan citra digital (Fadjeri et al., 2022).

Pada *deep learning*, satu citra berwarna direpresentasikan sebagai tensor berukuran $H \times W \times C$, dengan H tinggi, W lebar, dan C adalah jumlah kanal warna (umumnya 3 untuk RGB). Nilai piksel umumnya berada pada rentang bulat $[0, 255]$ untuk berkas mentah `uint8`, atau pada rentang *floating point* $[0, 1]$ atau $[-1, 1]$ setelah normalisasi.

F.1.2 *Resize* dan Interpolasi

Karena *encoder* citra menerima masukan dengan ukuran tetap, setiap citra harus dilakukan *resize* ke resolusi target. Tiga algoritma interpolasi yang paling umum adalah:

- **Nearest Neighbor:** piksel keluaran mengambil nilai piksel terdekat dari masukan. Cepat namun menghasilkan tepi bergerigi.
- **Bilinear:** rata-rata berbobot dari empat piksel tetangga. Halus dan menjadi standar pada banyak *pipeline*.
- **Bicubic:** rata-rata berbobot dari enam belas piksel tetangga dengan polinomial kubik. Lebih tajam namun lebih mahal.

AutoImageProcessor pada DINOv3 melakukan *resize* ke resolusi target 224×224 piksel mengikuti parameter `size` dan `interpolation` yang ditetapkan oleh kartu model resmi.

F.1.3 Aspek Rasio dan *Square Padding*

Pada konfigurasi *AutoImageProcessor* DINOv3 berlaku `default_to_square : true` dengan `do_center_crop` dinonaktifkan, sehingga citra dipaksa berbentuk persegi melalui kombinasi *resize* dan *padding* pada sisi yang lebih pendek. Strategi ini berbeda dengan *center crop* klasik yang memotong tepi citra setelah *resize* sisi pendek; *square padding* mempertahankan seluruh konten citra namun menambahkan piksel netral pada sisi yang kekurangan ukuran. Pada konteks laporan warga, pendekatan ini menguntungkan karena objek penting (lubang jalan, pohon tum-

bang, sampah) sering kali tersebar tidak terpusat sehingga *cropping* tepi berisiko membuang informasi diskriminatif.

F.1.4 Normalisasi Per-Kanal

Setelah *resize*, citra dinormalisasi per kanal menggunakan rerata dan simpangan baku ImageNet:

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}, \quad \mu = (0,485, 0,456, 0,406), \quad \sigma = (0,229, 0,224, 0,225), \quad (\text{F.1})$$

dengan c adalah indeks kanal RGB. Nilai μ dan σ ini termuat secara eksplisit pada `preprocessor_config.json` (kunci `image_mean` dan `image_std`) dan wajib digunakan ketika *encoder* DINOv3 pra-latih, agar distribusi masukan pada saat inferensi konsisten dengan distribusi yang dipakai pada saat pra-pelatihan.

F.1.5 Layout Tensor dan Konversi Warna

Dua *layout* tensor yang umum adalah NCHW (*batch, channel, height, width*) yang dipakai PyTorch dan ONNX, serta NHWC yang dipakai TensorFlow. Selain itu, beberapa pustaka (contohnya OpenCV) menggunakan urutan kanal BGR sementara model *deep learning* biasanya mengasumsikan RGB. *AutoImageProcessor* pada penelitian ini diatur dengan `data_format: channels_first` sehingga keluaran tensor langsung berformat NCHW. Konversi kanal ke RGB dilakukan secara eksplisit melalui pemanggilan `Image.open(...).convert("RGB")` pada lapisan *encoder* citra sehingga sumber gambar berformat lain (BGR, RGBA, atau *grayscale*) tetap menghasilkan tensor masukan yang konsisten.

F.1.6 Kompresi JPEG dan EXIF

Berkas JPEG biasanya sudah dikompresi dengan kerugian (*lossy*) dan dilengkapi metadata EXIF, termasuk orientasi sensor kamera. Pustaka PIL yang digunakan

oleh *AutoImageProcessor* membaca dan menerapkan tag orientasi EXIF secara otomatis melalui pemanggilan `Image.open()`, sehingga foto vertikal yang diambil dari kamera ponsel tidak dilewatkan ke model dalam orientasi miring. Kompresi JPEG yang berlebihan juga dapat menurunkan akurasi model, namun pada *pipeline* produksi penelitian ini gambar yang diunggah pengguna disimpan langsung pada Supabase Storage sebagai berkas asli sehingga tidak terjadi kompresi tambahan setelah pengambilan citra.

F.1.7 Catatan tentang Augmentasi Data

Teknik augmentasi data seperti *horizontal flip*, *color jitter*, *random erasing*, RandAugment (Cubuk et al., 2020), dan MixUp (Zhang et al., 2018) merupakan komponen standar pada pelatihan *encoder* citra dari nol. Pada penelitian ini teknik tersebut tidak diterapkan karena *encoder* DINOv3 dibekukan (*frozen*) selama seluruh tahap eksperimen dan hanya dipakai pada modus inferensi (`is_training=False`). *Embedding* dihitung sekali per sampel dan disimpan pada berkas *cache* sehingga ablasi pada *classifier head* dapat dijalankan tanpa perlu menjalankan kembali *encoder*. Augmentasi data tetap dibahas di sini sebagai latar teoretis karena merupakan teknik penting yang relevan ketika DINOv3 di masa depan akan dilatih ulang penuh atau di-*fine-tune* pada domain spesifik laporan warga.

F.2 Landasan Teori Pemrosesan Data Teks

F.2.1 Pembersihan dan Normalisasi

Sebelum dilewatkan ke *encoder* teks, sebuah dokumen biasanya melalui beberapa langkah pembersihan, yaitu:

- Normalisasi unicode ke bentuk NFC (*Canonical Composition*) agar karakter ber-aksen direpresentasikan secara konsisten.
- Konversi huruf ke *lowercase* jika model toleran terhadap kasus, atau dipertahankan apabila model membedakan kasus.

- Penghapusan karakter kontrol dan *whitespace* berlebih (*whitespace collapsing*).
- Substitusi emoji yang sangat jarang dengan *placeholder*, untuk menghindari pembengkakan kosakata.
- Pemotongan teks menjadi panjang maksimum, biasanya 128 atau 512 token, sesuai batas *encoder*.

F.2.2 Tokenisasi

Tokenisasi adalah proses memecah teks menjadi unit diskrit yang dapat dipetakan ke indeks. Tiga keluarga utama tokenizer adalah:

1. **Word-level:** setiap kata adalah satu token. Sederhana namun rentan terhadap kosakata yang membengkak dan kata di luar kosakata (*out-of-vocabulary*).
2. **Character-level:** setiap karakter adalah satu token. Tahan terhadap *out-of-vocabulary*, namun barisan menjadi sangat panjang.
3. **Subword:** kompromi antara dua pendekatan di atas, yaitu memecah kata menjadi unit subkata yang sering muncul. Ini adalah pilihan dominan pada *Transformer* modern.

Empat algoritma subword yang paling umum:

- **Byte Pair Encoding (BPE)** (Sennrich et al., 2016): mulai dari karakter sebagai unit, secara iteratif menggabungkan pasangan unit yang paling sering bersamaan menjadi satu unit baru hingga ukuran kosakata target tercapai.
- **WordPiece:** varian BPE yang dipakai oleh BERT (Devlin et al., 2019), dengan kriteria *merge* berbasis *likelihood* alih-alih frekuensi.
- **SentencePiece** (Kudo and Richardson, 2018): *wrapper* BPE atau Unigram yang bekerja pada level byte sehingga tidak memerlukan pra-proses spesifik bahasa, cocok untuk tokenizer multibahasa.

- **Unigram Language Model:** memodelkan distribusi token dengan probabilitas independen, kemudian memilih segmentasi yang memaksimalkan *likelihood*.

Tabel berikut merangkum perbandingan singkat keempatnya.

Tabel F.1. Perbandingan algoritma tokenizer subword.

Algoritma	Kriteria <i>merge</i>	Implementasi populer	Catatan
BPE	Frekuensi pasangan	GPT-2, RoBERTa	Sederhana, deterministik
WordPiece	<i>Likelihood</i> pasangan	BERT, mBERT, IndoBERT	Memakai prefiks ##
SentencePiece	BPE atau Unigram di level byte	T5, mT5, XLM-R, E5	Tidak butuh pra-tokenisasi spasi
Unigram LM	Probabilitas token	SentencePiece (mode Unigram)	Sampling segmentasi mungkin

F.2.3 *Embedding: Lookup Matrix* dan Dimensi

Setelah token diubah menjadi indeks bilangan bulat, indeks tersebut dipetakan ke vektor berdimensi d melalui matriks *lookup* berukuran $|V| \times d$, dengan $|V|$ ukuran kosakata. Vektor inilah yang menjadi masukan ke lapisan *Transformer* berikutnya.

F.3 Reduksi Dimensi dan *Principal Component Analysis*

F.3.1 Motivasi

Vektor representasi (*embedding*) yang dihasilkan oleh *encoder* modern umumnya berdimensi sangat tinggi, contohnya 768, 1024, atau bahkan 4096 untuk model besar. Pada banyak skenario, dimensi tinggi dapat menjadi masalah karena tiga alasan, yaitu (1) biaya memori dan komputasi pada model klasifikasi hilir, (2) *curse of dimensionality* pada metode berbasis jarak (k-Nearest Neighbors, klastering jarak Euclidean), dan (3) sulitnya visualisasi serta interpretasi. Reduksi dimensi adalah

keluarga teknik yang memetakan vektor berdimensi tinggi ke ruang berdimensi lebih rendah dengan menjaga sebanyak mungkin informasi yang relevan.

F.3.2 *Principal Component Analysis (PCA)*

PCA (Jolliffe, 2002) adalah metode reduksi dimensi linear yang menemukan basis ortogonal baru sehingga proyeksi data ke basis tersebut memaksimalkan varians yang dipertahankan. Diberikan matriks data $\mathbf{X} \in \mathbb{R}^{n \times d}$ yang sudah dipusatkan (*mean-centered*) sehingga setiap kolom memiliki rerata nol, matriks kovarians sampel dihitung sebagai:

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}. \quad (\text{F.2})$$

Vektor eigen $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_d$ dari $\mathbf{C}$ yang diurutkan berdasarkan nilai eigen $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$ disebut *principal components*. Komponen pertama menjelaskan varians terbesar; komponen kedua menjelaskan varians terbesar berikutnya pada arah yang ortogonal terhadap komponen pertama, dan seterusnya. Reduksi ke k dimensi dilakukan dengan memilih k vektor eigen pertama yang dikumpulkan ke matriks $\mathbf{V}_k \in \mathbb{R}^{d \times k}$, kemudian memproyeksikan data:

$$\mathbf{Z} = \mathbf{XV}_k, \quad \mathbf{Z} \in \mathbb{R}^{n \times k}. \quad (\text{F.3})$$

Implementasi praktis sering menggunakan *Singular Value Decomposition* (SVD) pada $\mathbf{X}$, yaitu $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, sehingga vektor eigen kovarians sama dengan kolom $\mathbf{V}$ dan nilai eigen sama dengan kuadrat nilai singular dibagi $n-1$. Pemilihan jumlah komponen k umumnya dipandu oleh *explained variance ratio*, yaitu rasio kumulatif $\sum_{i=1}^k \lambda_i / \sum_{i=1}^d \lambda_i$, dengan ambang umum 0,90 hingga 0,99.

F.3.3 *Penggunaan dalam Pipeline Embedding*

Pada *pipeline* klasifikasi berbasis *embedding*, PCA dapat dipakai untuk tiga tujuan:

- **Pra-proses fitur:** mengurangi dimensi *embedding* sebelum dilewatkan ke algoritma klasifikasi yang sensitif terhadap dimensi tinggi (contohnya regresi logistik dengan regularisasi atau k-NN).
- **Whitening:** PCA dengan normalisasi $1/\sqrt{\lambda_i}$ pada setiap komponen menghasilkan ruang yang fitur-fiturnya tidak berkorelasi dan memiliki skala seragam.
- **Visualisasi:** memproyeksikan *embedding* ke 2 atau 3 dimensi pertama untuk inspeksi kluster kualitatif, walaupun untuk visualisasi non-linear teknik t-SNE atau UMAP umumnya lebih informatif.

F.3.4 Keterbatasan dan Alternatif

PCA bersifat linear, sehingga tidak dapat menangkap struktur non-linear yang sering muncul pada *embedding* jaringan saraf dalam. Alternatif non-linear yang umum digunakan adalah *Kernel PCA*, *Autoencoder*, t-SNE, dan UMAP. Selain itu, PCA tidak memperhatikan label kelas; bila reduksi dimensi yang *discriminative* terhadap kelas dibutuhkan, *Linear Discriminant Analysis* (LDA) menjadi pilihan yang lebih tepat. Pada konteks pohon keputusan seperti CatBoost atau XGBoost, reduksi dimensi melalui PCA sering kali tidak memberi keuntungan karena pohon keputusan memilih dimensi yang relevan secara implisit melalui pemilihan *split* pada setiap simpul.

LAMPIRAN G

LANDASAN TEORI PELENGKAP

Lampiran ini menampung uraian teoretis yang sebelumnya berada di Bab 2 namun bersifat latar umum dan tidak menjadi keputusan metodologi langsung pada Bab 3. Pemindahan dilakukan agar Bab 2 tetap padat dan terfokus pada teori yang dipakai pada *pipeline* akhir, yaitu *encoder* DINOv3, *multilingual-E5*, fusi awal, dan *classifier head* CatBoost.

G.1 *Web Scraping*: Klasifikasi Halaman, Alur, dan Pustaka

Halaman web yang menjadi target *scraping* dibedakan menjadi dua kelompok berdasarkan cara konten dirender. **Halaman statis** adalah halaman yang seluruh kontennya sudah terbentuk pada respons HTML server, sehingga cukup memerlukan pengunduh HTTP dan pengurai HTML. **Halaman dinamis** adalah halaman yang kontennya dirender oleh JavaScript pada peramban (*client-side rendering*), sehingga memerlukan eksekusi JavaScript melalui peramban tanpa kepala (*headless browser*) atau pemanggilan API XHR yang ditemukan melalui inspeksi jaringan.

Alur umum *web scraping* terdiri atas lima tahap, yaitu (1) identifikasi sumber (halaman daftar, halaman rincian, dan pola URL), (2) pengunduhan melalui permintaan HTTP GET dengan memperhatikan *user-agent*, *cookie*, dan pembatasan kecepatan, (3) penguraian (*parsing*) elemen HTML yang relevan, (4) pembersihan dan normalisasi karakter, tanggal, serta pemetaan label bebas ke kategori standar, dan (5) penyimpanan data terstruktur (CSV, JSONL, Parquet) atau basis data relasional.

Beberapa pustaka populer pada ekosistem Python untuk *web scraping* antara

lain `requests` (pustaka HTTP sederhana untuk halaman statis), `BeautifulSoup` (pengurai HTML yang fleksibel), `lxml` (pengurai cepat berbasis `libxml2` dengan dukungan XPath), `Scrapy` (*framework* berbasis *spider* dan *pipeline* untuk *scraping* berskala besar), `Selenium` dan `Playwright` (otomatisasi peramban yang dapat mengeksekusi JavaScript pada halaman dinamis), serta `httpx` (alternatif modern `requests` dengan dukungan asinkron).

G.2 Multi-Head Attention: Formulasi Lengkap

Multi-head attention adalah perluasan *single-head attention* yang menjalankan beberapa proyeksi linear paralel dari *query*, *key*, dan *value*, kemudian menggabungkan keluarannya. Motivasi desain ini adalah bahwa satu *head* terbatas pada satu pola hubungan antar token, sementara teks dan citra alami memuat banyak pola hubungan secara simultan.

Misalkan dimensi *embedding* model adalah d_{model} dan jumlah *head* adalah h . Setiap *head* bekerja pada subruang berdimensi $d_k = d_{\text{model}}/h$. Untuk setiap *head* $i \in \{1, \dots, h\}$ didefinisikan tiga matriks proyeksi yang dapat dilatih:

$$\mathbf{Q}_i = \mathbf{Q}\mathbf{W}_i^Q, \quad \mathbf{K}_i = \mathbf{K}\mathbf{W}_i^K, \quad \mathbf{V}_i = \mathbf{V}\mathbf{W}_i^V, \quad (\text{G.1})$$

dengan $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ dan $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Setiap *head* menjalankan *scaled dot-product attention* secara independen:

$$\text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_k}}\right) \mathbf{V}_i. \quad (\text{G.2})$$

Keluaran h *head* dikonkatenasi sepanjang dimensi terakhir, kemudian diproyeksikan kembali ke ruang berdimensi d_{model} melalui matriks proyeksi keluaran $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O. \quad (\text{G.3})$$

Dimensi d_k pada setiap *head* biasanya jauh lebih kecil daripada d_{model} , con-

tohnya $d_{\text{model}} = 768$ dengan $h = 12$ menghasilkan $d_k = 64$, sehingga total komputasi tetap sebanding dengan *single-head attention* berdimensi d_{model} . Pada model bahasa modern jumlah *head* umumnya berkisar antara 8 dan 32, dan penambahan *head* melampaui jumlah optimal sering kali tidak meningkatkan performa serta membuat sebagian *head* menjadi redundan.

G.3 Backbone Visual Pembanding: EVA-02 dan Hiera

G.3.1 EVA-02

EVA-02 (Fang et al., 2023) adalah *vision foundation model* generasi kedua dari seri EVA yang menggabungkan *Masked Image Modeling* (MIM) dan penyelarasan dengan representasi CLIP. Inovasi utamanya adalah penggunaan fitur CLIP terlatih sebagai target sinyal supervisi dalam proses MIM, alih-alih piksel. Pada sisi arsitektur, EVA-02 memperbaiki ViT melalui Sub-LN (penambahan *layer normalization* di dalam blok *attention* dan FFN), SwiGLU FFN (varian *gated* pengganti FFN GELU), dan 2D RoPE (*rotary position embedding* dua dimensi). EVA-02 dilatih pada dataset Merged-2B dengan target fitur dari EVA-CLIP, dan menempati posisi kompetitif terhadap *state-of-the-art* pada *benchmark* ImageNet, COCO, dan ADE20K.

G.3.2 Hiera

Hiera (*Hierarchical Vision Transformer without the Bells and Whistles*) (Ryali et al., 2023) menyederhanakan ViT dengan menghapus komponen yang dianggap tidak diperlukan, berlandaskan hipotesis bahwa banyak komponen pada *hierarchical Transformer* sebelumnya (Swin, MViT) menjadi tidak perlu apabila pelatihan dilakukan dengan paradigma MAE. Inovasi utamanya adalah penghapusan *positional embedding* eksplisit dan penggunaan *pooling attention* hierarkis yang terinspirasi MAE (He et al., 2022), sehingga menghasilkan representasi hierarkis menyerupai piramida fitur CNN namun tetap dalam kerangka *Transformer* murni. *Masked Autoencoder* (MAE) sendiri menutup sekitar 75% *patch* citra dan meminta mod-

el merekonstruksi piksel pada *patch* yang ditutup; asimetri antara *encoder* (melihat *patch* terlihat) dan *decoder* (merekonstruksi semua *patch*) memberikan efisiensi komputasi yang signifikan.

G.4 Encoder Teks Pembanding

G.4.1 BERT dan mBERT

BERT (Devlin et al., 2019) adalah *Transformer encoder* dua arah yang dilatih dengan *Masked Language Modeling* (MLM) dan *Next Sentence Prediction* (NSP), menggunakan tokenizer WordPiece dengan kosakata sekitar 30 ribu pada varian Inggris. *Multilingual BERT* (mBERT) adalah varian yang dilatih pada Wikipedia 104 bahasa secara serentak dengan kosakata WordPiece bersama berukuran sekitar 110 ribu, sehingga memperoleh kemampuan transfer lintas bahasa *zero-shot*.

G.4.2 IndoBERT

IndoBERT (Koto et al., 2020; Wilie et al., 2020) adalah keluarga *Transformer encoder* yang dilatih khusus pada korpus Bahasa Indonesia melalui dua jalur publikasi, yaitu IndoLEM/IndoBERT (Koto et al., 2020) dan IndoNLU/IndoBERT (Wilie et al., 2020). Arsitekturnya identik dengan BERT, namun dengan tokenizer dan korpus pra-latih spesifik Bahasa Indonesia, sehingga menghasilkan representasi yang lebih kaya pada idiom, ejaan, dan istilah lokal dibanding mBERT.

G.4.3 Cendol-mT5

Cendol (Cahyawijaya et al., 2024) adalah keluarga model bahasa berbasis instruksi yang dirilis oleh IndoNLP untuk Bahasa Indonesia dan bahasa daerah Nusantara. Sub-keluarga Cendol-mT5 dibangun di atas arsitektur mT5 (*encoder-decoder*); pada penggunaan sebagai *encoder* kalimat, hanya bagian *encoder* mT5 yang dimanfaatkan dengan *mean pooling* yang dinormalisasi. mT5 (Xue et al., 2021) adalah varian multibahasa T5 yang dilatih pada korpus mC4 mencakup 101 bahasa dengan tujuan *span corruption*.

G.4.4 BGE-M3

BGE-M3 (Chen et al., 2024) adalah *embedding model* multibahasa dari BAAI yang mendukung multibahasa (100+ bahasa), multifungsi (*dense*, *sparse*, dan *multi-vector retrieval*), serta multigranularitas (hingga 8.192 token). Arsitekturnya didasarkan pada XLM-RoBERTa *large* (sekitar 560 juta parameter) dengan strategi *self-knowledge distillation*. *Embedding* yang dihasilkan berdimensi 1024 dari token CLS dengan normalisasi L2.

G.5 Unified Modeling Language (UML): Jenis Diagram dan C4 Model

UML (Booch et al., 2005) adalah bahasa permodelan visual standar yang dibakukan oleh *Object Management Group* (OMG). Versi UML 2.x mendefinisikan empat belas jenis diagram yang dikelompokkan menjadi diagram struktur dan diagram perilaku.

Diagram struktur memodelkan elemen sistem yang relatif statis. Tiga jenis yang paling sering dipakai adalah Diagram Kelas (*class diagram*: kelas, atribut, metode, dan relasi), Diagram Komponen (*component diagram*: komponen yang dapat di-*deploy* beserta antarmukanya), dan Diagram Deployment (*deployment diagram*: pemetaan komponen ke simpul perangkat keras).

Diagram perilaku memodelkan dinamika sistem. Empat jenis yang relevan adalah Diagram *Use Case* (interaksi aktor dengan fungsionalitas), Diagram *Sequence* (urutan pesan antar objek pada sumbu waktu), Diagram Aktivitas (*activity diagram*: alur kerja dengan simpul keputusan dan paralel), dan Diagram *State Machine* (keadaan objek dan transisinya).

Sebagai pelengkap, Brown (2018) memperkenalkan *C4 Model* yang menyederhanakan dokumentasi arsitektur menjadi empat tingkat abstraksi, yaitu *Context*, *Container*, *Component*, dan *Code*. C4 sering dipadukan dengan UML dan lebih mudah dibaca oleh pemangku kepentingan non-teknis.

G.6 Docker dan Kontainerisasi: Konsep Rinci

Docker ([Merkel, 2014](#)) adalah *platform* kontainerisasi yang mengemas aplikasi beserta dependensinya ke dalam satu unit yang konsisten di berbagai lingkungan. Berbeda dari mesin virtual, kontainer berbagi *kernel* sistem operasi induk dan mengisolasi proses melalui *Linux namespaces* dan *cgroups*, sehingga ringan dan cepat dijalankan.

Lima konsep inti Docker adalah **Image** (*template read-only* berisi sistem berkas aplikasi), **Container** (instansi *image* yang berjalan), **Dockerfile** (skrip deklaratif langkah *build*), **Volume** (penyimpanan persisten), dan **Registry** (gudang *image* terpusat seperti Docker Hub). Sebuah *Dockerfile* umumnya memuat instruksi `FROM`, `WORKDIR`, `COPY/ADD`, `RUN`, `EXPOSE`, `ENV`, `HEALTHCHECK`, serta `CMD/ENTRYPOINT`.

Strategi *multi-stage build* memisahkan tahap *build* (yang membutuhkan kompilator) dari tahap *runtime* (yang hanya membutuhkan binari hasil), sehingga *image* akhir lebih ramping. Docker Compose mendefinisikan aplikasi multi-kontainer melalui berkas `YAML`, sehingga lingkungan (server inferensi, basis data, *reverse proxy*) dapat dijalankan dengan satu perintah `docker compose up`. Pada *pipeline machine learning*, Docker memberikan keuntungan berupa *reproducibility*, isolasi dependensi (PyTorch, ONNX Runtime, CUDA), *portability*, dan akselerasi GPU melalui `nvidia-container-toolkit`.

LAMPIRAN H

DESKRIPSI *USE CASE* SMARTCITYAPPS

Lampiran ini memuat *use case description* lengkap untuk setiap *use case* utama SmartCityApps. Ringkasan keterkaitannya dengan kebutuhan sistem disajikan pada Bab 3; rincian alur utama, alur alternatif, pra-kondisi, dan pasca-kondisi setiap *use case* dipaparkan pada tabel-tabel berikut.

Tabel H.1. *Use case description* untuk Mendaftar atau Masuk Akun.

Atribut	Deskripsi
Use Case	Mendaftar atau Masuk Akun
Aktor	Warga Pelapor
Deskripsi	Pelapor mendaftar akun baru atau masuk ke akun yang sudah ada untuk mengakses fitur pelaporan.
Pra-kondisi	Aplikasi terinstal pada perangkat Android.
Alur Utama	1. Pelapor membuka aplikasi. 2. Pelapor memilih opsi daftar atau masuk. 3. Pelapor memasukkan kredensial. 4. Sistem memvalidasi dan menyimpan sesi lokal.
Alur Alternatif	Bila kredensial salah, sistem menampilkan pesan kesalahan.
Pasca-kondisi	Pelapor masuk dan menuju layar utama.

Tabel H.2. *Use case description* untuk Mengambil Foto Laporan.

Atribut	Deskripsi
Use Case	Mengambil Foto Laporan
Aktor	Warga Pelapor
Deskripsi	Pelapor mengambil foto kejadian dengan kamera atau memilih dari galeri sebagai bukti visual laporan.
Pra-kondisi	Pelapor sudah masuk dan berada di layar buat laporan.
Alur Utama	1. Pelapor menekan tombol <i>Ambil Foto</i> . 2. Sistem meminta izin kamera/galeri. 3. Pelapor mengambil atau memilih foto. 4. Sistem menyimpan referensi foto pada <i>state</i> laporan.
Alur Alternatif	Pelapor membatalkan pengambilan foto.
Pasca-kondisi	Foto tampil pada <i>preview</i> dan siap dikirim.

Tabel H.3. *Use case description* untuk Menulis Narasi Laporan.

Atribut	Deskripsi
Use Case	Menulis Narasi Laporan
Aktor	Warga Pelapor
Deskripsi	Pelapor menulis narasi laporan dalam Bahasa Indonesia untuk melengkapi foto.
Pra-kondisi	Pelapor berada di layar buat laporan.
Alur Utama	1. Pelapor mengetuk medan teks <i>Narasi Laporan</i> . 2. Pelapor menulis deskripsi singkat. 3. Sistem menyimpan teks pada <i>state</i> laporan.
Alur Alternatif	Pelapor mengosongkan medan teks; sistem menampilkan peringatan.
Pasca-kondisi	Narasi siap dikirim bersama foto.

Tabel H.4. *Use case description* untuk Mengirim Laporan untuk Klasifikasi.

Atribut	Deskripsi
Use Case	Mengirim Laporan untuk Klasifikasi
Aktor	Warga Pelapor, Server FastAPI
Deskripsi	Aplikasi mengirim foto, narasi, latitude, dan longitude ke server FastAPI untuk diklasifikasikan ke salah satu dari sembilan instansi.
Pra-kondisi	Foto dan narasi sudah lengkap, koneksi jaringan tersedia.
Alur Utama	1. Pelapor menekan tombol <i>Klasifikasi</i> . 2. Aplikasi membentuk permintaan <i>multipart</i> berisi foto, narasi, latitude, dan longitude. 3. Permintaan dikirim ke <i>endpoint</i> klasifikasi pada server. 4. Server menjalankan fusi awal dan algoritma CatBoost+PGS. 5. Server menghitung confidence dan instansi terdekat. 6. Server mengirim respons JSON ke aplikasi.
Alur Alternatif	Bila server tidak tersedia, sistem menampilkan pesan kesalahan dan menawarkan menyimpan laporan sebagai <i>draft</i> .
Pasca-kondisi	Pelapor diarahkan ke layar tinjauan AI.

Tabel H.5. *Use case description* untuk Meninjau Hasil Klasifikasi AI.

Atribut	Deskripsi
Use Case	Meninjau Hasil Klasifikasi AI
Aktor	Warga Pelapor
Deskripsi	Pelapor melihat instansi yang diprediksi beserta tingkat keyakinan dan rekomendasi instansi.
Pra-kondisi	Respons klasifikasi sudah diterima.
Alur Utama	1. Sistem menampilkan instansi prediksi, <i>confidence</i> , dan tiga prediksi teratas. 2. Sistem menampilkan instansi tujuan terdekat beserta estimasi jarak. 3. Pelapor membaca informasi.
Alur Alternatif	Bila <i>confidence</i> di bawah ambang, sistem menyarankan tinjauan manual.
Pasca-kondisi	Pelapor dapat menerima atau mengoreksi hasil.

Tabel H.6. *Use case description* untuk Mengoreksi Kategori Manual.

Atribut	Deskripsi
Use Case	Mengoreksi Kategori Manual
Aktor	Warga Pelapor
Deskripsi	Pelapor mengganti instansi yang diprediksi AI bila menurutnya salah.
Pra-kondisi	Pelapor berada di layar tinjauan AI.
Alur Utama	1. Pelapor menekan tombol <i>Edit Kategori</i> . 2. Sistem menampilkan daftar sembilan kategori. 3. Pelapor memilih kategori baru. 4. Sistem memperbarui <i>state</i> laporan.
Alur Alternatif	Pelapor membatalkan koreksi.
Pasca-kondisi	Kategori laporan diperbarui sesuai pilihan pelapor.

Tabel H.7. *Use case description* untuk Menyimpan Laporan ke Riwayat.

Atribut	Deskripsi
Use Case	Menyimpan Laporan ke Riwayat
Aktor	Warga Pelapor
Deskripsi	Pelapor menyimpan laporan beserta hasil klasifikasi ke basis data Supabase.
Pra-kondisi	Hasil klasifikasi (otomatis atau hasil koreksi) tersedia dan pelapor terotentikasi.
Alur Utama	1. Pelapor menekan tombol <i>Simpan</i> . 2. Aplikasi mengunggah berkas gambar ke Supabase Storage. 3. Aplikasi menulis baris baru ke tabel <code>reports</code> pada basis data Supabase. 4. Aplikasi menyisipkan baris audit ke tabel <code>report_history</code> . 5. Sistem menampilkan konfirmasi.
Alur Alternatif	Bila penyimpanan gagal, sistem menampilkan pesan kesalahan.
Pasca-kondisi	Laporan tersimpan dan terlihat pada layar Riwayat.

Tabel H.8. *Use case description* untuk Melihat Riwayat dan Peta Laporan.

Atribut	Deskripsi
Use Case	Melihat Riwayat dan Peta Laporan
Aktor	Warga Pelapor
Deskripsi	Pelapor melihat daftar laporan yang sudah disimpan, beserta lokasi pada peta.
Pra-kondisi	Sudah ada minimal satu laporan tersimpan.
Alur Utama	1. Pelapor membuka <i>tab</i> Riwayat atau Peta. 2. Sistem membaca tabel laporan dari basis data. 3. Sistem menampilkan daftar laporan atau <i>pin</i> lokasi pada peta.
Alur Alternatif	Bila tidak ada laporan, sistem menampilkan pesan <i>empty state</i> .
Pasca-kondisi	Pelapor dapat memilih laporan untuk melihat rincian.